

Lecture 6 — Clean Code & Design Patterns

Lecture id: L06 **Source decks:** `CleanCode.pdf` (81 slides — a slide-per-topic walkthrough of Robert C. Martin's *Clean Code*; the title text is embedded, the bodies are rendered code-screenshots/handwritten labels which I read page-by-page as images), `DesignPrinciplesAndPatterns.pdf` (34 pages — Robert C. Martin's chapter "Design Principles and Design Patterns", © 2000, ObjectMentor; full machine-readable text). **Labs:** None. **Process phase(s):** Actualization (code quality) · cross-cutting (touches Prefactoring, Postfactoring/refactoring, Verification, and the whole change process — clean code and good OO structure are what make *every* phase cheaper). **Citation key:** (`CleanCode` p.X) and (`DesignPrinciplesAndPatterns` p.X) cite the exact slide/page I read. Reading keys: [Martin] / [MC09] = Robert C. Martin & James Coplien, *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall (2009) — the deck's own reference slide (`CleanCode` p.81); [GHJV94] = Gamma, Helm, Johnson, Vlissides ("Gang of Four"), *Design Patterns: Elements of Reusable Object-Oriented Software* (1994), cited inside the deck as [GHJV94] (`DesignPrinciplesAndPatterns` p.28, p.34); [Raj13] = Václav Rajlich, *Software Engineering: The Current Practice* (course textbook, for the change-process framing); [Fowler99] = Martin Fowler, *Refactoring* (1999, for the refactoring connection). Martin's own principle papers [OCP97], [LSP97], [DIP97], [ISP97], [Granularity97], [Stability97] are cited on `DesignPrinciplesAndPatterns` p.34. **Grounding note:** This is a cross-cutting code-quality lecture with **no JHotDraw deck of its own** and **no labs**. Both source decks are by Robert C. Martin. The `CleanCode` deck is image-heavy (each content slide is a screenshot of code or a handwritten section label); I rendered all 81 pages and read every one, so the rules and code examples below are transcribed from the actual slides, not recalled from the book. The `DesignPrinciplesAndPatterns` deck extracted as clean text and I read all 34 pages. Every non-obvious claim is cited to the page I read. Where I connect a rule or pattern to JHotDraw or to the course's eight-step change process, that is **inference / connective tissue** flagged as such, because neither deck names JHotDraw.

Overview

Lecture 6 is the course's **code-quality** lecture. It steps *out* of the linear eight-step change process — **Initiation** → **Concept Location** → **Impact Analysis** → **Prefactoring** → **Actualization** → **Postfactoring** → **Conclusion** → **Verification** — and asks a different, orthogonal question: *given that you are about to edit code, what makes that code good, and what structural principles keep a system changeable over its whole lifetime?* It answers at two altitudes:

1. **Clean Code (micro level), deck `CleanCode`**. Robert C. Martin's rules for the *small* things: how to name (`Meaningful Names`), how to write functions (`Functions`), when comments help vs. hurt (`Comments`), how to lay code out (`Formatting`), how to design objects vs. data structures, how to handle errors, how to write unit tests, and how to organize classes — culminating in the "rules of simple design" (`Emergence`). The deck opens with **two reasons** you should care: "1. You are a programmer" and "2. You want to be a better programmer" (`CleanCode` p.2), and the governing ethic, the **Boy Scout Rule**: "You should always leave the code cleaner than you found it" (`CleanCode` p.3).
2. **Design Principles & Patterns (macro level), deck `DesignPrinciplesAndPatterns`**. Martin's classic chapter on *why software rots* and the principles/patterns that arrest the rot. It defines the four

symptoms of rotting design (rigidity, fragility, immobility, viscosity) (DesignPrinciplesAndPatterns p.2–3), traces them all to **bad dependencies** (p.4), then presents the **OO class-design principles** (OCP, LSP, DIP, ISP), the **package cohesion principles** (REP, CCP, CRP), the **package coupling principles** (ADP, SDP, SAP) with their stability/abstractness metrics, and finally a tour of five **design patterns** (Abstract Server, Adapter, Observer, Bridge, Abstract Factory) (p.28–32), pointing readers to the Gang of Four book [GHJV94] for the rest.

Why it matters for software maintenance. Maintenance is change, and change is what makes software rot. The deck's opening narrative is explicitly a maintenance story: a clean design "starts to rot," accumulates "ugly festering sores and boils," until "the sheer effort required to make even the simplest of changes ... becomes so high that the engineers ... cry for a redesign" (DesignPrinciplesAndPatterns p.1–2). Clean code and good OO principles are the antidote: they reduce the cost of **Impact Analysis** (fewer surprise dependencies), shrink the **estimated impact set** (changes don't cascade), and make **Verification** cheaper (small, testable units). This lecture is the "how to write the code well" companion to the "how to find and propagate the change" mechanics taught earlier.

Scope boundaries — what is *not* in these decks

To calibrate expectations (and avoid revising phantom material), the explicit limits of the two source decks:

- **No labs and no lab tooling.** LO6 ships no lab handout, so none of the course's lab stack (JHotDraw exercises, Featureous, JRipples, Maven, GitHub workflows, AssertJ, JUnit, JGiven) appears in these decks. The unit-test slides (CleanCode p.74–77) state TDD's laws and F.I.R.S.T. in tool-neutral terms; the JUnit-style `assertEquals` / `assertTrue` calls on CleanCode p.30–31 and p.36 appear only as *naming/comment examples*, not as framework instruction.
- **No refactoring catalogue.** The decks motivate refactoring (Boy Scout Rule, CleanCode p.3; postfactoring connections flagged as inference throughout) and use individual moves informally (extract method/variable on CleanCode p.45, p.67; replace switch with polymorphism on p.25–26), but the systematic catalogue of named refactoring types belongs to the course's refactoring lecture, not to LO6.
- **Only five patterns, deliberately.** The chapter is explicit that "the topic of Design Patterns cannot be adequately covered in a single chapter of a single book" and covers only the five patterns "you will come across while reading through the case studies later in the book," pointing to the GoF book (cited there as [GOF96]) for the rest (DesignPrinciplesAndPatterns p.28). Singleton, Strategy, Composite, Decorator, Visitor, etc. are *not* developed in the deck — any exam treatment of those leans on the GoF reference or the JHotDraw lectures, not on LO6's sources.
- **Architecture above module level is out of scope.** The chapter brackets away the higher tiers — application-shape "architecture patterns" ([Shaw96]) and purpose-specific architecture — and works strictly at the level of "design patterns, packages, components, and classes" (DesignPrinciplesAndPatterns p.1).
- **The CleanCode deck is examples-first.** Its slides carry code screenshots and one-line handwritten labels, not definitional prose; the canonical definitions live in the book it references (Martin & Coplien 2009, CleanCode p.81). Where this guide gives a definition more precise than a slide label, the precision comes from the slide's *code example* itself.

Complete deck maps — every page of both decks, indexed

For navigation and recall: one row per page/slide, with title and content. (These maps double as a coverage checklist — every row is treated somewhere in this guide.)

CleanCode.pdf — all 81 slides:

Page	Slide title	Content
1	Clean Code (title)	Title slide, lecturer's name, code-backdrop imagery
2	Two Reasons	"1. You are a programmer / 2. You want to be a better programmer"; book cover
3	The Boy Scout Rule	"You should always leave the code cleaner than you found it"; Uncle Bob portrait
4	Elegance	Stroustrup: "elegant and efficient ... does one thing well"
5	Simple, direct, prose	Booch: "simple and direct ... reads like well-written prose"
6	Literate	Dave Thomas: "can be read ... should be literate"
7	Care	Feathers: "looks like it was written by someone who cares"
8	Small, expressive, simple	Jeffries: "reduced duplication, high expressiveness, early building of simple abstractions"
9	What you expected	Cunningham: "each routine ... pretty much what you expected"
10	(untitled cartoon)	"The only valid measurement of code quality: WTFs/minute" — good-code vs bad-code review doors
11	Meaningful Names — Use Intention-Revealing Names	<code>getThem() / x[0]==4</code> → <code>getFlaggedCells() / cell.isFlagged()</code>
12	Avoid Disinformation; Make Meaningful Distinctions	<code>l / 0</code> vs <code>1 / 0</code> confusion; <code>copyChars(char a1[], char a2[])</code>
13	Use Pronounceable Names	<code>DtaRcrd102 / genymdhms / pszqint</code> → <code>Customer / generationTimestamp / recordId</code>
14	Use Searchable Names	<code>magic 34 / 4 / 5</code> loop → <code>WORK_DAYS_PER_WEEK, NUMBER_OF_TASKS, taskEstimate</code>
15	Member Prefixes (Avoid Encodings)	<code>m_dsc / setName</code> → <code>description / setDescription</code>
16	Hungarian Notation (Avoid Encodings)	<code>PhoneNumber phoneString</code> ; "name not changed when type changed!" → <code>phone</code>
17	Avoid Mental Mapping	<code>a / b</code> nested loops → conventional <code>i / j</code>
18	Class Names	bad <code>Manager, Processor, Data, Info</code> ; good <code>Customer, WikiPage, Account, AddressParser</code> ; not a verb
19	Method Names	<code>postPayment, deletePage, save</code> ; <code>get / set / is</code> ; <code>Complex.fromRealNumber(23.0)</code>
20	Pick One Word per Concept; Don't Pun	<code>fetch/retrieve/get</code> equivalent; <code>controller/manager/driver</code> confusing
21	Use Solution Domain Names; Add Meaningful Context	<code>AccountVisitor, JobQueue</code> ; <code>addrFirstName...</code> → <code>class Address</code>
22	Don't Add Gratuitous Context	<code>Address</code> fine class name; <code>PostalAddress, MAC, URI</code>
23	Functions — Small; Do One Thing	"< 150 characters per line; < 20 lines"; "DO ONE THING ... DO IT ONLY"
24	One Level of Abstraction; Top to Bottom	<code>getHtml() / PathParser.render / .append("\n")</code> ; the Stepdown Rule
25	Switch Statements (bad)	<code>payAmount()</code> switching on <code>getType()</code> with <code>_monthlySalary</code> etc.
26	Switch Statements (fix)	<code>abstract EmployeeType.payAmount(Employee)</code> ; <code>Salesman, Manager</code> overrides

Page	Slide title	Content
27	Use Descriptive Names; Function Arguments	<code>testableHtml ⇒ includeSetupAndTeardownPages</code> ; "ideal number ... is zero"
28	Common Monadic Forms	question / operate-and-return / transformation-as-return-value / event
29	Flag Arguments	<code>render(true) → renderForSuite() / renderForSingleTest()</code>
30	Functions (Bad) — Dyadic; Triads	<code>writeField(outputStream, name); Point(0,0)</code> ok; <code>assertEquals</code> problematic
31	Argument Objects; Verbs and Keywords	<code>makeCircle(Point center, double radius);</code> <code>assertExpectedEqualsActual</code>
32	Command Query Separation	<code>set()</code> verb-or-adjective ambiguity; <code>attributeExists</code> + <code>setAttribute</code>
33	DRY; Structured Programming	"root of all evil"; Dijkstra's one-entry-one-exit, relaxed for small functions
34	Comments	"don't comment bad code, rewrite it!"; <code>isEligibleForFullBenefits()</code>
35	Comments (Good)	Legal (Osoco/GPL header); Informative (<code>responderInstance</code> , <code>timeMatcher</code> regex format)
36	Comments (Good)	Explanation of Intent (25,000-thread race loop); Clarification (<code>compareTo</code> → <code>// a < b</code>)
37	Comments (Good)	Amplification (the <code>trim()</code> warning); Javadocs in Public APIs
38	Comments (Bad)	Mumbling — the properties-file <code>catch</code> comment
39	Comments (Bad)	Redundant — <code>waitForClose</code> header
40	Comments (Bad)	Redundant — <code>backgroundProcessorDelay</code> field Javadocs
41	Comments (Bad)	Mandated — <code>addCD @param</code> boilerplate
42	Comments (Bad)	Journal — the 2001–2002 changelog
43	Comments (Bad)	Noise — <code>/** Default constructor. */</code> , <code>dayOfMonth</code>
44	Comments (Bad)	Scary Noise — <code>info</code> documented as "The version"
45	Comments (Bad)	Use a function/variable instead — <code>moduleDependees</code> refactor
46	Comments (Bad)	Position Markers (<code>// Actions</code> <code>////</code>); Closing Brace (<code>}</code> <code>//while</code>)
47	Comments (Bad)	Attributions (<code>/* Added by Rick */</code>); Commented-Out Code
48	Comments (Bad)	HTML Comments — entity-escaped Ant taskdef Javadoc
49	Comments (Bad)	Nonlocal Information — <code>setFinessePort</code> "Defaults to 8082"
50	Comments (Bad)	Too Much Information — RFC 2045 Base64 excerpt
51	Comments (Bad)	Inobvious Connection (<code>pngBytes</code> + 200); Function Headers
52	Comments (Bad)	Javadocs in Nonpublic Code
53	Formatting	Purpose = communication; Newspaper Metaphor; Vertical Openness
54	Formatting	Vertical Density — reporter-listener fields split by noise comments
55	Formatting	Vertical Distance — variables near use; instance vars on top; caller above callee; conceptual affinity
56	Formatting	Horizontal Openness/Density — <code>measureLine</code> , <code>root2</code> precedence spacing
57	Formatting	Horizontal Alignment — aligned <code>FitNesseExpediter</code> columns (discouraged)

Page	Slide title	Content
58	Formatting	Horizontal Alignment — the plain, unaligned version (preferred)
59	Formatting	Breaking Indentation — collapsed vs expanded <code>CommentWidget</code>
60	Formatting	Team Rules — "the team rules" win
61	Objects and Data Structures	Data Abstraction — concrete vs abstract <code>Point</code>
62	Objects and Data Structures	Data Abstraction — concrete vs abstract <code>Vehicle</code> (percent fuel)
63	Objects and Data Structures	Data/Object Anti-Symmetry
64	Objects and Data Structures	Law of Demeter; Train Wrecks; field-reaching worst form
65	Error Handling	Prefer Exceptions — the nested <code>E_OK</code> ladder
66	Error Handling	Prefer Exceptions — the <code>try/catch</code> rewrite
67	Error Handling	Extract Try/Catch Blocks — <code>delete / deletePageAndAllReferences / logError</code>
68	Error Handling	Error Handling Is One Thing — <code>try</code> first word, nothing after catch/finally
69	Error Handling	Define the Normal Flow — per-diem <code>catch</code> version
70	Error Handling	Define the Normal Flow — Special-Case caller version
71	Error Handling	Don't Return Null — <code>if (employees != null)</code> caller
72	Error Handling	Don't Return Null — <code>Collections.emptyList()</code> provider fix
73	Error Handling	Don't Pass Null — <code>xProjection</code> crash vs defensive clutter
74	Unit Tests	The Three Laws of TDD ("not compiling is failing")
75	Unit Tests	Keeping Tests Clean; Clean Tests ("readability ×3")
76	Unit Tests	One Assert per Test; Single Concept per Test
77	Unit Tests	F.I.R.S.T.
78	Classes	Class Organization — constants → statics → instance vars → public functions → private utilities after their caller
79	Classes	Should Be Small (responsibilities); SRP; Cohesion
80	Emergence	Simple Design Rules 1–4
81	Reference	Martin & Coplien, <i>Clean Code</i> , Prentice Hall (2009)

DesignPrinciplesAndPatterns.pdf — all 34 pages:

Page	Content
1	Title; architecture is multitiered ([Shaw96] , design-pattern level is this chapter's scope); the rot story begins ("vital image ... clean, elegant, and compelling")
2	Why redesigns rarely succeed (moving target); Symptoms of Rotting Design; Rigidity (roach motel, official rigidity); Fragility (probability → 1)
3	Fragility's trust collapse; Immobility (rewritten instead of reused); Viscosity (design + environment); Changing Requirements begins
4	"Designs at fault"; Dependency Management and dependency firewalls; OCP statement (Meyer)

Page	Content
5	OCP techniques are all abstraction-based; Dynamic Polymorphism; Listing 2-1 (modem structs + if/else <code>LogOn</code>); recompilation cost of the shared enum
6	Scattered selection statements; the local-optimization trap (<code>SendErnie</code> / <code>SendHayes</code>); Figure 2-13 (<code>Modem</code> «interface»); Listing 2-2 begins
7	Listing 2-2 (pure-virtual <code>Modem</code> , one-line <code>LogOn</code>); Listing 2-3 static polymorphism (template); Architectural Goals of the OCP ("only adding new code")
8	LSP statement (Liskov, DbC/Meyer); Figure 2-14 schema; Listing 2-4 (<code>User(d)</code>)
9	The Circle/Ellipse Dilemma; Figures 2-15/2-16 (<code>Ellipse</code> with <code>itsFocusA/B</code> , <code>itsMajorAxis</code> and 8 operations); Circle's extra data
10	Listing 2-5 (<code>SetFoci</code> keeps foci coincident); "Clients Ruin Everything"; the <code>f(Ellipse&)</code> contract fragment; implicit postcondition of <code>SetFoci</code>
11	Eiffel and explicit contracts; precondition/postcondition definitions; the two-clause substitutability rule; "expect no more, provide no less"; Repercussions; Listing 2-6 begins
12	Listing 2-6 is an OCP violation → "violations of LSP are latent violations of OCP"; DIP statement; COM/CORBA/EJB; procedural dependency structure
13	"Draconian" caveat; concrete volatile vs abstract stable; "hinge points"; Figures 2-17/2-18 (procedural vs OO dependency structures)
14	COM enforces DIP; Mitigating Forces (<code>string.h</code> , UNICODE caution, non-volatility ≠ substitutability); Object Creation → ABSTRACTFACTORY; ISP statement
15	Figure 2-19 (fat service); segregated technique; recompile/redeploy blast radius; "What does Client Specific Mean?"; Changing Interfaces
16	The <code>dynamic_cast</code> query idiom; over-segregation warning; Principles of Package Architecture intro; Figure 2-20 (segregated interfaces)
17	REP (release system, version support); CCP (minimize packages per release, prescience); CRP statement
18	CRP's OS-vendor analogy; Tension between cohesion principles (mutually exclusive; CCP→large, CRP→small; jitter over time); Package Coupling intro; ADP statement
19	Packages focus manpower; Figure 2-21 (GUI/Comm/ModemControl/Protocol/CommError/Analysis/Database); cheap <code>Protocol</code> release ("Yuk" architecture note)
20	A Cycle Creeps In (<code>CommError</code> → <code>GUI</code> , Figure 2-22); the six-package build disaster; watch-and-break governance; Breaking a Cycle intro
21	Figure 2-23 (extract <code>MessageManager</code>); package jitter; Figure 2-24 (invert <code>Y→B</code> with interface <code>BY</code>)
22	Interface-placement rule (in the <i>user's</i> package); SDP statement; the penny analogy begins
23	Stability = work to change; incoming dependencies dominate; Figure 2-25 (X: responsible, independent); Figure 2-26 (Y: irresponsible, dependent); <code>Ca</code> defined
24	<code>Ce</code> defined; $I = Ce / (Ca + Ce)$; SDP rephrased ("lower I than yours"); instability is desirable for flexible packages; Figure 2-27 (Stable→Flexible violation); SAP statement
25	Instable-top/stable-bottom picture; the hard-to-change dilemma; the OCP loophole (stable+abstract = extensible); SAP = DIP restated; abstractness metrics intro
26	<code>Nc</code> , <code>Na</code> (abstract class = ≥ 1 pure interface, non-instantiable); $A = Na / Nc$; "I should increase as A decreases"; Figure 2-28 (A-vs-I graph, two black dots, Main Sequence, both zones)

Page	Content
27	Zone of Uselessness; Zone of Pain; Main Sequence rationale (DIP-conformance); $D = (A+I-1)/\sqrt{2}$, $D' = \lfloor A+I-1 \rfloor$; metrics-imperfection caveat; Figure 2-29
28	Patterns of OO Architecture; pattern definition ("well-worn and known good solution"); [GOF96] pointer; Abstract Server + Figure 2-30
29	Adapter + Figure 2-31 ("translates and then delegates"); Observer problem (sensor/meter); structure begins
30	Observer dynamics (Check→Notify→Update→GetValue); Figures 2-32/2-33 ; Bridge problem (tight base/derived coupling)
31	MusicSynthesizer example (PlayMidi / EmitVoice welded); BRIDGE solution described; Figure 2-34 (badly coupled hierarchy)
32	Abstract Factory (creation quarantined to one place; GtheFactory ; only main knows ModemFactory_I); Conclusion begins (four-adjective definition); Figure 2-35 (Bridge-decoupled hierarchy)
33	Conclusion ("a little knowledge is a dangerous thing"); Bibliography begins ([Shaw96] , [GOF96] , [OOSC98]); Figure 2-36 (Abstract Factory)
34	Bibliography: [OCP97] , [LSP97] , [DIP97] , [ISP97] , [Granularity97] , [Stability97] , [Liksov88] , [Martin99]

Learning Objectives

After this lecture you should be able to:

1. State the **Boy Scout Rule** and the "two reasons" framing for caring about clean code (CleanCode p.2–3).
2. Apply the **Meaningful Names** rules: intention-revealing, no disinformation, meaningful distinctions, pronounceable, searchable, no encodings (no member prefixes, no Hungarian notation), no mental mapping, noun class names / verb method names, one word per concept, no puns, solution-domain names, meaningful (not gratuitous) context (CleanCode p.11–22).
3. Apply the **Functions** rules: small (< 20 lines), do one thing, one level of abstraction per function, the Stepdown Rule, replace switch statements with polymorphism, descriptive names, minimize arguments (niladic > monadic > dyadic > triadic), avoid flag arguments, Command/Query Separation, DRY, and disciplined structured programming (CleanCode p.23–33).
4. Distinguish **good comments** (legal, informative, intent, clarification, amplification, warning, TODO, public-API Javadoc) from **bad comments** (mumbling, redundant, mandated, journal, noise, position markers, closing-brace, attributions, commented-out code, HTML, nonlocal, too-much-info, non-obvious, too-much) and apply the rule "Don't comment bad code — rewrite it" and "explain yourself in code" (CleanCode p.34–52).
5. Apply **Formatting** rules: formatting is communication; the newspaper metaphor; vertical openness / density / distance; horizontal openness, alignment and indentation; and "team rules win" (CleanCode p.53–60).
6. Explain the **objects-vs-data-structures** distinction, data/object **anti-symmetry**, and the **Law of Demeter** ("don't talk to strangers"; avoid train wrecks) (CleanCode p.61–64).
7. Apply **Error Handling** rules: prefer exceptions to error codes, extract try/catch bodies, "error handling is one thing," define the normal flow, **don't return null**, **don't pass null** (CleanCode p.65–73).
8. State the **Three Laws of TDD**, why test code matters as much as production code, "one assert / one concept per test," and the **F.I.R.S.T.** properties (Fast, Independent, Repeatable, Self-validating, Timely)

(CleanCode p.74–77).

9. Apply **Classes** rules: standard class organization, classes should be small, the **Single Responsibility Principle (SRP)**, and cohesion (CleanCode p.78–79).
10. State the four **Rules of Simple Design / Emergence**: (1) runs all the tests, (2) no duplication, (3) expressive, (4) minimal classes and methods (CleanCode p.80).
11. Name and explain the four **symptoms of rotting design**: rigidity, fragility, immobility, viscosity, and trace them all to bad dependencies (DesignPrinciplesAndPatterns p.2–4).
12. State and apply the **OO class-design principles**: **OCF** (open for extension, closed for modification), **LSP** (subtypes substitutable, expressed via Design-by-Contract pre/postconditions), **DIP** (depend on abstractions, not concretions), **ISP** (many client-specific interfaces beat one fat interface) (DesignPrinciplesAndPatterns p.4–16).
13. State the three **package cohesion principles** (REP, CCP, CRP) and the tension between them, and the three **package coupling principles** (ADP, SDP, SAP) including the stability metric $I = Ce / (Ca + Ce)$, abstractness $A = Na / Nc$, the **Main Sequence**, the Zone of Pain and Zone of Uselessness (DesignPrinciplesAndPatterns p.16–27).
14. Describe the intent and structure of the five patterns covered — **Abstract Server, Adapter, Observer, Bridge, Abstract Factory** — and relate each to the principle it serves (DesignPrinciplesAndPatterns p.28–32).
15. Connect all of the above to the course change process and (by inference) to JHotDraw as a textbook GoF-pattern case study.
16. Attribute each clean-code definition to its author — Stroustrup, Booch, Dave Thomas, Feathers, Jeffries, Cunningham — and state the WTFs/minute criterion (CleanCode p.4–10).
17. Given a small package-dependency diagram, compute Ca , Ce , I , Na , Nc , A , and D' , place the package on the A-vs-I graph, and prescribe the refactoring (DesignPrinciplesAndPatterns p.23–27).
18. Retell the CommError→GUI cycle story and apply *both* cycle-breaking techniques — extracting a shared package and inverting a dependency with a client-side interface — including the interface-placement rule (DesignPrinciplesAndPatterns p.19–22).
19. Reproduce the deck-exact wording of the Three Laws of TDD (including "not compiling is failing"), the class-organization order, and the four Simple Design Rules (CleanCode p.74, 78, 80).

Key Concepts

Bulk section. One ### per concept. Clean-code rules first (in deck order), then the macro design principles, then each named design pattern with intent + structure. Every non-obvious claim is cited inline.

The two reasons & the Boy Scout Rule (the ethic)

What it is. This is the deck's framing of *why* clean code is a professional obligation, not a matter of taste. Martin gives **two reasons** to care: "1. You are a programmer" and "2. You want to be a better programmer" (CleanCode p.2) — i.e. writing code is your job, and improvement is the craftsman's duty. The governing rule of conduct is the **Boy Scout Rule**: "*You should always leave the code cleaner than you found it*" (CleanCode p.3), borrowed from the scouting maxim "leave the campground cleaner than you found it." The opening slides also gesture at what clean code *feels* like via a series of one-word ideals attributed to master

programmers: **elegance**, "simple, direct, prose", **literate**, **care**, "small, expressive, simple", and "what you expected" (CleanCode p.4–9).

What it's used for / why it matters. The Boy Scout Rule is the practical engine of *continuous* code improvement. Most code rots not because anyone decides to make it bad, but because everyone who touches it leaves it a tiny bit worse. The rule reverses that arrow: if every developer makes one small improvement on every visit, quality trends *upward* over time at near-zero marginal cost, with no special "clean-up project" ever needed. It also sidesteps the political problem of justifying refactoring — you don't need a manager's permission to rename a variable in code you were already editing.

When & how it's applied. Concretely: you are fixing a bug in a 200-line method. You don't rewrite the whole method, but you *do* rename the two cryptically-named locals you had to decipher, extract the one duplicated block you noticed, and delete a stale comment — then commit the fix. The campground is now cleaner than you found it.

Process anchor (inference): The Boy Scout Rule is the micro-level justification for **Postfactoring** (refactoring after a change). Even if your change only touches a few classes, you should improve any messy code you pass through. This is the same incremental-improvement ethic Fowler [Fowler99] and the course's refactoring lecture promote, expressed as a personal habit rather than a formal phase.

The six "masters" and their definitions of clean code (CleanCode p.4–9)

What it is. The ideals named in the Boy Scout section above are not anonymous: each of the deck's opening slides pairs its one-line ideal (the slide title) with a hand-drawn portrait and a handwritten quotation from a *named* authority of the field. (The Boy Scout Rule slide itself carries a hand-drawn portrait labelled "**Robert C. Martin / Uncle Bob**" — the deck's author quoting himself, CleanCode p.3.) Knowing *who said what* is cheap exam currency, so here is the complete slide-by-slide mapping:

Slide title (the ideal)	Authority pictured on the slide	Quotation (as handwritten on the slide)	Source
Elegance	Bjarne Stroustrup	"I like my code to be elegant and efficient. ... Clean code does one thing well."	CleanCode p.4
Simple, direct, prose	Grady Booch	"Clean code is simple and direct. Clean code reads like well-written prose."	CleanCode p.5
Literate	Dave Thomas	"Clean code can be read. ... Clean code should be literate."	CleanCode p.6
Care	Michael Feathers	"Clean code always looks like it was written by someone who cares."	CleanCode p.7
Small, expressive, simple	Ron Jeffries	"Reduced duplication, high expressiveness, and early building of simple abstractions."	CleanCode p.8
What you expected	Ward Cunningham	"You know you are working on clean code when each routine you read turns out to be pretty much what you expected."	CleanCode p.9

(Identifying the figures' wider roles — Stroustrup as the creator of C++, Booch as a UML co-creator, Thomas as a *Pragmatic Programmer* author, Feathers as the author of *Working Effectively with Legacy Code*, Jeffries as an XP co-founder, Cunningham as the inventor of the wiki — is **connective tissue**; the slides show only the names, portraits, and quotes.)

What it's used for / why it matters. The six quotes triangulate the same target from six different angles, and each contributes a *testable criterion* for "clean": Stroustrup contributes **elegance + efficiency** and the seed of "do one thing well" (which becomes the Functions rule on CleanCode p.23); Booch contributes **readability as prose** (which becomes the Formatting/newspaper material, CleanCode p.53); Thomas contributes **literacy** — code is written primarily to be *read*; Feathers contributes the human signal of **care** — clean code *looks cared for*, which is exactly what the Boy Scout Rule produces over time; Jeffries contributes the three *measurable* properties — **reduced duplication, high expressiveness, early building of simple abstractions** — which anticipate, almost word for word, rules 2–4 of the four Rules of Simple Design (CleanCode p.80); and Cunningham contributes the **no-surprises criterion** — every routine turns out to be "pretty much what you expected," i.e. the reader's predictions are never violated. Together they make "clean" an operational property, not a matter of taste.

When & how it's applied. Use the quotes as review heuristics: when reviewing a change, ask Cunningham's question ("was every routine what I expected?"), Feathers' question ("does this look like someone cared?"), and Jeffries' three checks (duplication? expressiveness? simple abstractions built early?). A "no" on any of them names the cleanup to do before committing.

The WTFs/minute cartoon — "the only valid measurement of code quality" (CleanCode p.10)

What it is. Slide 10 is a single hand-drawn cartoon whose caption reads "**The only valid measurement of code quality: WTFs/minute**" (CleanCode p.10). It shows two closed doors, one labelled "**Good code**" and one labelled "**Bad code**", each with a *code review* in progress behind it. From the good-code door escape a few, measured "WTF?"s; from the bad-code door pours a dense storm of them ("WTF?!", "WTF is this?!", and so on).

What it's used for / why it matters. The joke carries two serious points. First, code quality is defined **by the reader, not the writer** — it is measured at *review time*, by the rate of incomprehension events per minute, not by any property the author can self-certify. This is the same reader-first stance as the Formatting chapter ("the purpose of formatting is communication," CleanCode p.53) and Cunningham's "what you expected" criterion (CleanCode p.9). Second, note that even the *good* code door emits some WTFs — the difference between good and bad code is the **rate**, not the total absence, of surprises. No code is perfectly self-evident; clean code just minimizes the reader's stumbling frequency.

When & how it's applied. Treat every "WTF moment" during code reading or review as a *signal with a target*: each one marks a spot where a rename, an extracted function, or a deleted bad comment would lower the codebase's WTFs/minute for the next reader. It is the lecture's most compact summary of why all the following rules exist.

Meaningful Names — use intention-revealing names

What it is. An intention-revealing name is one that answers, on its own, *why a thing exists, what it does, and how it is used* — so that no comment or extra lookup is needed to understand it. The deck contrasts a meaningless `getThem()` returning `List<int[]>` with `int[] x` and the magic `if (x[0] == 4)` against the self-documenting `getFlaggedCells()` over a `gameBoard` of `Cell` objects with `cell.isFlagged()` (CleanCode p.11). The two snippets are logically identical; only the names changed, yet the second version explains itself.

What it's used for / why it matters. Reading code is the dominant activity in maintenance — we read code far more often than we write it — so every second a reader spends decoding `x[0] == 4` is paid many times over. Intention-revealing names collapse that decoding cost to zero: the *what* lives in the names, not in the reader's head or in fragile comments that drift out of date. This is the single highest-leverage clean-code habit because it makes Concept Location (finding the code that implements a concept) almost free.

When & how it's applied. Whenever a name needs a comment to explain it, the name has failed — replace it. *Bad:* `int d; // elapsed time in days`. *Good:* `int elapsedTimeInDays;`. *Bad:* the minesweeper `getThem() / x[0] == 4`. *Good:* `getFlaggedCells() / cell.isFlagged() / Cell.FLAGGED`. The discipline is: name the variable, function, or class after the domain concept it represents, and keep renaming as your understanding sharpens.

Meaningful Names — avoid disinformation, make meaningful distinctions

What it is. Two related rules. **Avoid disinformation** means never pick a name that actively misleads — a name that suggests something the code does not do, or that is visually confusable. The deck shows the trap of single-letter names that look like other characters — `int a = l;` next to `if (0 == l)` and `a = 01; ... l = 01;` where lowercase `l` and uppercase `0` are indistinguishable from `1` and `0` (CleanCode p.12). (Other disinformation: calling something `accountList` when it is actually a `Map`, or `hp / aix / sco` which read as platform names.) **Make meaningful distinctions** means that when two things differ, their names must encode *how* they differ — noise-word distinctions (`a1 / a2`, `data / info`, `theMessage / message`) carry no information. The deck's `copyChars(char a1[], char a2[])` should name the arrays `source` and `destination`, not `a1 / a2` (CleanCode p.12).

What it's used for / why it matters. Disinformation is worse than vagueness: a vague name slows the reader, but a *wrong* name plants a false belief that causes bugs. Meaningful distinctions matter because the reader must be able to tell two variables apart *and know why both exist*; if `a1` and `a2` differ only by a digit, the reader can't choose between them or spot when the wrong one is used.

When & how it's applied. *Bad:* `int l = 1; if (0 == l)`. *Good:* avoid `l`, `0`, `I` as names entirely. *Bad:* `getActiveAccount()`, `getActiveAccounts()`, `getActiveAccountInfo()` in the same class — a caller cannot guess which to use. *Good:* names whose difference reflects a real semantic difference (`getActiveAccount()` vs `getActiveAccountsForRegion(region)`).

Meaningful Names — use pronounceable names

What it is. A pronounceable name is one a human can say out loud as a word, not spell out letter by letter. The deck contrasts `class DtaRcrd102` with fields `genymdhms`, `modymdhms`, `pszqint` against `class Customer` with `generationTimestamp`, `modificationTimestamp`, `recordId` (CleanCode p.13). Same data, but only the second can be spoken aloud.

What it's used for / why it matters. Programming is a social, collaborative activity: people discuss code in standups, code reviews, and pair-programming sessions. If you can't pronounce a name, you can't have a conversation about it — you end up saying "the gen-why-em-dee-aitch-em-ess field," which is error-prone and exhausting. Pronounceable names let the code be *talked about* as fluently as it is read, which matters enormously for a maintenance team handing work back and forth.

When & how it's applied. *Bad:* `Date genymdhms;` (generation date, year/month/day/hour/minute/second). *Good:* `Date generationTimestamp;`. The test is simple — read the name aloud to a colleague; if it comes out as an unsayable string of consonants, expand the abbreviations into real words.

Meaningful Names — use searchable names

What it is. A searchable name is one you can locate reliably with a text/grep search across the codebase. Single-letter names and raw numeric literals fail this test — searching for `e` or `7` matches thousands of irrelevant places. The deck contrasts `for (int j=0; j<34; j++) { s += (t[j]*4)/5; }` against a version

using named constants `realDaysPerIdealDay = 4`, `WORK_DAYS_PER_WEEK = 5`, `NUMBER_OF_TASKS`, and descriptive locals `realTaskDays`, `realTaskWeeks`, `sum` (CleanCode p.14). The magic `34`, `4`, `5` become searchable, named entities.

What it's used for / why it matters. Maintenance constantly requires finding *every* place a concept appears — e.g. to change the work-week from 5 to 6 days. If that 5 is a bare literal it is invisible among every other 5 in the system; if it is `WORK_DAYS_PER_WEEK` you find and change exactly the right occurrences. Searchability directly shrinks the risk of missing a spot during Impact Analysis. The length of a name should grow with the size of its scope: a one-line loop counter may be `i`, but a value referenced across a class deserves a long, searchable name.

When & how it's applied. *Bad:* `for (int j=0; j<34; j++) s += (t[j]*4)/5;` — the `34`, `4`, `5` are magic numbers and `s`, `t` are unsearchable. *Good:* lift the literals into named constants and give the locals descriptive names, so a future maintainer can grep `WORK_DAYS_PER_WEEK` and find every dependency at once.

Meaningful Names — avoid encodings (member prefixes, Hungarian notation)

What it is. An *encoding* is extra cruft baked into a name to carry metadata the name shouldn't have to carry. Two anti-patterns, each its own slide:

- **Member prefixes** (CleanCode p.15): prefixing instance fields to mark them as members, e.g. `m_dsc`. This is a relic of plain-text-editor days; `private String m_dsc; void setName(String name){ m_dsc = name; }` should be `String description; void setDescription(String description){ this.description = description; }`.
- **Hungarian notation** (CleanCode p.16): encoding a variable's *type* into its name, e.g. `PhoneNumber phoneString;`. The deck's own caption is "name not changed when type changed!" — just write `PhoneNumber phone;`.

What it's used for / why it matters. Both rules exist because encodings *lie over time and add cognitive load*. The `m_` prefix is pure noise that modern IDEs (which colour and scope members for you) make redundant — and noise the reader's eye must skip on every line. Hungarian notation is worse: it duplicates the compiler-checked type into the un-checked name, so the moment someone changes the declared type from `String` to `PhoneNumber` without renaming, the name `phoneString` becomes an active lie. Removing encodings keeps the name honest and lets the reader trust it.

When & how it's applied. *Bad:* `private String m_dsc;` and `PhoneNumber phoneString;`. *Good:* `private String description;` and `PhoneNumber phone;`. Rule of thumb: the type system already records the type and the IDE already records the scope — don't restate either in the name.

Meaningful Names — avoid mental mapping

What it is. Mental mapping is forcing the reader to hold a private translation table in their head — "in this loop, `r` means the URL without the host and scheme" — instead of giving the variable a name that means what it is. The deck's example: nested loops `for (a=0; ...) for (b=0; ...)` are *worse*, not better, than the conventional `for (i=0; ...) for (j=0; ...)` — `i` / `j` are an established loop-counter convention every programmer already reads automatically, whereas `a` / `b` force the reader to remember what they stand for (CleanCode p.17).

What it's used for / why it matters. Every name that needs translation is a small tax on working memory, and working memory is the scarcest resource when reading unfamiliar code. The rule is *not* "always use long names" — its real point is to use names the reader **already understands**: lean on established conventions

where they exist (`i`, `j`, `k` for simple loop counters) and on descriptive domain names everywhere else, but never invent a clever private code the reader must decode. "Clarity is king"; the professional makes the reader's job easy, not their own.

When & how it's applied. *Bad:* `for (int a = 0; a < n; a++) for (int b = 0; b < m; b++) grid[a][b] = ...` — `a / b` aren't a recognised counter convention, so the reader must map them. *Good:* either the conventional `i / j`, or fully descriptive `row / col`. Use the most-understood name available for the role the variable plays.

Meaningful Names — class names & method names

What it is. Conventions that align a name's grammar with the *kind* of thing it names. A class represents a *thing* and a method represents an *action*, so:

- **Class names** should be **nouns or noun phrases** — `Customer`, `WikiPage`, `Account`, `AddressParser` — and should **not** be vague (`Manager`, `Processor`, `Data`, `Info`) and "should not be a verb" (CleanCode p.18).
- **Method names** should be **verbs or verb phrases** — `postPayment`, `deletePage`, `save` (CleanCode p.19). Accessors/mutators/predicates follow the JavaBean `get / set / is` convention (`employee.getName()`, `customer.setName("mike")`, `if (paycheck.isPosted())`). For overloaded constructors, prefer **static factory methods** with descriptive names: `Complex.fromRealNumber(23.0)` is "generally better than" `new Complex(23.0)` (CleanCode p.19).

What it's used for / why it matters. These conventions let a reader infer a name's role from its grammar without looking it up: a noun is something you can hold a reference to, a verb is something you can invoke. Vague nouns like `Manager / Processor` are flagged because they describe *no* concrete responsibility — they become dumping grounds that attract unrelated code and violate SRP. Static factory methods matter because a constructor's name is fixed (it must equal the class name), so two constructors with the same signature shape can't be told apart; a named factory like `fromRealNumber` documents *which* construction you mean.

When & how it's applied. *Bad:* `class ParseAddress` (verb), `class DataManager` (vague), `account.payment()` (noun method). *Good:* `class AddressParser`, `class Account`, `account.postPayment()`. When you have overloaded constructors that confuse callers, replace them with named factories and make the real constructor private.

Meaningful Names — pick one word per concept, don't pun

What it is. Two complementary consistency rules. **Pick one word per concept** says choose a single verb for a single abstract operation and use it everywhere — don't have `fetch`, `retrieve`, and `get` as the names of equivalent methods on different classes, and don't mix `controller / manager / driver` for the same kind of object; the deck calls such mixing "confusing" (CleanCode p.20). **Don't pun** is the inverse: "avoid using the same word for two purposes" — e.g. don't reuse `add` for both arithmetic addition and appending to a list (CleanCode p.20).

What it's used for / why it matters. A consistent vocabulary turns the codebase into a *lexicon* the reader can rely on: once they learn that `get` means "retrieve a value," they can predict every API in the system, and IDE autocomplete becomes a reliable guide. Puns break that contract — a reader who has learned that `add` means arithmetic addition will be ambushed when `add` silently means "append" elsewhere, causing real bugs. One word per concept reduces the mental dictionary; no puns keeps each entry in that dictionary unambiguous.

When & how it's applied. *Bad:* `userRepo.fetch(id)`, `orderRepo.retrieve(id)`, `cartRepo.get(id)` for the same lookup operation. *Good:* pick `get` and use it for all three. *Pun bad:* `list.add(x)` meaning append and `total.add(x)` meaning arithmetic sum — give the append method a distinct name like `insert` or `append` if `add` already means arithmetic in your domain.

Meaningful Names — use solution-domain names, add meaningful context

What it is. Three rules about *where* a name's meaning comes from. **Solution-domain names** are CS/technical terms — pattern names, data-structure names, algorithm names — and they are encouraged because "people who read your code will be programmers"; `AccountVisitor` (a Visitor pattern) or `JobQueue` carry precise, shared meaning (CleanCode p.21). **Add meaningful context** means a name should make its role clear given its surroundings: bare `firstName`, `lastName`, `street`, `city`, `state`, `zipcode` are ambiguous alone (is `state` a US state or a status?), so either prefix them (`addrFirstName`, `addrState`) or, better, group them into a `class Address` so the context is the enclosing type (CleanCode p.21). **Don't add gratuitous context** is the guard-rail: don't over-qualify when context is already clear (CleanCode p.22) — `Address` is a fine class name; `AccountAddress` / `CustomerAddress` are fine *instance* names but poor *class* names; for MAC/port/web addresses prefer `PostalAddress`, `MAC`, `URI` rather than bolting a redundant qualifier onto everything.

What it's used for / why it matters. Using solution-domain names lets you say a lot in one word to the audience that will actually read the code — naming a class `...Visitor` instantly tells a programmer how it is meant to be used. Adding context disambiguates names that are meaningless in isolation, which is essential for searchability and for reading a variable far from its declaration. But *gratuitous* context is the failure mode in the other direction: prefixing every class in a "Gas Station Deluxe" app with `GSD` makes autocomplete useless (17 classes all start with `GSD`) and adds noise without information. The skill is supplying *exactly enough* context.

When & how it's applied. *Solution-domain good:* `class AccountVisitor`, `JobQueue`, `Deque`. *Context good:* group `street/city/state/zip` into `class Address` instead of free-floating fields. *Gratuitous-context bad:* `class GSDAccountAddress`; trim to `Address`. The test: add context until a stranger can tell what the name refers to, then stop.

Meaningful Names — the slide code, transcribed verbatim (CleanCode p.11–22)

This subsection records the *exact* code shown on the Meaningful Names slides, so the examples can be recognized and reproduced on the exam. Each pair is "before → after" as drawn on the slide (the decks draw a red arrow from the bad version to the good version).

Intention-revealing names (CleanCode p.11) — the minesweeper example, bad version:

```
public List<int[]> getThem() {
    List<int[]> list1 = new ArrayList<int[]>();
    for (int[] x : theList)
        if (x[0] == 4)
            list1.add(x);
    return list1;
}
```

and the good version — identical logic, self-explaining names:

```
public List<Cell> getFlaggedCells() {
    List<Cell> flaggedCells = new ArrayList<Cell>();
}
```



```

    for (Cell cell : gameBoard)
        if (cell.isFlagged())
            flaggedCells.add(cell);
    return flaggedCells;
}

```

Avoid disinformation (CleanCode p.12) — the visually-confusable single letters:

```

int a = l;
if (0 == l)
    a = 01;
else
    l = 01;

```

Lowercase `l` reads as `1`, uppercase `O` reads as `0` — the snippet is nearly impossible to parse by eye, which is the point.

Make meaningful distinctions (CleanCode p.12) — number-series naming carries zero information:

```

public static void copyChars(char a1[], char a2[]) {
    for (int i = 0; i < a1.length; i++) {
        a2[i] = a1[i];
    }
}

```

The fix is to name the arrays by *role*: `source` and `destination`.

Use pronounceable names (CleanCode p.13) — the full before/after class:

```

class DtaRcrd102 {
    private Date genymdhms;
    private Date modymdhms;
    private final String pszqint = "102";
    /* ... */
};

class Customer {
    private Date generationTimestamp;
    private Date modificationTimestamp;
    private final String recordId = "102";
    /* ... */
};

```

Note the detail that even the cryptic constant field (`pszqint = "102"`) gets an honest name (`recordId = "102"`) — same value, readable role.

Use searchable names (CleanCode p.14) — the magic-number loop:

```

for (int j = 0; j < 34; j++) {
    s += (t[j] * 4) / 5;
}

```

and the searchable rewrite, exactly as on the slide:

```

int realDaysPerIdealDay = 4;
const int WORK_DAYS_PER_WEEK = 5;
int sum = 0;

```

```
for (int j = 0; j < NUMBER_OF_TASKS; j++) {
    int realTaskDays = taskEstimate[j] * realDaysPerIdealDay;
    int realTaskWeeks = (realdays / WORK_DAYS_PER_WEEK);
    sum += realTaskWeeks;
}
```

Every former magic number (34 , 4 , 5) is now a named, greppable constant (NUMBER_OF_TASKS , realDaysPerIdealDay , WORK_DAYS_PER_WEEK), and the accumulator `s` over mystery array `t` became `sum` over `taskEstimate`.

Member prefixes (CleanCode p.15) — the full before/after:

```
public class Part {
    private String m_dsc; // The textual description
    void setName(String name) {
        m_dsc = name;
    }
}

public class Part {
    String description;
    void setDescription(String description) {
        this.description = description;
    }
}
```

Three smells die at once in the rewrite: the `m_` prefix, the abbreviation `dsc`, and the comment needed to decode it — and the misleading `setName` (which actually set the *description*) becomes the honest `setDescription`.

Hungarian notation (CleanCode p.16) — the deck's two-liner:

```
PhoneNumber phoneString;
// name not changed when type changed!

PhoneNumber phone;
```

Avoid mental mapping (CleanCode p.17) — both loops verbatim:

```
for (a = 0; a < 10; a++)
    for (b = 0; b < 10; b++)

for (i = 0; i < 10; i++)
    for (j = 0; j < 10; j++)
```

The slide's red arrow points from the `a / b` version to the `i / j` version: the established convention is the improvement.

Class names (CleanCode p.18) — bad: `Manager`, `Processor`, `Data`, `Info`; good: `Customer`, `WikiPage`, `Account`, `AddressParser`; rule annotation: "a class name should not be a verb."

Method names (CleanCode p.19) — verbatim examples: `postPayment`, `deletePage`, `save` ("methods should have verb or verb phrase names"); the accessor/mutator/predicate trio `string name = employee.getName()`; `customer.setName("mike")`; `if (paycheck.isPosted())...`; and the static-factory comparison:

```
Complex fulcrumPoint = Complex.fromRealNumber(23.0);  
// is generally better than  
Complex fulcrumPoint = new Complex(23.0);
```

Pick one word per concept / Don't pun (CleanCode p.20) — verbatim: `fetch`, `retrieve`, `get` // as equivalent methods and `controller`, `manager`, `driver` // confusing; Don't pun: "avoid using the same word for two purposes."

Use solution domain names / Add meaningful context (CleanCode p.21) — verbatim:

```
AccountVisitor, JobQueue // people who read your code will be programmers; context: firstName,  
lastName, street, city, state, zipcode → "a better solution" addrFirstName, addrLastName, addrState  
→ "a better solution" class Address.
```

Don't add gratuitous context (CleanCode p.22) — verbatim, all three steps: "Address is a fine name for a class"; "AccountAddress, CustomerAddress are fine names for *instances* of the class Address but could be poor names for *classes*"; "MAC addresses, port addresses, Web addresses — a better solution: PostalAddress, MAC, URI."

Exam tip: the Names slides run p.11–22 in a fixed order — intention-revealing (11), disinformation + meaningful distinctions (12), pronounceable (13), searchable (14), member prefixes (15), Hungarian notation (16), mental mapping (17), class names (18), method names (19), one word per concept + don't pun (20), solution-domain + meaningful context (21), gratuitous context (22). Recognizing *which rule* a given code snippet violates is the classic MCQ format for this material.

Functions — small, do one thing

What it is. The two foundational rules for functions. **Small:** the deck's "1. should be small / 2. should be smaller than that," with the concrete heuristics < **150 characters per line** and < **20 lines** (CleanCode p.23). **Do one thing:** "FUNCTIONS SHOULD DO ONE THING. THEY SHOULD DO IT WELL. THEY SHOULD DO IT ONLY" (CleanCode p.23). "One thing" means the function's body contains only steps that are all one level of abstraction below the function's stated purpose; if you can extract another function from it with a name that isn't just a restatement of the original, it was doing more than one thing.

What it's used for / why it matters. Small, single-purpose functions are the atomic unit of readability and reuse. A 200-line function hides its branches and forces the reader to hold the whole thing in working memory; a handful of well-named 5-line functions can each be understood in isolation and read like sentences in a paragraph. For maintenance this is decisive: small functions are easy to name, easy to unit-test (one concept per test), easy to reuse, and easy to change without side-effects — so they directly shrink the Impact Set of any change.

When & how it's applied. *Bad:* one long `payEmployee()` that validates input, computes pay, formats a report, and writes to the database. *Good:* `payEmployee()` orchestrates four tiny calls — `validate(emp)`, `int pay = computePay(emp)`, `Report r = format(pay)`, `persist(r)` — each doing exactly one thing. When a function grows past ~20 lines or you feel the urge to write a comment introducing its next "section," extract that section into a named function instead.

Functions — one level of abstraction per function & the Stepdown Rule

What it is. **One level of abstraction per function** means every statement in a function should sit at roughly the same conceptual altitude — don't mix a high-level policy call with a low-level string operation in

the same body. The deck illustrates three levels jammed together — high (`getHtml()`), intermediate (`String pagePathName = PathParser.render(pagePath);`), and "remarkably low" (`.append("\n")`) — and warns against blending them (CleanCode p.24). The **Stepdown Rule** is the consequence at file scale: code should read **top to bottom** like a narrative, every function followed by those one level of abstraction below it, so the whole file descends one step at a time (CleanCode p.24).

What it's used for / why it matters. Mixing abstraction levels is what makes a function hard to read: the reader can't tell which lines express the *essential* algorithm and which are incidental plumbing, so they can't skim. Keeping one level per function means each function reads as a coherent paragraph at a single altitude, and the Stepdown Rule lets a reader start at the top, grasp the overall flow, and descend into detail only where they need to — exactly the way you read a well-structured document. This makes Concept Location fast and keeps each function independently understandable.

When & how it's applied. *Bad:* `getHtml()` that calls `PathParser.render(...)` and hand-appends `"\n"` to a buffer in the same method. *Good:* `getHtml()` calls `renderPageWithSetupsAndTeardowns(...)`, which calls `includeSetupAndTeardownPages(...)`, which does the low-level buffer work — each one level down from its caller, arranged top-to-bottom in the file.

Functions — switch statements (replace with polymorphism)

What it is. A rule about a specific structural smell: a `switch` (or long `if-else` chain) that branches on a *type code*. The deck shows a `payAmount()` that `switches` on `getType()` returning different formulas per `EmployeeType.ENGINEER / SALESMAN / MANAGER` with a `default: throw` (CleanCode p.25). Such a switch violates SRP (the function does the work of three different employee types) and OCP (adding a fourth type forces editing it). The fix is to **replace the switch with polymorphism**: push the switch *once* into a factory that creates the right subclass, and let an abstract `EmployeeType.payAmount(Employee)` be overridden per subclass (`Salesman` returns `salary + commission`, `Manager` returns `salary + bonus`) (CleanCode p.26).

What it's used for / why it matters. The danger of a type-code switch is not one switch — it's that the *same* switch tends to get copy-pasted everywhere an operation depends on the type (`isPayday`, `deliverPay`, ...), so a single new type means hunting down and editing every copy (a rigidity symptom). Polymorphism inverts this: the type-specific behaviour lives *with* each type, the dispatch is done by the language's virtual call, and adding a new type means adding one new class and changing nothing else. You don't eliminate the switch entirely — you confine it to a single factory so concrete-type knowledge exists in exactly one place.

When & how it's applied. *Bad:* `payAmount()`, `isPayday()`, and `deliverPay()` each `switch` (`emp.getType()`). *Good:* a `Employee` (or `EmployeeType`) base type with `Engineer / Salesman / Manager` subclasses overriding `payAmount / isPayday / deliverPay`, plus one `EmployeeFactory.makeEmployee(record)` holding the lone remaining switch. Apply this whenever you see the same branch-on-type appear in more than one place.

Cross-link: this is the *same* example Martin uses in the macro deck for **OCP** — the modem `LogOn` `if/else` chain (DesignPrinciplesAndPatterns p.5–6). Clean Code and OO principles converge here.

Functions — descriptive names & function arguments

What it is. Two rules. **Use descriptive names:** a function's name should describe what it does, and it is fine for the name to be long — "a long descriptive name is better than a short enigmatic one" — as long as the naming is *consistent* (note how `includeSetupAndTeardownPages`, `includeSetupPages`,

`includeSuiteSetupPage ... share phrasing`) (CleanCode p.27). **Minimize arguments:** "the ideal number of arguments for a function is **zero**" (niladic); then one (monadic); two (dyadic); three (triadic) should be avoided; more than three needs special justification (CleanCode p.27).

What it's used for / why it matters. A descriptive name is the function's documentation — if naming the function well is hard, that usually signals the function is doing too much. Arguments are expensive because each one is something the reader must understand, something a caller can get wrong (e.g. swap the order), and something a test must enumerate combinations of. Fewer arguments means fewer ways to misuse the function and fewer test cases to cover; a niladic function is the easiest of all to read, call, and test.

When & how it's applied. *Bad name:* `proc()`. *Good name:* `includeSetupAndTeardownPages()` — long but unambiguous. *Too many args:* `Circle makeCircle(double x, double y, double r);` reduce by bundling (see argument objects below). When a function needs three or more arguments, treat it as a prompt to bundle them into an object or to split the function.

Functions — common monadic forms

What it is. A catalogue of the *legitimate* reasons a function takes exactly one argument, so you can recognise when a single argument is natural versus when it hides a second responsibility. The three good forms (CleanCode p.28): (1) **ask a question about the argument** — `boolean fileExists("MyFile");` (2) **operate on it and return a transformed result** — `InputStream fileOpen("MyFile");` (3) the **transformation** form, where "the transformation should appear as the return value," so `StringBuffer transform(StringBuffer in)` (returns a new buffer) is better than `void transform(StringBuffer out)` (mutates through the argument). A separate **event** form takes an input that changes system state and returns nothing: `void passwordAttemptFailedNtimes(int attempts)`.

What it's used for / why it matters. Knowing these forms lets the reader form the right expectation from the signature alone — a monadic function returning a value is a query or transform; a monadic `void` function is an event with a side-effect. The transformation rule (return the result rather than mutate the argument) matters because a `void transform(out)` is surprising: the reader can't tell from the call site that `out` was changed, which is a common source of bugs. Returning the transformed value makes the data-flow explicit.

When & how it's applied. *Good question:* `if (fileExists(path))`. *Good transform:* `StringBuffer cleaned = transform(raw);`. *Bad transform:* `void transform(StringBuffer out)` — the mutation is hidden; prefer the value-returning form. *Good event:* `passwordAttemptFailedNtimes(3)` — clearly a state-changing notification, and its event-y name signals that.

Functions — flag arguments (avoid)

What it is. A *flag argument* is a boolean (or enum) parameter passed to select between two behaviours inside the function. It is a code smell because, by definition, a function that branches on a flag does *two* things — one for `true`, one for `false`. The deck's example: `render(true)` is opaque at the call site (*true what?*); split it into `renderForSuite()` and `renderForSingleTest()` (CleanCode p.29).

What it's used for / why it matters. Flag arguments hurt on two fronts. At the **call site** they are unreadable — `render(true)` tells the reader nothing; they must open the function to learn what `true` selects. In the **function body** they violate "do one thing" and tangle two code paths together, making the function harder to test (you must test both branches through one entry point) and harder to change (the two behaviours can't evolve independently). Splitting into two named functions makes both call site and implementation self-explanatory.

When & how it's applied. *Bad:* `render(boolean isSuite)`, called as `render(true)`. *Good:* two functions `renderForSuite()` and `renderForSingleTest()`, with the caller choosing by name. The same applies to any argument whose only job is to pick a branch — promote the choice into the function name.

Functions — dyadic, triads, argument objects, verbs & keywords

What it is. A set of finer rules for handling functions that genuinely need more than one argument.

- **Dyadic functions** (two args) are sometimes fine when the two arguments are naturally ordered components of one thing (`Point p = new Point(0,0)` — x then y is conventional), but they add a "cost of ordering" you must remember. `writeField(name)` is "easier to understand than" `writeField(outputStream, name)`, and `assertEquals(expected, actual)` is "problematic" because there is no natural order to *expected* vs *actual* — readers routinely transpose them (CleanCode p.30). **Triads** like `assertEquals(message, expected, actual)` compound the ordering problem and should be avoided (CleanCode p.30).
- **Argument objects:** when several arguments belong together, wrap them into a named object — `makeCircle(double x, double y, double radius)` becomes `makeCircle(Point center, double radius)` (CleanCode p.31). The `(x, y)` pair *was* a concept (a point) hiding in the argument list.
- **Verbs and keywords:** encode the meaning (and order) of arguments into the function name. `write(name)` → `writeField(name)` (the keyword *field* says what *name* is); `assertEquals(expected, actual)` → `assertExpectedEqualsActual(expected, actual)`, where the name now *tells you the order* and removes the ambiguity (CleanCode p.31).

What it's used for / why it matters. These techniques attack the central problem with multi-argument functions — that arguments carry meaning only by position, which the reader must remember and the caller can get wrong. Bundling related arguments into an object both shortens the list *and* names a concept that was implicit; encoding meaning into the function name turns positional guesswork into something the call site states out loud. The result is call sites that read correctly without consulting the signature.

When & how it's applied. *Argument-object refactor:* turn `makeCircle(x, y, r)` into `makeCircle(center, r)` once you notice `x, y` always travel together. *Keyword refactor:* rename `assertEquals` to `assertExpectedEqualsActual` so `assertExpectedEqualsActual(expected, actual)` reads in the only correct order. Reach for these whenever an argument list is long or its order is non-obvious.

Functions — Command Query Separation

What it is. Command/Query Separation (CQS, from Bertrand Meyer) says a function should *either do* something (a command that changes state and returns nothing) *or answer* something (a query that returns a value and changes nothing) — **never both** (CleanCode p.32). The deck's offender: `public boolean set(String attribute, String value);` used as `if (set("username", "unclebob")) ...`. The call is genuinely ambiguous — does `set` mean "set the value" (and return success), or "is this attribute already set"? The fix is either to rename it honestly (`setAndCheckIfExists`) or, better, to separate the query from the command: `if (attributeExists("username")) { setAttribute("username", "unclebob"); ... }`.

What it's used for / why it matters. A function that both mutates state *and* returns a value reads ambiguously and invites side-effect bugs: the reader of `if (set(...))` cannot tell whether the condition is a question or an action with a status. Keeping commands and queries separate means queries are safe to call freely (no surprises, side-effect-free, easy to reason about and test) and commands have a single obvious purpose. It is what makes expressions like `if (attributeExists(x))` trustworthy.

When & how it's applied. *Bad:* `if (set("username", "unclebob"))` — command masquerading as query. *Good:* split into a side-effect-free `attributeExists("username")` query and a `setAttribute("username", "unclebob")` command, used in sequence. Apply whenever a method both changes something and returns a meaningful value about that change. (Note: this is the *small* CQS rule, not the architectural CQRS pattern.)

Functions — DRY & structured programming

What it is. Two final function rules. **Don't Repeat Yourself (DRY):** every piece of knowledge should have a single authoritative representation in the code; the deck calls duplication possibly "the root of all evil in software" (CleanCode p.33). **Structured programming:** Dijkstra's classic rules demand **one entry, one exit** per function (no `break`, `continue`, or multiple `return`); the deck's nuance is that the rule pays off for large functions, but for the *small* functions clean code favours, "an occasional multiple return, break, or continue statement ... can sometimes even be more expressive" than rigidly forcing a single exit (CleanCode p.33).

What it's used for / why it matters. DRY matters because duplicated logic means a change must be made in N places, and the bug is forgetting one of them — duplication directly inflates the Impact Set and is a leading cause of fragility. Eliminating it (extract a method, pull up a superclass, introduce a helper) means each rule lives once and changes once. The structured-programming nuance matters as a corrective to dogma: single-exit discipline keeps *large* functions readable, but in a tiny function an early `return` for a guard clause is clearer than nesting the whole body in an `if`. The deck's point is to apply the spirit of the rule, not the letter, once functions are already small.

When & how it's applied. *DRY bad:* the same five-line validation copied into three handlers; *good:* extract `validate(request)` and call it from all three. *Structured-programming good (small function):* `if (input == null) return EMPTY;` as a guard clause at the top, rather than wrapping the rest of a short method in `if (input != null) { ... }`.

Functions — the slide code, transcribed verbatim (CleanCode p.23–33)

The exact wording and code of the Functions slides, for recognition and reproduction.

Small (CleanCode p.23) — the slide's full annotation set:

```
// rules of functions:
//   1. should be small
//   2. should be smaller than that
// < 150 characters per line
// < 20 lines
```

Do One Thing (CleanCode p.23) — verbatim, in the deck's capitals: "FUNCTIONS SHOULD DO ONE THING. THEY SHOULD DO IT WELL. THEY SHOULD DO IT ONLY."

One level of abstraction (CleanCode p.24) — the three altitude examples exactly as annotated:

```
// high level of abstraction
getHtml()

// intermediate level of abstraction
String pagePathName = PathParser.render(pagePath);
```

```
// remarkably low level
.append("\n")
```

and "Reading Code from Top to Bottom" is glossed on the slide simply as "// the Stepdown Rule" (CleanCode p.24).

Switch statements, the offending version (CleanCode p.25) — full transcription:

```
class Employee...
    int payAmount() {
        switch (getType()) {
            case EmployeeType.ENGINEER:
                return _monthlySalary;
            case EmployeeType.SALESMAN:
                return _monthlySalary + _commission;
            case EmployeeType.MANAGER:
                return _monthlySalary + _bonus;
            default:
                throw new Exception("Incorrect Employee");
        }
    }
}
```

Switch statements, the polymorphic replacement (CleanCode p.26) — full transcription; note that the slide itself shows only the type hierarchy (the single-factory refinement is the book's accompanying advice):

```
class EmployeeType...
    abstract int payAmount(Employee emp);

class Salesman...
    int payAmount(Employee emp) {
        return emp.getMonthlySalary() + emp.getCommission();
    }

class Manager...
    int payAmount(Employee emp) {
        return emp.getMonthlySalary() + emp.getBonus();
    }
}
```

The underscore fields (`_monthlySalary` , `_commission` , `_bonus`) of the switch version became proper accessors (`getMonthlySalary()` , `getCommission()` , `getBonus()`) in the polymorphic version — the slide quietly fixes an encodings smell at the same time.

Use descriptive names (CleanCode p.27) — the slide shows a real rename from the book's FitNesse example, `testableHtml => includeSetupAndTeardownPages` , then demonstrates *consistent phrasing* across the family: `includeSetupAndTeardownPages` , `includeSetupPages` , `includeSuiteSetupPage` , `includeSetupPage` — and asks, teasingly, "what happened to `includeTeardownPages` , `includeSuiteTeardownPage` , `includeTeardownPage` ?" The point: a consistent naming scheme makes *gaps* in the API visible.

Function arguments (CleanCode p.27) — verbatim: "the ideal number of arguments for a function is zero."

Common monadic forms (CleanCode p.28) — all four, exactly as annotated:

```
// asking a question about that argument
boolean fileExists("MyFile")

// operating on that argument, transforming and returning it
InputStream fileOpen("MyFile")

// if a function is going to transform its input argument,
// the transformation should appear as the return value
StringBuffer transform(StringBuffer in)
// is better than
void transform(StringBuffer out)

// event, use the argument to alter the state of the system
void passwordAttemptFailedNtimes(int attempts)
```

Flag arguments (CleanCode p.29) — the slide is a single red arrow from `render(true)` to the pair `renderForSuite()` / `renderForSingleTest()`.

Dyadic functions and triads (CleanCode p.30) — note the slide's title is literally "Functions (Bad)"; all annotations verbatim:

```
writeField(name)
// is easier to understand than
writeField(outputStream, name)

// perfectly reasonable
Point p = new Point(0,0)

// problematic
assertEquals(expected, actual)
```

and under "Triads (Avoid)": `assertEquals(message, expected, actual)`.

Argument objects / Verbs and keywords (CleanCode p.31) — verbatim:

```
Circle makeCircle(double x, double y, double radius);
Circle makeCircle(Point center, double radius);

write(name)          → writeField(name)
assertEquals(expected, actual) → assertExpectedEqualsActual(expected, actual)
```

Command Query Separation (CleanCode p.32) — the slide is heavily hand-annotated. The offender is `public boolean set(String attribute, String value);` used as `if (set("username", "unclebob"))...`; the handwriting asks "**what does it mean in context of if ? (verb or adjective?)**" — i.e. is `set` a *verb* ("set it!") or an *adjective* ("is it set?"). A second annotation concedes that renaming to `setAndCheckIfExists` would be "**a bit better**", before the slide gives the real fix, labelled **command / query / separation** down the margin:

```
if (attributeExists("username")) {    // query
    setAttribute("username", "unclebob"); // command
    ...
}
```

DRY (CleanCode p.33) — verbatim: "duplication may be the root of all evil in software."

Structured programming (CleanCode p.33) — the slide names its source: "Edsger Dijkstra's rules: one entry, one exit"; then the clean-code nuance: "functions small ... occasional multiple return, break, or continue statement can sometimes even be more expressive [than] Dijkstra's rules."

Comments — don't comment bad code; explain yourself in code

What it is. The governing attitude to comments. The headline rule is "**Comments do not make up for bad code**" — "don't comment bad code, rewrite it!" (CleanCode p.34) — and its positive form, **explain yourself in code**: before writing a comment, try to make the code itself say what the comment would have said. The deck's worked example replaces `// Check to see if the employee is eligible for full benefits` sitting above `if ((employee.flags & HOURLY_FLAG) && (employee.age > 65))` with a self-naming predicate, `if (employee.isEligibleForFullBenefits())` (CleanCode p.34). The comment becomes unnecessary because the code now states its own intent.

What it's used for / why it matters. Martin's core thesis is that *a comment is, at best, a necessary evil and usually a failure to express intent in code*. The reason is rot: comments are not checked by the compiler and not run by the tests, so they drift out of sync with the code they describe. A stale comment is worse than no comment — it actively misleads the maintainer. Code, by contrast, cannot lie about what it does. So the rule is to invest the effort that would have gone into a comment into a better name or an extracted, well-named function instead; the explanation then lives in something that *stays true*.

When & how it's applied. Whenever you feel the urge to write a comment explaining *what* a block does, ask whether a rename or an Extract Method would make the comment redundant. *Bad*: a cryptic boolean expression with a comment above it. *Good*: extract the expression into `isEligibleForFullBenefits()` and delete the comment. (Comments that explain *why* — intent, warnings, legal notices — are the legitimate exceptions covered next.)

Comments — the good comments

What it is. The minority of comments that genuinely earn their keep — almost all of them explain *why* or supply information the code legitimately cannot carry, rather than restating *what* (CleanCode p.35–37):

- **Legal comments** — copyright/license headers required by company or law (CleanCode p.35).
- **Informative comments** — supply information the code can't, e.g. documenting the format a regex matches: `// format matched kk:mm:ss EEE, MMM dd, yyyy` (CleanCode p.35).
- **Explanation of intent** — the *why* behind a non-obvious decision: `//This is our best attempt to get a race condition by creating large number of threads.` (CleanCode p.36).
- **Clarification** — translate an obscure expression into readable terms when you can't change it (e.g. a library API): `assertTrue(a.compareTo(b) == -1); // a < b` (CleanCode p.36).
- **Amplification** — stress the importance of something a reader might dismiss as trivial: a comment explaining that a `.trim()` "is real important. It removes the starting spaces..." (CleanCode p.37).
- **Warning of consequences, TODO comments, and Javadocs in public APIs** — "there is nothing quite so helpful and satisfying as a well-described public API" (CleanCode p.37).

What it's used for / why it matters. These categories exist because some knowledge truly cannot be expressed in code: code can show *what* it does but not *why a human chose to do it that way*, nor a legal obligation, nor a warning about a non-obvious consequence. A good comment fills exactly that gap — it tells the future maintainer something they could not have recovered by reading the code, which prevents them from "fixing" something deliberate or repeating a mistake the author already hit. Public-API Javadoc matters

because the consumers of an API can't read its implementation, so the contract has to be documented at the boundary.

When & how it's applied. Write a comment only when it carries information of one of the kinds above and the code genuinely can't. *Good intent:* `// best effort to provoke a race condition above a thread-stress loop.` *Good warning:* `// Don't run unless you have time to kill – this test is very slow.` *Good clarification:* `// a < b` next to a cryptic `compareTo` check you can't rewrite. If the comment merely echoes the code, it belongs to the bad-comment taxonomy below.

Comments — the good comments, slide examples transcribed (CleanCode p.35–37)

The exact examples the deck shows for each *good* comment category.

Legal comments (CleanCode p.35):

```
// Copyright (C) 2011 by Osoco. All rights reserved.  
// Released under the terms of the GNU General Public License  
// version 2 or later.
```

Informative comments (CleanCode p.35) — two examples. First, a comment that names what an abstract method's return is *for* — together with the slide's note that a rename would do the job better:

```
// Returns an instance of the Responder being tested.  
protected abstract Responder responderInstance();  
// renaming the function: responderBeingTested
```

Second, the regex-documenting comment, where the comment supplies the matched format that the pattern itself cannot show readably:

```
// format matched kk:mm:ss EEE, MMM dd, yyyy  
Pattern timeMatcher = Pattern.compile(  
    "\\d*:\\d*:\\d* \\w*, \\w* \\d*, \\d*");
```

Explanation of intent (CleanCode p.36) — the race-condition stress loop, verbatim:

```
//This is our best attempt to get a race condition  
//by creating large number of threads.  
for (int i = 0; i < 25000; i++) {  
    WidgetBuilderThread widgetBuilderThread =  
        new WidgetBuilderThread(widgetBuilder, text, failFlag);  
    Thread thread = new Thread(widgetBuilderThread);  
    thread.start();  
}
```

No rename could carry this information: the *why* (deliberately provoking a race with 25,000 threads) lives legitimately in the comment.

Clarification (CleanCode p.36) — translating an API you cannot change:

```
assertTrue(a.compareTo(b) == -1); // a < b  
assertTrue(b.compareTo(a) == 1); // b > a
```

Amplification (CleanCode p.37) — stressing a seemingly-trivial call, verbatim:


```
String listItemContent = match.group(3).trim();
// the trim is real important. It removes the starting
// spaces that could cause the item to be recognized
// as another list.
new ListItemWidget(this, listItemContent, this.level + 1);
return buildList(text.substring(match.end()));
```

Javadocs in public APIs (CleanCode p.37) — verbatim: "there is nothing quite so helpful and satisfying as a well-described public API." (Its mirror-image — Javadocs in *nonpublic* code — appears at the end of the bad list, CleanCode p.52.)

Comments — the bad comments (the full taxonomy)

What it is. Martin's catalogue of comment *smells* — the many ways comments add clutter, noise, or misinformation instead of value. Each is a slide in the deck (CleanCode p.38–52). **Why it matters:** the deck spends roughly six times as many slides on bad comments as on good ones, which is the real lesson — the default expectation for a comment should be skepticism. Each of these categories is a comment that *should not have been written*: it either duplicates the code, restates the obvious, embeds information version control already holds, or has drifted into being wrong. The maintenance cost is concrete — every such comment is something a reader must read, distrust, and mentally discard, and the journal/attribution/commented-out kinds positively rot the source. **When & how it's applied:** when you encounter one of these, the remedy is almost always to *delete* it (the information lives in the code or in Git) or to *replace the code* so no comment is needed. Memorize the *category names* — they're prime exam material:

- **Mumbling** — a comment so terse it's meaningless, e.g. a `catch(IOException e) { // No properties files means all defaults are loaded }` that forces you to read other code to understand it (CleanCode p.38).
- **Redundant comments** — the comment restates the code and is longer than the code it documents; e.g. a Javadoc'd `waitForClose(...)` whose comment adds nothing (CleanCode p.39); or `/** The processor delay for this component. */ protected int backgroundProcessorDelay = -1;` (CleanCode p.40).
- **Mandated comments** — Javadoc forced on every function/variable by policy, producing clutter like a fully `@param`-documented `addCD(...)` where the params are obvious (CleanCode p.41).
- **Journal comments** — a change-log embedded in the source ("Changes (from 11-Oct-2001) ...") that version control should hold instead (CleanCode p.42).
- **Noise comments** — say nothing: `/** Default constructor. */`, `/** The day of the month. */ private int dayOfMonth;` (CleanCode p.43).
- **Scary noise** — copy-pasted noise comments that are subtly *wrong* (e.g. `/** The version. */ private String info;`) (CleanCode p.44).
- **Don't use a comment when you can use a function or variable** — refactor the condition into a named variable instead of commenting it (CleanCode p.45).
- **Position markers** — banner comments like `// Actions //////////////////////////////////` (CleanCode p.46).
- **Closing-brace comments** — `} //while` — a sign your blocks are too big; shorten the function instead (CleanCode p.46).
- **Attributions and bylines** — `/* Added by Rick */` (version control's job) (CleanCode p.47).
- **Commented-out code** — dead code left in comments; delete it, VCS remembers it (CleanCode p.47).
- **HTML comments** — HTML markup inside source comments is "an abomination" (CleanCode p.48).
- **Nonlocal information** — a comment describing something far from the code it sits next to (e.g. documenting a default port value `8082` on a setter) (CleanCode p.49).
- **Too much information** — dumping an RFC excerpt into a comment (CleanCode p.50).

- **Inobvious connection** — a comment whose relationship to the code isn't clear (the "200 bytes for header info" example) (CleanCode p.51).
- **Function headers** — "short functions don't need much description" — a well-named small function beats a header comment (CleanCode p.51).
- **Javadocs in nonpublic code** — the "extra formality of the javadoc comments" is wasted on internal code (CleanCode p.52).

Comments — the bad comments, slide examples transcribed (CleanCode p.38–52)

The taxonomy above gives the category names; this records the *actual code* each slide shows, since the exam may present the code and ask for the category.

Mumbling (CleanCode p.38) — the full snippet; the catch-comment is so terse you must go read other code to learn what it means:

```
try {
    String propertiesPath = propertiesLocation + "/" + PROPERTIES_FILE;
    FileInputStream propertiesStream = new FileInputStream(propertiesPath);
    loadedProperties.load(propertiesStream);
}
catch(IOException e) {
    // No properties files means all defaults are loaded
}
```

(Who loads the defaults? Where? Were they loaded *before* this call? The comment mumbles.)

Redundant comments, example 1 (CleanCode p.39) — a comment *longer and less precise* than the code it restates:

```
// Utility method that returns when this.closed is true.
// Throws an exception if the timeout is reached.
public synchronized void waitForClose(final long timeoutMillis) throws Exception {
    if(!closed) {
        wait(timeoutMillis);
        if(!closed)
            throw new Exception("MockResponseSender could not be closed");
    }
}
```

Redundant comments, example 2 (CleanCode p.40) — the Tomcat-style field Javadocs that restate each declaration:

```
/**
 * The processor delay for this component.
 */
protected int backgroundProcessorDelay = -1;

/**
 * The lifecycle event support for this component.
 */
protected LifecycleSupport lifecycle = new LifecycleSupport(this);

/**
 * The container event listeners for this Container.
 */
protected ArrayList listeners = new ArrayList();
```

Mandated comments (CleanCode p.41) — policy-forced Javadoc where every `@param` is obvious from the signature:

```
/**
 * @param title The title of the CD
 * @param author The author of the CD
 * @param tracks The number of tracks on the CD
 * @param durationInMinutes The duration of the CD in minutes
 */
public void addCD(String title, String author,
                  int tracks, int durationInMinutes) {
    CD cd = new CD();
    cd.title = title;
    cd.author = author;
    cd.tracks = tracks;
    cd.duration = durationInMinutes;
}
```

Journal comments (CleanCode p.42) — the embedded changelog (from the JCommon/JFreeChart date class), verbatim highlights:

```
* Changes (from 11-Oct-2001)
* -----
* 11-Oct-2001 : Re-organised the class and moved it to new
*              package com.jrefinery.date (DG);
* 05-Nov-2001 : Added a getDescription() method, and
*              eliminated NotableDate class (DG);
* 12-Nov-2001 : IBD requires setDescription() method, now
*              that NotableDate class is gone (DG); Changed
*              getPreviousDayOfWeek(), getFollowingDayOfWeek()
*              and getNearestDayOfWeek() to correct bugs (DG);
* 05-Dec-2001 : Fixed bug in SpreadsheetDate class (DG);
* 29-May-2002 : Moved the month constants into a separate
*              interface (MonthConstants) (DG);
```

Version control holds all of this; in the source it is dead weight that must be maintained by hand.

Noise comments (CleanCode p.43) — verbatim:

```
/**
 * Default constructor.
 */
protected AnnualDateRule() { }

/** The day of the month. */
private int dayOfMonth;

/**
 * Returns the day of the month.
 * @return the day of the month.
 */
public int getDayOfMonth() {
    return dayOfMonth;
}
```

Scary noise (CleanCode p.44) — noise comments copy-pasted until one is *wrong*; spot the cut-paste error in the last entry:

```

/** The name. */
private String name;

/** The version. */
private String version;

/** The licenceName. */
private String licenceName;

/** The version. */
private String info;

```

(`info` is documented as "The version" — the comment was pasted from the field above and never edited. That is what makes the noise *scary*: it has decayed into disinformation.)

Don't use a comment when you can use a function or a variable (CleanCode p.45) — full before/after:

```

// does the module from the global list <mod> depend on the
// subsystem we are part of?
if (smodule.getDependSubsystems()
    .contains(subSysMod.getSubSystem()))

// this could be rephrased without the comment as
ArrayList moduleDependees = smodule.getDependSubsystems();
String ourSubSystem = subSysMod.getSubSystem();
if (moduleDependees.contains(ourSubSystem))

```

(Note the bonus lesson: the before-version is also a Law-of-Demeter-style chained call; introducing explaining variables removes both the comment *and* flattens the chain.)

Position markers (CleanCode p.46) — verbatim: `// Actions //////////////////////////////////////`.

Closing-brace comments (CleanCode p.46) — verbatim:

```

while ((line = in.readLine()) != null) {
    lineCount++;
    charCount += line.length();
    String words[] = line.split("\\W");
    wordCount += words.length;
} //while

```

If you need `} //while` to find your way back, the block is too long — shorten the function instead.

Attributions and bylines (CleanCode p.47) — verbatim: `/* Added by Rick */`. Version control already knows who added what, and Rick's comment will still be there years after Rick's code has been rewritten by others.

Commented-out code (CleanCode p.47) — verbatim:

```

InputStreamResponse response = new InputStreamResponse();
response.setBody(formatter.getResultStream(),
    formatter.getByteCount());
// InputStream resultsStream = formatter.getResultStream();
// StreamReader reader = new StreamReader(resultsStream);
// response.setContent(reader.read(formatter.getByteCount()));

```

Others who see commented-out code won't dare delete it ("someone must have left it for a reason"), so it accumulates "like dregs at the bottom of a bad bottle of wine" — delete it; the VCS remembers.

HTML comments (CleanCode p.48) — the slide shows a Javadoc stuffed with markup and entity-escaped tags (an Ant `taskdef` usage block full of `<taskdef name="execute-fitness-tests"`; `classname="fitnesse.ant.ExecuteFitnessTestsTask"`; `...>`), rendering the comment unreadable exactly where it is read most — in the source editor. The deck's verdict (after the book): an abomination.

Nonlocal information (CleanCode p.49) — verbatim; the comment states a *default* that is decided somewhere else entirely, next to a setter that has no control over it:

```
/**
 * Port on which fitnesse would run. Defaults to <b>8082</b>.
 *
 * @param fitnessePort
 */
public void setFitnessePort(int fitnessePort)
{
    this.fitnessePort = fitnessePort;
}
```

When the real default changes, nobody will remember to update this distant comment.

Too much information (CleanCode p.50) — the slide pastes a chunk of **RFC 2045 (MIME), section 6.8 Base64 Content-Transfer-Encoding** into a comment — the 24-bit input groups encoded as four concatenated 6-bit groups, output strings of 4 encoded characters, the most-significant-bit ordering, all of it. Historical/technical trivia at this depth belongs in the referenced document, not the source file.

Inobvious connection (CleanCode p.51) — verbatim:

```
/*
 * start with an array that is big enough to hold all
 * the pixels (plus filter bytes), and an extra 200 bytes
 * for header info
 */
this.pngBytes = new byte[((this.width + 1) * this.height * 3) + 200];
```

The comment and the code don't connect: what is a filter byte? Which term of the expression is the "plus filter bytes" part — the `+ 1`? Why 200? A comment that itself needs explaining has failed at its only job.

Function headers (CleanCode p.51) — verbatim: "short functions don't need much description" — a well-chosen name on a small function beats a header comment.

Javadocs in nonpublic code (CleanCode p.52) — verbatim: the "extra formality of the javadoc comments" is wasted on code not consumed through a published API; inside the codebase the formal `@param` / `@return` apparatus is just more mandated-comment clutter.

Formatting — purpose, the newspaper metaphor, vertical formatting

What it is. The rules for how code is laid out on the page — chiefly *vertical* spacing (blank lines, ordering of declarations and functions). The deck's framing:

- **Purpose of formatting = communication** (CleanCode p.53). Formatting is a professional courtesy to readers, not decoration.
- **The newspaper metaphor:** a source file should read like a news article — a headline (the most important, high-level stuff) at the top, increasing detail as you scroll down: read **high-level** → **details**, top of file to bottom (CleanCode p.53).
- **Vertical openness between concepts:** "each blank line is a visual cue that identifies a new and separate concept" — separate distinct ideas with blank lines (CleanCode p.53).
- **Vertical density:** lines that are closely related should be kept vertically close — "vertical density implies close association"; a gratuitous comment shoved between two tightly-related lines breaks that density and obscures the relationship (CleanCode p.54).
- **Vertical distance:** declare variables "as close to their usage as possible"; put instance variables at the top of the class; keep **dependent functions** vertically close, with the caller above the callee (so reading top-down follows the call flow); and group code by **conceptual affinity** — related code near related code (CleanCode p.55).

What it's used for / why it matters. Layout is the reader's map of the code. Because we read code far more than we write it, formatting is a force-multiplier on every future read: the newspaper structure lets a reader grasp a file's purpose from the top without reading it all; blank lines chunk the code into digestible concepts; keeping declarations near their use and callers above callees means the reader's eye rarely has to jump around or scroll back. Good vertical formatting reduces the effort of Concept Location and lowers the chance of misreading related lines as unrelated.

When & how it's applied. *Good:* a class opens with its constants and fields, then its public methods in calling order, each private helper just below the public method that uses it; blank lines separate logical sections; a local is declared on the line before it is first used. *Bad:* fields scattered through the class, a helper defined 300 lines above its caller, no blank lines so three unrelated concepts blur together.

Formatting — horizontal formatting & team rules

What it is. The rules for layout *within* a line, plus the meta-rule that settles disputes:

- **Horizontal openness and density** — use spaces to visually group operands that bind tightly and separate those that don't, e.g. `(-b - Math.sqrt(determinant)) / (2*a)`, where the spacing around `/` separates the two major operands while `2*a` stays tight (CleanCode p.56).
- **Horizontal alignment** — Martin actually *discourages* aligning columns of declarations (lining up all the `=` signs); it draws the eye to the wrong thing and breaks under maintenance. Ordinary indentation is enough (CleanCode p.57–58).
- **Breaking indentation** — never collapse a block onto a single line (`if (x) doThing();` on one line, or a one-line `while` body) to save space; preserve the indentation structure so the block scope is visible (CleanCode p.59).
- **Team rules:** "every programmer has his own favorite formatting rules, but if he works in a team then the team rules" win (CleanCode p.60).

What it's used for / why it matters. Horizontal formatting communicates operator precedence and grouping at a glance, helping the reader parse an expression the way the compiler does. The anti-alignment and don't-break-indentation rules guard against layouts that *look* tidy but mislead or hide structure. The **team rules** point is the most important on this slide: the goal of formatting is a consistent, predictable reading experience, and a codebase where every file follows one programmer's personal style is more readable than one where each file follows its author's taste — so individual preference yields to a single agreed convention (ideally enforced by an auto-formatter).

When & how it's applied. *Good:* spacing that reflects precedence (`2*a` tight, `/` spaced); every block indented even when one line; the whole team's files formatted identically by a shared tool. *Bad:* hand-aligned declaration columns that one rename throws off, a one-line `if` -body that hides the scope, or two files in the same project formatted by different rules.

Formatting — the slide examples transcribed (CleanCode p.53–60)

The exact illustrations behind the formatting rules.

Vertical density (CleanCode p.54) — the slide shows two field declarations whose tight association is *broken* by interposed noise Javadocs:

```
/**
 * The class name of the reporter listener
 */
private String m_className;

/**
 * The properties of the reporter listener
 */
private ArrayList m_properties = new ArrayList();
```

The two fields describe one concept (a reporter listener) and belong on adjacent lines; the useless comments push them apart, so the eye no longer reads them as a unit. (Bonus smell on the same slide: both fields wear `m_` member prefixes, the encoding banned on CleanCode p.15.)

Vertical distance (CleanCode p.55) — the slide's four rules verbatim:

```
// variables
//   should be declared as close to their usage as possible
// instance variables
//   should be declared at the top of the class
// dependent functions
//   if one function calls another, they should be vertically
//   close, and the caller should be above the called
// conceptual affinity
//   certain bits of code want to be near other bits
```

Horizontal openness and density (CleanCode p.56) — both worked examples:

```
private void measureLine(String line) {
    lineCount++;
    int lineSize = line.length();
    totalChars += lineSize;
    lineWidthHistogram.addLine(lineSize, lineCount);
    recordWidestLine(lineSize);
}

public static double root2(int a, int b, int c) {
    double determinant = determinant(a, b, c);
    return (-b - Math.sqrt(determinant)) / (2*a);
}
```

In `root2`, spaces surround the low-precedence operators (`-b - Math.sqrt(...)`, the division) while the tightly-binding factor `2*a` is written without spaces — the whitespace mirrors operator precedence.

Horizontal alignment (CleanCode p.57–58) — the before/after is the `FitNesseExpediter` declaration list. p.57 shows the *aligned-columns* version:

```
public class FitNesseExpediter implements ResponseSender
{
    private    Socket          socket;
    private    InputStream      input;
    private    OutputStream     output;
    private    Request          request;
    private    Response         response;
    private    FitNesseContext  context;
    protected long             requestParsingTimeLimit;
    private    long             requestProgress;
    private    long             requestParsingDeadline;
    private    boolean          hasError;
    ...
}
```

p.58 shows the same list with ordinary single spaces (`private Socket socket;` etc.). The aligned version *looks* tidy but draws the eye down the column of names without reading their types, and one new long type name forces re-aligning every row — so the deck favours the plain version.

Breaking indentation (CleanCode p.59) — the before/after is the `CommentWidget` class. Collapsed (bad):

```
public class CommentWidget extends TextWidget {
    public static final String REGEXP =
        "^#[^\\r\\n]*(?:(?:\\r\\n)|\\n|\\r)?";
    public CommentWidget(String text) { super(text); }
    public String render() throws Exception { return ""; }
}
```

Properly indented (good): identical members, but the constructor and `render()` bodies expand onto their own indented lines. Scope structure should be *visible*, even for one-statement bodies.

Team rules (CleanCode p.60) — the slide's full text: "every programmer has his own favorite formatting rules / but if he works in a team / then the team rules."

Objects & Data Structures — data abstraction

What it is. Data abstraction means exposing data through an interface that hides *how* it is stored, rather than publishing the raw representation. The deck contrasts a **concrete Point** exposing `public double x;` `public double y;` against an **abstract Point** interface exposing `getX()/getY()/setCartesian(...)/getR()/getTheta()/setPolar(...)` (CleanCode p.61). The abstract version never reveals whether a point is stored in Cartesian or polar form — it exposes *behaviour*, not representation. Likewise a concrete `Vehicle` exposing `getFuelTankCapacityInGallons()` + `getGallonsOfGasoline()` forces callers to do the division themselves and leaks that fuel is measured in gallons; a more abstract `getPercentFuelRemaining()` hides both the unit and the arithmetic (CleanCode p.62).

What it's used for / why it matters. Abstraction is what lets implementation change without breaking clients — the heart of maintainability. If every caller reads `point.x / point.y`, then switching the internal storage to polar coordinates breaks them all; if they call `getX()/getY()`, the switch is invisible. Exposing *concepts* (`getPercentFuelRemaining`) rather than *mechanics* (`getGallons ÷ getCapacity`) also raises the level of the API, so callers express intent instead of re-deriving it. Note the subtlety the deck makes: merely

wrapping fields in trivial getters/setters is *not* abstraction — true abstraction offers operations in terms of the problem domain, not just accessors over the representation.

When & how it's applied. *Bad (leaks representation):* `public double x, y; or getGallonsOfGasoline() / getTankCapacity()`. *Good (hides it):* a `Point` interface with `getX/getY/setPolar/...` and a `Vehicle` with `getPercentFuelRemaining()`. Apply whenever clients shouldn't depend on, or shouldn't have to know, how a value is computed or stored.

Objects & Data Structures — data/object anti-symmetry

What it is. A precise statement of the difference between the two ways to bundle data (CleanCode p.63): **objects** "hide their data behind abstractions and expose functions that operate on that data"; **data structures** "expose their data and have no meaningful functions." They are exact opposites — and, crucially, they make *opposite* kinds of change easy. Object-oriented (polymorphic) code makes it easy to add new **types** without changing existing functions, but hard to add a new *function* (you must touch every class). Procedural / data-structure code makes it easy to add new **functions** without changing existing data structures, but hard to add a new *type* (you must touch every function). This trade-off is the "anti-symmetry."

What it's used for / why it matters. Knowing the anti-symmetry lets you choose the right shape for the change you expect: if the system will grow by adding new *kinds of thing*, prefer objects/polymorphism (the OCP-friendly choice); if it will grow by adding new *operations* over a fixed set of things, a data structure with free functions may be simpler. The mistake the rule warns against is the **hybrid** — a class that both exposes its fields *and* has significant methods — which gets the worst of both: it is hard to add new functions *and* hard to add new types, and it tempts callers to reach inside (violating the Law of Demeter). The lesson is to commit to one or the other, not blur them.

When & how it's applied. *Object (good for new types):* a `Shape` hierarchy with `area()` overridden per shape — adding `Triangle` touches nothing else. *Data structure (good for new functions):* a plain `Shape` record switched over by free `area(shape)`, `perimeter(shape)` functions — adding `centroid(shape)` touches nothing else. *Bad hybrid:* a `Shape` with public fields *and* fat methods. Pick the form that matches the axis of change you expect.

Objects & Data Structures — the Law of Demeter (don't talk to strangers)

What it is. The Law of Demeter is a coupling rule: "talk only to your immediate friends." Formally, a method `m` of an object `O` may call methods only on `O` itself, on `m`'s parameters, on objects `m` creates, and on `O`'s direct component objects — but **not** on objects *returned by* those calls. The deck shows a violating chain `ctxt.getOptions().getScratchDir().getAbsolutePath()` — each call returns a stranger the next call then talks to. It also shows the **train wreck** form spelled out into locals (`Options opts = ctxt.getOptions(); File scratchDir = opts.getScratchDir(); ...`), which is the same coupling just laid flat, and the worst form, reaching through exposed fields `ctxt.options.scratchDir.absolutePath` (CleanCode p.64).

What it's used for / why it matters. Every link in a chain like `a.getB().getC().getD()` hard-codes a path through the *internal structure* of three other objects — so a change to `Options`' or `ScratchDir`'s shape ripples out to every method that navigated through it. The law limits that blast radius by forbidding code from depending on the internals-of-internals: a method should ask its immediate collaborator to do the work ("*tell, don't ask*") rather than fetching the collaborator's collaborator and operating on it. The benefit is lower coupling, which is precisely what keeps changes local.

When & how it's applied. *Bad:* `String path = ctxt.getOptions().getScratchDir().getAbsolutePath();` — couples the caller to the structure of `Options`

and `ScratchDir`. *Good*: give `ctxt` (or `Options`) the responsibility and call `ctxt.getScratchDirAbsolutePath()` or, better, `ctxt.createScratchFileStream(name)` so the caller never sees the intermediate objects at all. The deck notes the rule is clearest when these are true *objects* (with hidden data); pure data structures, which legitimately expose their fields, are not the target. (Watch for: chains where each step returns a *new* object you then drive — that's the smell.)

Maintenance link (inference): Law-of-Demeter violations are exactly what blow up the **estimated impact set** during Impact Analysis — a single change to `Options` ripples to every caller that train-wrecks through it.

Objects & Data Structures — the slide code transcribed (CleanCode p.61–64)

Concrete vs abstract Point (CleanCode p.61) — both declarations verbatim:

```
// Concrete Point
public class Point {
    public double x;
    public double y;
}

// Abstract Point
public interface Point {
    double getX();
    double getY();
    void setCartesian(double x, double y);
    double getR();
    double getTheta();
    void setPolar(double r, double theta);
}
```

Notice the abstract interface's *policy* details: reads are individual (`getX()`, `getY()`, `getR()`, `getTheta()`) but writes are **atomic pairs** (`setCartesian(x, y)`, `setPolar(r, theta)`) — you cannot set half a coordinate, so the interface enforces consistency the public-fields version cannot.

Concrete vs abstract Vehicle (CleanCode p.62) — verbatim:

```
// Concrete Vehicle
public interface Vehicle {
    double getFuelTankCapacityInGallons();
    double getGallonsOfGasoline();
}

// Abstract Vehicle
public interface Vehicle {
    double getPercentFuelRemaining();
}
```

The deck's subtlety: the "concrete" version *is already an interface* syntactically — yet it is still concrete in spirit, because its two accessors are a window straight onto the stored data (capacity and gallons). Abstraction is about *what the API reveals*, not about the `interface` keyword.

Data/Object Anti-Symmetry (CleanCode p.63) — the slide's exact wording: "objects hide their data behind abstractions and expose functions that operate on that data"; "data structures expose their data and have no meaningful functions."

Law of Demeter and Train Wrecks (CleanCode p.64) — all three forms verbatim:

```
// The Law of Demeter (violating chain)
final String outputDir = ctxt.getOptions()
    .getScratchDir()
    .getAbsolutePath();

// Train Wrecks (the same coupling laid flat)
Options opts = ctxt.getOptions();
File scratchDir = opts.getScratchDir();
final String outputDir = scratchDir.getAbsolutePath();

// worst form: reaching through exposed fields
final String outputDir = ctxt.options.scratchDir.absolutePath;
```

Error Handling — prefer exceptions to error codes

What it is. The rule to signal failure by *throwing an exception* rather than *returning an error code* the caller must check. Error codes force the caller to test the return of every call inline, which produces deeply nested `if (x == E_OK)` ladders that bury the happy path. The deck shows the nested mess `if (deletePage(page) == E_OK) { if (registry.deleteReference(...) == E_OK) { ... } }` (CleanCode p.65), replaced by a clean `try { deletePage(page); registry.deleteReference(page.name); ... } catch (Exception e) { logger.log(e.getMessage()); }` (CleanCode p.66).

What it's used for / why it matters. Exceptions *separate* the error-handling code from the business logic. With error codes the two are interleaved — every line of normal logic is wrapped in a status check, so the reader can't follow the main algorithm, and any caller who *forgets* a check silently proceeds with bad data. Throwing an exception lets the happy path read as a clean straight-line sequence, while the failure path is handled once in a `catch`; and an unhandled exception fails loudly rather than being silently ignored. This makes both the normal flow and the error flow easier to read, test, and maintain.

When & how it's applied. *Bad:* functions returning `E_OK` / error constants, checked by nested `if`s that dominate the code. *Good:* functions that `throw` on failure, called inside a single `try`, with one `catch` doing the recovery/logging. Apply whenever a function can fail in a way the caller shouldn't be allowed to forget.

Error Handling — extract try/catch blocks & error handling is one thing

What it is. Two refinements of exception handling.

- **Extract try/catch blocks:** the *bodies* of `try` and `catch` should be one-line calls to well-named extracted methods. The deck's `delete()` does nothing but `try { deletePageAndAllReferences(page); } catch (Exception e) { logError(e); }` — the real work lives in `deletePageAndAllReferences` and the recovery in `logError` (CleanCode p.67).
- **Error handling is one thing:** "if the keyword `try` exists in a function it should be the very first word, and there should be nothing after the catch/finally blocks" (CleanCode p.68). A function that handles errors should do *only* that.

What it's used for / why it matters. `try/catch` blocks are ugly and they confuse the structure of a method by mixing normal processing with error processing. Extracting the bodies, and making error handling its own dedicated function, applies "do one thing" to exceptions: a method either does the work *or* manages the failure of work, never both. This keeps each function tiny and single-purpose, lets the normal-logic function be read without the visual noise of error plumbing, and makes the error-handling policy easy to find and change in one place.

When & how it's applied. *Bad:* a long method with a big `try` containing twenty lines of logic and a `catch` containing more logic, followed by yet more statements after the block. *Good:* a thin `delete()` whose entire body is `try { doRealWork(); } catch (e) { handle(e); }`, with `try` as its first keyword and nothing after the `catch`.

Error Handling — define the normal flow

What it is. A rule for the case where an "error" is really just a routine alternative outcome — use the **Special Case pattern** (Fowler) so the caller has *no* special case to handle. The deck's example computes a meal total with a `try { expenses = expenseReportDAO.getMeals(employee.getID()); m_total += expenses.getTotal(); } catch (MealExpensesNotFound e) { m_total += getMealPerDiem(); }` (CleanCode p.69). The improvement: have `getMeals` *always* return a `MealExpenses` object, and in the "no meals" case return a special-case object whose `getTotal()` already yields the per-diem default — so the caller just writes `m_total += expenseReportDAO.getMeals(id).getTotal();` with no `try/catch` at all (CleanCode p.70).

What it's used for / why it matters. Sometimes throwing/catching an exception is overkill: the "missing meals" case isn't exceptional, it's a normal business rule (use the per-diem). Encoding that rule as a Special Case object pushes the branch *out of the caller* and *into the type system*, so every caller automatically gets the right default and none of them carries duplicated `catch` logic. This is the deck's "define the normal flow": arrange the design so the main path has no detours, rather than littering callers with handling for routine alternatives.

When & how it's applied. *Bad:* every caller of `getMeals` wraps it in `try/catch (MealExpensesNotFound)` and adds the per-diem itself — duplicated logic, easy to forget. *Good:* `getMeals` returns a `PerDiemMealExpenses` special-case object in the empty case; callers are oblivious. Use this when the "exceptional" branch is actually a common, well-defined business behaviour.

Error Handling — don't return null, don't pass null

What it is. Two distinct discipline rules about the `null` value.

- **Don't return null:** an API that can return `null` forces *every* caller to remember a `if (x != null)` guard, and one forgotten guard is a `NullPointerException` in production. The deck's `if (employees != null) for (Employee e : employees) ...` is fixed by having the method `return Collections.emptyList();` instead of `null`, so callers can iterate unconditionally (CleanCode p.71–72).
- **Don't pass null:** passing `null` *into* a method is even worse, because the method usually can't do anything sensible with it. `xProjection(Point p1, Point p2)` either dereferences `null` and crashes, or must defend with `if (p1 == null || p2 == null) throw new IllegalArgumentException(...)` — clutter that exists only because someone might pass `null`. The rule: forbid `null` arguments by convention so the defensive checks aren't needed (CleanCode p.73).

What it's used for / why it matters. `null` is a special value that *isn't* an object, so it slips past the type system and explodes at the worst time. Returning an empty collection (or a Special Case object) instead of `null` removes a whole category of caller bugs and lets the normal path stay branch-free. Refusing to pass `null` keeps methods free of paranoid `null`-checks and makes their preconditions honest. Together these two rules eliminate most `NullPointerExceptions` at the source — which is why the deck lists them as *two separate* rules to remember.

When & how it's applied. *Return-null bad:* `List<Employee> getEmployees()` that returns `null` when there are none. *Good:* return `Collections.emptyList()` (or `Optional` in modern Java). *Pass-null bad:* calling

`xProjection(null, p2)`. *Good*: treat null arguments as a programming error and never pass them; design APIs so null is never a valid input.

Error Handling — the slide code transcribed (CleanCode p.65–73)

Error codes, the nested mess (CleanCode p.65) — full transcription:

```
if (deletePage(page) == E_OK) {
    if (registry.deleteReference(page.name) == E_OK) {
        if (configKeys.deleteKey(page.name.makeKey()) == E_OK){
            logger.log("page deleted");
        } else {
            logger.log("configKey not deleted");
        }
    } else {
        logger.log("deleteReference from registry failed");
    }
} else {
    logger.log("delete failed");
    return E_ERROR;
}
```

Three operations, four logging branches, three levels of nesting — and the *happy path* (delete page, delete reference, delete key) is shredded across the condition heads.

The exception version (CleanCode p.66) — full transcription; the happy path becomes three straight lines:

```
try {
    deletePage(page);
    registry.deleteReference(page.name);
    configKeys.deleteKey(page.name.makeKey());
}
catch (Exception e) {
    logger.log(e.getMessage());
}
```

Extract try/catch blocks (CleanCode p.67) — the complete three-method refactor:

```
public void delete(Page page) {
    try {
        deletePageAndAllReferences(page);
    } catch (Exception e) {
        logError(e);
    }
}

private void deletePageAndAllReferences(Page page) throws Exception {
    deletePage(page);
    registry.deleteReference(page.name);
    configKeys.deleteKey(page.name.makeKey());
}

private void logError(Exception e) {
    logger.log(e.getMessage());
}
```

`delete()` now does exactly one thing: manage the failure of `deletePageAndAllReferences`. The work and the recovery each live in their own one-purpose method.

Error handling is one thing (CleanCode p.68) — the slide's exact bullet text: "functions should do one thing / error handling is one thing / if the keyword `try` exists in a function it should be the very first word in the function and ... there should be nothing after the catch/finally blocks."

Define the normal flow (CleanCode p.69–70) — before:

```
try {
    MealExpenses expenses = expenseReportDAO
        .getMeals(employee.getID());
    m_total += expenses.getTotal();
} catch(MealExpensesNotFound e) {
    m_total += getMealPerDiem();
}
```

after (the caller's entire handling apparatus disappears):

```
MealExpenses expenses = expenseReportDAO
    .getMeals(employee.getID());
m_total += expenses.getTotal();
```

The per-diem rule now lives inside the special-case `MealExpenses` object that `getMeals` returns when there are no meals — Fowler's **Special Case** pattern.

Don't return null (CleanCode p.71–72) — before:

```
List<Employee> employees = getEmployees();
if (employees != null) {
    for(Employee e : employees) {
        totalPay += e.getPay();
    }
}
```

after — the guard vanishes from *every* caller because the provider stops returning null:

```
List<Employee> employees = getEmployees();
for(Employee e : employees) {
    totalPay += e.getPay();
}

public List<Employee> getEmployees() {
    if( .. there are no employees .. )
        return Collections.emptyList();
}
```

Don't pass null (CleanCode p.73) — both versions verbatim:

```
public double xProjection(Point p1, Point p2) {
    return (p2.x - p1.x) * 1.5;
}

public double xProjection(Point p1, Point p2) {
    if (p1 == null || p2 == null) {
        throw new IllegalArgumentException("Invalid argument
```

```

        for MetricsCalculator.xProjection");
    }
    return (p2.x - p1.x) * 1.5;
}

```

The first crashes with a `NullPointerException` if handed null; the second defends, but at the price of clutter that exists *only because callers might pass null*. The deck's conclusion: forbid passing null by convention, so neither failure mode nor clutter is needed.

Unit Tests — the Three Laws of TDD

What it is. Test-Driven Development is the practice of writing the test *before* the production code, governed by three interlocking laws (CleanCode p.74): 1. **First law:** "you may not write production code until you have written a failing unit test." 2. **Second law:** "you may not write more of a unit test than is sufficient to fail, and not compiling is failing." 3. **Third law:** "you may not write more production code than is sufficient to pass the currently failing test."

Together they force a tight **red-green-refactor** cycle measured in seconds: write a tiny failing test (red), write just enough code to pass it (green), then clean up (refactor) — repeat.

What it's used for / why it matters. The laws guarantee two things that are gold for maintenance. First, you end up with a comprehensive suite of tests that covers essentially all the production code, because no production code was written that wasn't demanded by a test — so the suite becomes a *safety net* that catches regressions the moment a future change breaks something. Second, writing the test first forces you to design the code from the *caller's* point of view, which tends to produce smaller, decoupled, more testable units. The net effect is that you can change the system fearlessly: the tests tell you instantly if you broke it, which is exactly what makes ongoing maintenance affordable.

When & how it's applied. In practice you never write more than a few lines before switching between test and code: write `assertEquals(4, add(2,2))` (fails to compile — that counts as failing), write a stub `add` returning 0 (now compiles, test fails), implement `add` to return `a+b` (test passes), refactor, then write the next failing test. Apply it to any new behaviour or bug fix (write a failing test that reproduces the bug first).

Unit Tests — keeping tests clean, one assert / one concept, F.I.R.S.T.

What it is. Rules for the *quality* of test code, not just its existence.

- **Keeping tests clean:** "test code is just as important as production code" — it must be kept clean, because dirty tests rot and get abandoned, and abandoned tests are *worse* than no tests (you lose the safety net but still pay to maintain them) (CleanCode p.75).
- **Clean tests:** the single most important quality of a test is "three things: readability, readability, and readability" (CleanCode p.75).
- **One assert per test** (ideal): a test should "come to a single conclusion that is quick and easy to understand," so when it fails you know immediately what broke (CleanCode p.76).
- **Single concept per test** (the practical rule): "minimize the number of asserts per concept and test just one concept per test function" — one test verifies one idea (CleanCode p.76).
- **F.I.R.S.T.** — clean tests are **F**ast (run quickly, so you run them often), **I**ndependent (don't depend on each other or on order), **R**epeatable (same result in any environment), **S**elf-validating (a boolean pass/fail, no manual inspection), and **T**imely (written just before the code they test) (CleanCode p.77).

What it's used for / why it matters. A test suite is only valuable if it is *trusted and run*. Dirty, slow, order-dependent, or flaky tests get ignored, and an ignored suite stops catching regressions. Each F.I.R.S.T. property

removes a reason a team would stop running the tests: *Fast* means they run on every save; *Independent* and *Repeatable* mean a failure points at a real bug, not at test-order or environment noise; *Self-validating* means no human has to eyeball output; *Timely* keeps the code testable. "One concept per test" makes a failure diagnostic — you learn *what* broke from *which* test failed, not just *that* something broke. Clean tests are therefore the precondition for the whole TDD safety net to keep working over a project's life.

When & how it's applied. *Bad*: one giant test method with twelve asserts spanning three behaviours, that hits a real database and only passes when run second. *Good*: three small, well-named tests (`addsTwoPositives`, `addsNegative`, `addsZero`), each asserting one concept, each running in milliseconds against in-memory fixtures, each passing in isolation and in any order.

Verification link (inference): F.I.R.S.T. and the Three Laws are the micro-foundation of the course's **Verification** phase and of Continuous Integration's "self-testing build." Fast, independent, self-validating tests are what make a CI build trustworthy.

Classes — organization, small, SRP, cohesion

What it is. Four rules for the class as a unit.

- **Class organization** — the standard top-to-bottom order: public static constants → private static variables → private instance variables → public methods, with each private utility placed right after the public method that uses it (so the file reads in stepdown order) (CleanCode p.78).
- **Classes should be small** — "the first rule is that they should be small; the second rule is that they should be smaller than that," but a class's size is measured in **responsibilities**, not lines of code (CleanCode p.79).
- **The Single Responsibility Principle (SRP)** — "a class or module should have one, and only one, reason to change," which Martin calls "one of the more important concepts in OO design" (CleanCode p.79).
- **Cohesion** — "maintaining cohesion results in many small classes"; a class is cohesive when its methods all use its instance variables, and splitting a low-cohesion class along the lines of which methods use which variables naturally yields several small, focused classes (CleanCode p.79).

What it's used for / why it matters. These rules are SRP applied at the class level, and SRP is the load-bearing idea: a class with many responsibilities has many reasons to change, so unrelated changes collide in the same file, and a change for one reason risks breaking the others (fragility). Keeping classes small and single-responsibility means each change touches one well-named class, the system grows by *adding* classes rather than *bloating* existing ones, and you can understand any class in isolation. Cohesion is the measurable property that drives this: low cohesion is the signal that a class should be split. Standard organization makes any class navigable by convention. Together they keep the Impact Set of a change small and the codebase comprehensible.

When & how it's applied. *Bad*: a 2,000-line `SuperDashboard` class that handles UI layout, data fetching, and persistence (three reasons to change). *Good*: split it by responsibility into `Dashboard`, `DataProvider`, and `Repository`, each small and cohesive. The trigger to split is a name like `...Manager` / `...Processor`, a class you can't describe without "and," or methods that fall into disjoint groups by which fields they touch (low cohesion).

Emergence — the four Rules of Simple Design

What it is. Kent Beck's four "rules of simple design," which the deck presents as the way good design *emerges* from disciplined practice rather than being imposed up front. Apply them **in priority order** (CleanCode p.80): 1. **Runs all the tests** — the design must be verifiable; a system you can't test isn't known to work, so this comes first. 2. **No duplication** — eliminate redundancy (DRY); duplication is the enemy of clean design. 3. **Expressive** — the code clearly communicates the author's intent through good names, small functions, and standard structure. 4. **Minimal classes and methods** — keep the count of classes and methods small, but only *after* satisfying the first three.

What it's used for / why it matters. These four rules are a complete, prioritised recipe for refactoring toward good design without needing a grand architecture diagram. The ordering is the point: testability first (so you can refactor safely), then remove duplication and maximise expressiveness (the bulk of the cleanup), and only then trim incidental classes/methods — never sacrificing clarity or test coverage just to reduce the count. The deep claim, which this slide embodies, is that if you relentlessly apply these *small* rules during ordinary work, sound *architecture* emerges as a by-product. This makes the slide the explicit bridge from the micro level (clean code) to the macro level (good design / OO principles) that the rest of the lecture covers.

When & how it's applied. During the refactor step of any change: first make sure the tests still pass, then hunt and remove duplication, then improve names and split overlong functions for expressiveness, and finally collapse any classes/methods that no longer earn their keep — in that order. *Bad*: deleting a "redundant-looking" class for rule 4 while breaking a test (rule 1) — the priorities forbid it. *Good*: a green test suite, no duplicated logic, intention-revealing names, and no gratuitous extra abstractions.

Unit Tests, Classes & Emergence — the exact slide wording (CleanCode p.74–81)

For memorization-grade precision, the closing slides' literal text.

The Three Laws of TDD (CleanCode p.74) — the deck's phrasing, law by law:

```
// first law
//  you may not write production code until
//  you have written a failing unit test
// second law
//  you may not write more of a unit test
//  than is sufficient to fail, and not compiling is failing
// third law
//  you may not write more production code
//  than is sufficient to pass the currently failing test
```

The second law's tail — "**and not compiling is failing**" — is the detail exams like: a test that references a class or method that doesn't exist yet *is already a failing test*, so you may immediately switch to production code and create the stub.

Keeping tests clean / Clean tests (CleanCode p.75) — verbatim: "test code is just as important as production code" and "what makes a clean test? three things: readability, readability, and readability."

One assert / single concept (CleanCode p.76) — verbatim: "tests come to a single conclusion that is quick and easy to understand" (One Assert per Test); "the best rule is that you should minimize the number of asserts per concept and test just one concept per test function" (Single Concept per Test). Note the deck's nuance: *one assert* is the ideal, *one concept* is the rule — multiple asserts in one test are acceptable when they all verify the same single concept.

F.I.R.S.T. (CleanCode p.77) — the slide is exactly five lines: Fast / Independent / Repeatable / Self-validating / Timely.

Class organization (CleanCode p.78) — the slide's exact ordering list:

```
// public static constants
// private static variables
// private instance variables
// public functions
// private utilities called by a public function right after [it]
```

(Variables descend by visibility, then functions follow the stepdown pattern — each private helper directly after the public function that calls it.)

Classes should be small / SRP / Cohesion (CleanCode p.79) — verbatim: "the first rule is that they should be small / the second rule is that they should be smaller than that" (deliberately echoing the Functions rule on p.23); "a class or module should have one, and only one, reason to change"; "SRP is one of the more important concept[s] in OO design"; "maintaining cohesion results in many small classes."

Emergence (CleanCode p.80) — the slide gives the four rules as four handwritten lines, each titled "Simple Design Rule N": "Simple Design Rule 1: Runs All the Tests / Simple Design Rule 2: No Duplication / Simple Design Rule 3: Expressive / Simple Design Rule 4: Minimal Classes and Methods."

Reference (CleanCode p.81) — the deck's lone bibliography slide: "Clean code: a handbook of agile software craftsmanship. R. Martin, and J. Coplien. Prentice Hall, Upper Saddle River, NJ etc., (2009)." Also worth knowing: the deck's *title* slide credits the course lecturer (CleanCode p.1) — the deck is the course's curated tour of the book, not Martin's own slide set.

Macro level: Symptoms of rotting design

What it is. A diagnostic vocabulary for *why* maintenance gets expensive — four observable symptoms that signal a design is "rotting," each ultimately traceable to bad dependencies (DesignPrinciplesAndPatterns p.2–3):

- **Rigidity** — "the tendency for software to be difficult to change, even in simple ways. Every change causes a cascade of subsequent changes in dependent modules." A change estimated at two days balloons into "a multi-week marathon" because each fix forces the next (p.2).
- **Fragility** — "the tendency of the software to break in many places every time it is changed," often in modules "that have no conceptual relationship with the area that was changed." As fragility worsens, the probability that a change breaks something "asymptotically approach[es] 1" (p.2).
- **Immobility** — "the inability to reuse software from other projects," because the part you'd like to reuse "has too much baggage that it depends upon"; the team finds it cheaper to rewrite than to disentangle and reuse (p.3).
- **Viscosity** — two forms: **viscosity of design** (the design-preserving way to make a change is *harder* than a quick hack, so "it is easy to do the wrong thing, but hard to do the right thing," and developers slowly degrade the design), and **viscosity of environment** (slow builds / clumsy source control tempt developers into shortcuts that also degrade the design) (p.3).

What it's used for / why it matters. This vocabulary lets you *name and diagnose* the pain a maintenance team feels, and trace it to a cause you can fix. The key teaching is the shared **root cause**: all four "are either directly, or indirectly caused by improper dependencies between the modules" (p.4). So you don't fix rigidity

by working harder — you fix it by repairing the dependency structure. The cure is **dependency management**: building "dependency firewalls" across which "dependencies do not propagate" (p.4), and the OO principles that follow (OCP, LSP, DIP, ISP, and the package principles) are precisely those firewalls.

When & how it's applied. Use the symptoms as a checklist when a codebase feels expensive: *Does a small change cascade?* (rigidity) *Does unrelated code break?* (fragility) *Can't lift a useful class out for another project?* (immobility) *Is the hack easier than the clean change?* (viscosity). Each diagnosis points you at the dependency that needs inverting or breaking. **Don't confuse rigidity (changes cascade — hard to change) with fragility (changes break unexpectedly — change in one place, break in another)** — a common exam trap.

Macro level: the full rot narrative — redesigns, roach motels, and changing requirements (DesignPrinciplesAndPatterns p.1–4)

What it is. The chapter's opening pages contain more exam-worthy detail than the four symptom definitions alone. In full:

- **The architecture altitude framing (p.1).** "What is software architecture? The answer is multitiered." At the highest level sit the **architecture patterns** that define the overall shape of applications (credited to [Shaw96]); a level down is the architecture specific to the application's purpose; "yet another level down resides the architecture of the modules and their interconnections — **the domain of design patterns, packages, components, and classes**" — and *that* is the chapter's scope. The chapter explicitly limits itself and points to [Martin99] for more.
- **The rot story (p.1–2).** A design "begins as a vital image in the minds of its designers," at which stage it is "clean, elegant, and compelling," with "a simple beauty that makes the designers and implementers itch to see it working." Some applications keep that purity through the first release — "but then something begins to happen. The software starts to rot." First "an ugly wart here, a clumsy hack there," then "the ugly festering sores and boils accumulate until they dominate the design," and the program becomes "a festering mass of code" ever harder to maintain, until engineers and front-line managers "cry for a redesign project."
- **Why redesigns rarely succeed (p.2).** The redesign team is "shooting at a moving target": the old system keeps evolving, and the new design must keep chasing it. "The warts and ulcers accumulate in the new design before it ever makes it to its first release," and on that fateful day — "usually much later than planned" — the new design may already be so problematic that its designers "are already crying for another redesign." This is the chapter's argument that **you cannot out-rewrite rot; you must prevent it with dependency management.**
- **Rigidity's management spiral (p.2).** Rigid software makes managers "fear to allow engineers to fix non-critical problems," because nobody can predict when the engineers will finish: the design "begins to take on some characteristics of a **roach motel — engineers check in, but they don't check out.**" When the fear becomes acute, managers *refuse* changes outright, and "**official rigidity** sets in: what starts as a design deficiency winds up being adverse management policy."
- **Fragility's trust collapse (p.2–3).** As fragility worsens, "every fix makes it worse, introducing more problems than are solved," the probability of breakage "asymptotically approach[es] 1," and managers and customers come to "suspect that the developers have lost control of their software. Distrust reigns, and credibility is lost."
- **Immobility's economics (p.3).** An engineer discovers a colleague's module that is *almost* what they need — but it carries "too much baggage that it depends upon," and after much work the engineers find "the work and risk required to separate the desirable parts ... from the undesirable parts are too great to tolerate. And so the software is simply **rewritten instead of reused.**"

- **Viscosity of environment, concretely (p.3).** Long compile times tempt engineers into changes that avoid large recompiles; a source-control system that takes hours to check in a few files tempts them into changes that minimize check-ins — in both cases "regardless of whether the design is preserved." Tooling friction *causes* design decay.
- **Changing Requirements (p.3–4).** The immediate cause of degradation: "the requirements have been changing in ways that the initial design did not anticipate," often via quick changes "made by engineers who are not familiar with the original design philosophy," whose individually-working fixes "somehow violate the original design. Bit by bit ... these violations accumulate until **malignancy sets in.**" But the chapter refuses the excuse: "we cannot blame the drifting of the requirements" — "the requirements document is the most volatile document in the project," and "**if our designs are failing due to the constant rain of changing requirements, it is our designs that are at fault.**" Designs must be made resilient to change.

Why it matters. This narrative is the course's motivation for the whole lecture in Martin's own words: maintenance pressure (changing requirements) is a constant; rot is the default outcome; redesign is not an exit; therefore the only sustainable strategy is to manage dependencies continuously with the principles that follow. For the exam, be able to retell the causal chain: changing requirements → quick changes by design-unfamiliar engineers → unplanned dependencies → the four symptoms → dependency firewalls (OCP/LSP/DIP/ISP + package principles) as the cure (DesignPrinciplesAndPatterns p.3–4).

OCP — The Open/Closed Principle

"A module should be open for extension but closed for modification." (DesignPrinciplesAndPatterns p.4)

What it is. The OCP, the **most important** OO principle (p.4, originating with Bertrand Meyer), says a module should be extendable to do new things *without editing its existing source code*. "Open for extension" means its behaviour can be augmented; "closed for modification" means you achieve that augmentation without touching (and risking) the code that already works. The enabling mechanism is **abstraction** — depend on an abstract interface, and add new behaviour by adding new implementations of that interface:

- **Dynamic polymorphism:** the bad `LogOn` function `if (m.type == Modem::hayes) DialHayes(...)` `else if (... courrier) ... else if (... ernie) ...` must be modified for every new modem and recompiled (p.5). Programs like this "tend to be littered with similar if/else or switch statements" (p.6). The OCP fix: depend only on a `Modem` **interface** (`Dial/Send/Recv/Hangup`), so `LogOn` calls `m.Dial(pno)` and new modems require *no change* to `LogOn` (p.6–7).
- **Static polymorphism:** templates/generics achieve the same at compile time — `template<typename MODEM> void LogOn(MODEM& m, ...)` (p.7).
- **Architectural goal:** add new features "by only adding new code" — "if you don't have to change working code, you aren't likely to break it." Even *partial* OCP compliance "can make dramatic improvements" (p.7).

What it's used for / why it matters. OCP is the direct cure for **rigidity** and **fragility**: when new requirements arrive as *new code* rather than *edits to old code*, existing tested modules stay untouched, so they can't regress and changes don't cascade. It is the architectural goal the rest of the principles serve — DIP is its mechanism, LSP its precondition, the patterns its concrete realisations. The pragmatic caveat is that you cannot close a module against *every* possible change; you predict the *likely* axes of change and abstract along those, accepting that an unforeseen kind of change may still force an edit.

When & how it's applied. Apply OCP wherever you anticipate adding new variants of a thing: new device types, new file formats, new report kinds, new shapes. Introduce an abstract base/interface for the varying

behaviour, route all clients through it, and implement each variant as a new subclass. *Bad*: a `switch (modem.type)` that grows a case per new modem. *Good*: a `Modem` interface that each new modem class implements, leaving `LogOn` forever closed.

Cross-link: this is the Clean-Code "replace switch with polymorphism" rule (CleanCode p.25–26) at the architectural scale.

OCF — the modem listings in full (DesignPrinciplesAndPatterns p.5–7)

The chapter's OCF example is given as three numbered listings plus a figure; here they are in full, because the exam can show any fragment and ask what principle/technique it illustrates.

Listing 2-1 — "Logon, must be modified to be extended" (p.5):

```
struct Modem
{
    enum Type {hayes, courier, ernie} type;    // (sic - typo in the original)
};
struct Hayes
{
    Modem::Type type;
    // Hayes related stuff
};
struct Courier
{
    Modem::Type type;
    // Courier related stuff
};
struct Ernie
{
    Modem::Type type;
    // Ernie related stuff
};
void LogOn(Modem& m,
    string& pno, string& user, string& pw)
{
    if (m.type == Modem::hayes)
        DialHayes((Hayes&)m, pno);
    else if (m.type == Modem::courier)
        DialCourier((Courier&)m, pno);
    else if (m.type == Modem::ernie)
        DialErnie((Ernie&)m, pno)
    // ...you get the idea
}
```

Two distinct costs are called out (p.5): `LogOn` "must be changed every time a new kind of modem is added," **and** — subtler — because every modem struct embeds the shared `Modem::Type` enumeration, "**each modem must be recompiled every time a new kind of modem is added.**" The type-code enum couples all variants to each other through their shared discriminator.

The local-optimization trap (p.6). Worse than the visible if/else chains: programmers compress them with "local optimizations that hide the structure of the selection statements." If Hayes and Courier happen to share a send function, you find:

```
if (modem.type == Modem::ernie)
    SendErnie((Ernie&)modem, c);
```

```
else
    SendHayes((Hayes&)modem, c);
```

The `else` silently bundles *two* types (and any future type!) into the Hayes branch — "such structures make the system much harder to maintain, and are very prone to error" (p.6). Adding a fourth modem type to this code does the wrong thing *without failing visibly*.

Figure 2-13 + Listing 2-2 — "LogOn has been closed for modification" (p.6–7). The figure shows `LogOn` («function») depending only on a `Modem` «interface» with operations `+ Dial(pno) + Send(char) + Recv() : char + Hangup()`, beneath which hang `Hayes Modem`, `Courrier Modem`, and `Ernie's Modem` as implementations:

```
class Modem
{
public:
    virtual void Dial(const string& pno) = 0;
    virtual void Send(char) = 0;
    virtual char Recv() = 0;
    virtual void Hangup() = 0;
};
void LogOn(Modem& m,
    string& pno, string& user, string& pw)
{
    m.Dial(pno);
    // you get the idea.
}
```

Listing 2-3 — static polymorphism (p.7):

```
template <typename MODEM>
void LogOn(MODEM& m,
    string& pno, string& user, string& pw)
{
    m.Dial(pno);
    // you get the idea.
}
```

Templates/generics close `LogOn` at **compile time**: any type that *has* a `Dial` works, with no virtual dispatch and no common base class — the trade-off being that the binding can no longer change at run time. Know both mechanisms as the deck's two named OCP techniques: **dynamic polymorphism** (abstract base class + virtual functions) and **static polymorphism** (templates/generics) (p.5–7).

Architectural goals, verbatim (p.7): "with a little forethought, we can add new features to existing code, **without changing the existing code and by only adding new code.**" OCP is "an ideal that can be difficult to achieve," demonstrated "several times, in the case studies later on in this book," but "even partial OCP compliance can make dramatic improvements ... If you don't have to change working code, you aren't likely to break it."

LSP — The Liskov Substitution Principle

"Subclasses should be substitutable for their base classes." (DesignPrinciplesAndPatterns p.8)

What it is. The LSP (coined by Barbara Liskov, related to Meyer's **Design by Contract**) defines *what makes inheritance valid*: "a user of a base class should continue to function properly if a derivative of that base class is passed to it" (p.8). In other words, a subtype must be usable *anywhere* its base type is expected, without the client noticing or breaking. The canonical violation is the **Circle/Ellipse dilemma** (p.9–11): a `Circle` *is-a* `Ellipse` mathematically, so inheritance is tempting, but `Circle` overriding `SetFoci` to keep both foci coincident **breaks the contract** that callers of `Ellipse.SetFoci(a,b)` rely on (`assert(e.GetFocusA()==a)` fails for a `Circle`) (p.10). "Clients ruin everything" — the model looks self-consistent until a real client exercises the inherited contract.

Design by Contract restatement of LSP (p.11): a derived class is substitutable iff 1. its **preconditions are no stronger** than the base method's (it must accept everything the base accepted — *expect no more*), and 2. its **postconditions are no weaker** than the base method's (it must guarantee everything the base guaranteed — *provide no less*). "Derived methods should expect no more and provide no less."

What it's used for / why it matters. LSP is the rule that keeps **OCP** working. OCP relies on clients programming to an abstraction and any subtype slotting in unnoticed; if a subtype violates its base's contract, that promise breaks and the client must be modified to cope with the rogue subtype. That is why **LSP violations are latent OCP violations** — the ugly `if (typeid(e) == typeid(Ellipse))` guard a client adds to defend itself against the misbehaving `Circle` *is itself* an OCP violation that must be revisited for every new subtype (p.11–12). LSP also reframes inheritance: it models **behavioural substitutability**, not real-world is-a taxonomy — which is why "a `Circle` is an `Ellipse`" being true in mathematics does not make the inheritance valid in code.

When & how it's applied. Before subclassing, check the contract: can the subtype honour every precondition and postcondition the base advertises? If overriding a method would tighten what it accepts or weaken what it guarantees (e.g. a `Square` that constrains `setWidth` / `setHeight` its `Rectangle` base left independent), inheritance is wrong — prefer composition or a common abstraction over a forced is-a. *Bad*: `Circle` extends `Ellipse` overriding `SetFoci` to defeat it. *Good*: a shared `Shape` / `ConicSection` abstraction where neither imposes the other's contract.

LSP — the full Circle/Ellipse mechanics, listing by listing (DesignPrinciplesAndPatterns p.8–12)

The schema (p.8). Figure 2-14 is the minimal LSP picture: a `User` depends on `Base`; `Derived` inherits from `Base`. Listing 2-4 makes it concrete: given `void User(Base& b);`, "it should be legal" to write `Derived d; User(d);`. Substitutability means *that call must work* — for every `Derived` anyone ever writes.

The Ellipse declaration (p.9, Figure 2-16). `Ellipse` has exactly three data members and eight operations:

```
Ellipse
- itsFocusA   : Point
- itsFocusB   : Point
- itsMajorAxis : double
+ Circumference() : double
+ Area() : double
+ GetFocusA() : Point
+ GetFocusB() : Point
+ GetMajorAxis() : double
+ GetMinorAxis() : double
+ SetFoci(a:Point, b:Point)
+ SetMajorAxis(double)
```

First objection (p.9): if `Circle` inherits from `Ellipse` it drags in all three data members, although "Circle really only needs two data elements, a center point and a radius" — a space overhead the chapter says we could *tolerate* if behaviour were right.

Listing 2-5 — "Keeping the Circle Foci coincident" (p.10):

```
void Circle::SetFoci(const Point& a, const Point& b)
{
    itsFocusA = a;
    itsFocusB = a;    // note: 'a' twice - the second argument is ignored
}
```

This makes `Circle` *internally* self-consistent (either focus acts as the center, the major axis as the diameter), and the chapter concedes: "the model we have created is self consistent ... There is nothing you can do to it to make it violate those rules" (p.10). The flaw is invisible until a client appears.

"Clients Ruin Everything" — the contract fragment (p.10). Users of `Ellipse` "have the right to expect the following code fragment to succeed":

```
void f(Ellipse& e)
{
    Point a(-1,0);
    Point b(1,0);
    e.SetFoci(a,b);
    e.SetMajorAxis(3);
    assert(e.GetFocusA() == a);
    assert(e.GetFocusB() == b);
    assert(e.GetMajorAxis() == 3);
}
```

Pass an `Ellipse`: fine. Pass a `Circle`: the second assert fails (`GetFocusB()` returns `a`, not `b`). Made explicit, `SetFoci`'s implied **postcondition** is "the input values got copied to the member variables, and the major-axis variable was left unchanged" — and `Circle` "violates this guarantee because it ignores the second input variable of `SetFoci`" (p.10).

Design by Contract, the precise definitions (p.11). Meyer "has invented a language named **Eiffel** in which contracts are explicitly stated for each method, and explicitly checked at each invocation"; everyone else "ha[s] to make do with simple assertions and comments." Definitions worth quoting: the **precondition** is "what must be true before the method is called — if the precondition fails, the results of the method are undefined, and the method ought not be called"; the **postcondition** is "what the method guarantees will be true once it has completed — a method that fails its postcondition should not return." Then the two-clause substitutability test (preconditions no stronger / postconditions no weaker) and the slogan "derived methods should expect no more and provide no less."

Repercussions + Listing 2-6 (p.11–12). "LSP violations are difficult to detect until it is too late" — and once the design is heavily used, "it might not be economical to go back and change the design, and then rebuild and retest all the existing clients." So the discovering client patches *itself*:

```
void f(Ellipse& e)
{
    if (typeid(e) == typeid(Ellipse))
    {
        Point a(-1,0);
        Point b(1,0);
        e.SetFoci(a,b);
    }
}
```



```

        e.SetMajorAxis(3);
        assert(e.GetFocusA() == a);
        assert(e.GetFocusB() == b);
        assert(e.GetMajorAxis() == 3);
    }
    else
        throw NotAnEllipse(e);
}

```

The chapter's verdict: "Careful examination of Listing 2-6 will show it to be a violation of the OCP. Now, whenever some new derivative of `Ellipse` is created, this function will have to be checked to see if it should be allowed to operate upon it. Thus, **violations of LSP are latent violations of OCP**" (p.12). Note the exact-type check `typeid(e) == typeid(Ellipse)` even *rejects future well-behaved subtypes* — the defensive patch is maximally closed against extension.

DIP — The Dependency Inversion Principle

"Depend upon Abstractions. Do not depend upon concretions." (DesignPrinciplesAndPatterns p.12)

What it is. The DIP says high-level policy should not depend on low-level detail; both should depend on **abstractions**. "If the OCP states the goal of OO architecture, the DIP states the primary mechanism" (p.12). In a naive procedural design, dependencies point *downward* — the high-level policy calls, and therefore depends on, the concrete low-level modules (p.12–13). OO **inverts** that arrow: the low-level details depend on an abstract interface *owned by* the high-level policy, not the other way round. The rule, stated absolutely: "Every dependency in the design should target an interface, or an abstract class. No dependency should target a concrete class" (p.12).

What it's used for / why it matters. The rationale is volatility: "concrete things change a lot, abstract things change much less frequently," so by pointing every dependency at the stable abstraction you stop the volatile details from rippling upward into your policy. Abstractions become **"hinge points"** where the design "can bend or be extended, without themselves being modified (OCP)" (p.13) — DIP is literally *how* you achieve OCP. The inversion also liberates the high-level policy for reuse: because it depends only on interfaces it owns, it can be lifted into a new context and given fresh low-level implementations (curing **immobility**).

When & how it's applied. Whenever a high-level module would otherwise `new` or directly reference a concrete service (a database, a device, a third-party lib), define an abstract interface for what the policy *needs*, have the policy depend on that, and have the concrete service implement it. *Bad:* `OrderService` directly instantiates and calls `MySQLOrderRepository`. *Good:* `OrderService` depends on an `OrderRepository` interface; `MySQLOrderRepository` implements it and is injected in. **Mitigating forces** (p.14): the DIP assumes concrete = volatile, so depending on a *stable* concretion (e.g. `string.h`, the standard library) is acceptable. And **object creation** is the one unavoidable place you must name a concrete class — quarantined by the **Abstract Factory** pattern (p.14).

DIP — the two figures, COM, and the mitigating forces in detail (DesignPrinciplesAndPatterns p.12–14)

The two dependency-structure figures (p.12–13). Figure 2-17, "Dependency Structure of a Procedural Architecture": `main` at the top depends on mid-level modules (`mid 1`, `Mid 2`, `Mid 3`), which depend on `Detail` modules at the bottom — every arrow points *downward from policy to detail*. Figure 2-18, "Dependency Structure of an Object Oriented Architecture": `High level Policy` depends on `Abstract Interface s`, and the `Detailed Implementation s` *also* depend (upward!) on those same abstract interfaces —

the details' arrows have been **inverted** to point at abstractions. Being able to sketch and label these two diagrams is a plausible exam task: the OO picture is policy → interface ← implementation; nothing points at a concretion.

Why "inversion" (p.12). The chapter's framing: the high-level modules "deal with the high level policies of the application. These policies generally care little about the details that implement them. Why then, must these high level modules directly depend upon those implementation modules?" In the OO structure "the modules that contain detailed implementation are no longer depended upon, rather they depend themselves upon abstractions" — the dependency *upon the details* has been inverted into a dependency *of the details*.

DIP as enabling technology (p.12, p.14). The chapter twice ties DIP to component substrates: dependency inversion "is the enabling force behind **component design, COM, CORBA, EJB**, etc." (p.12), and "substrates such as **COM** enforce this principle, at least between components. The only visible part of a COM component is its abstract interface. Thus, in COM, there is little escape from the DIP" (p.14). The point to carry into the exam: DIP is not just a style preference — it is what makes *binary, independently-deployed components* possible at all, because components can only couple through interfaces.

Mitigating forces, in full (p.13–14). The absolute rule ("no dependency should target a concrete class") is conceded to be "draconian." The DIP's hidden assumption is "anything concrete is volatile" — usually true in early development, but with exceptions: "the `string.h` standard C library is very concrete, but is not at all volatile," so depending on it "in an ANSI string environment is not harmful"; likewise "tried and true modules that are concrete, but not volatile" are acceptable dependencies. Two cautions follow. First: even that can sour — "a dependency upon `string.h` could turn very ugly when the requirements ... forced you to change to UNICODE characters." Second, the principle-grade sentence: "**Non-volatility is not a replacement for the substitutability of an abstract interface**" — a stable concretion still cannot be *swapped*, so you lose extension even if you escape volatility.

Object creation, in full (p.14). "By definition, you cannot create instances of abstract classes. Thus, to create an instance, you must depend upon a concrete class," and since "creation of instances can happen all through the architecture," it might seem "the entire architecture will be littered with dependencies upon concrete classes. However, there is an elegant solution ... named ABSTRACTFACTORY" — the pattern that confines creation's unavoidable concrete dependencies to a single place (covered with the patterns below, p.32).

ISP — The Interface Segregation Principle

"Many client-specific interfaces are better than one general-purpose interface."
(DesignPrinciplesAndPatterns p.14)

What it is. The ISP says clients should not be forced to depend on methods they do not use. A single "fat" service interface that bundles the methods needed by *all* its clients couples those clients together: ClientA, ClientB, and ClientC all depend on the whole interface, so a change made to the interface *for* ClientA forces ClientB and ClientC to recompile (and re-test) even though their behaviour didn't change (p.15). The fix: give each *kind* of client its own narrow, client-specific interface (`ServiceA`, `ServiceB`, `ServiceC`), each containing only the methods that client uses; the concrete `Service` class then **multiply inherits / implements** all of them (p.15–16). "Clients should be categorized by their type, and interfaces for each type of client should be created" (p.15).

What it's used for / why it matters. A fat interface is a hidden coupling channel: it makes unrelated clients dependent on each other through their shared dependency on one bloated type, so a change demanded by one leaks recompilation and risk into the others (a fragility source). Segregating the interface along client lines installs a *dependency firewall* between the clients — each sees only what it needs, so changes for one client's interface cannot reach the others. ISP is thus the interface-level analogue of SRP/cohesion: an interface should have one client-facing reason to change.

When & how it's applied. Apply when one interface is consumed by several clients with divergent needs (a classic symptom: a client that implements an interface but leaves half the methods empty or throwing `NotSupported`). Split the interface by client role and let the implementer aggregate the roles. *Bad*: one `Worker` interface with `work()`, `eat()`, `recharge()` forced on both `Human` and `Robot` clients. *Good*: separate `Workable`, `Feedable` (and so on) interfaces, each depended on only by the clients that need it. **Caution**: don't over-segregate into "hundreds of different interfaces" — segregate by genuine client type, not per method (p.16).

ISP — client types, changing interfaces, and the dynamic_cast idiom (DesignPrinciplesAndPatterns p.15–16)

The two figures (p.15–16). Figure 2-19, "Fat Service with Integrated Interfaces": `ClientA`, `ClientB`, `ClientC` all depend on a single `Service` class whose interface lumps together «client A methods», «client B methods», «client C methods». Figure 2-20, "Segregated Interfaces": the same three clients now each depend on their own «interface» — `ServiceA`, `ServiceB`, `ServiceC` — and the one concrete `Service` class multiply inherits and implements all three. The payoff, verbatim: "If the interface for `ClientA` needs to change, `ClientB` and `ClientC` will remain unaffected. They will not have to be recompiled or redeployed" (p.15).

"What does Client Specific Mean?" (p.15) — the boundary the deck draws explicitly, and a favourite exam nuance. ISP does **not** mean one interface per client object: "If that were the case, the service would depend upon each and every client in a bizarre and unhealthy way. Rather, clients should be **categorized by their type**, and interfaces for each *type* of client should be created." And the sharing rule: "If two or more different client types need the same method, the method should be **added to both of their interfaces**. This is neither harmful nor confusing to the client." (So some duplication *across interfaces* is explicitly fine — segregation is by client role, not by method uniqueness.)

Changing Interfaces (p.15–16) — ISP applied over time, an explicitly *maintenance* technique: when interfaces of existing classes must change, the impact "can be mitigated by **adding new interfaces** to existing objects, rather than changing the existing interface." Old clients keep using the old interface untouched; clients that want the new methods *query* the object for the new interface:

```
void Client(Service* s)
{
    if (NewService* ns = dynamic_cast<NewService*>(s))
    {
        // use the new service interface
    }
}
```

(This is exactly the COM `QueryInterface` style of versioning — interfaces are added, never mutated — consistent with the DIP section's COM discussion on p.14.)

The overdose warning (p.16) — verbatim: "As with all principles, care must be taken not to overdo it. The specter of a class with **hundreds of different interfaces, some segregated by client and other segregated by version**, would be frightening indeed."

Bridge into packages (p.16). The class-principles part of the chapter closes by scaling up: "Classes are a necessary, but insufficient, means of organizing a design. The larger granularity of **packages** are needed to help bring order. But how do we choose which classes belong in which packages?" — the question the three Package Cohesion Principles answer next.

Package cohesion principles — REP, CCP, CRP

What it is. Once a system is too big to reason about one class at a time, classes are grouped into **packages** (deployable/releasable units), and these three principles answer the question *which classes belong in the same package?* — i.e. they govern package **cohesion** (DesignPrinciplesAndPatterns p.16–18):

- **REP — Release Reuse Equivalency Principle:** *"The granule of reuse is the granule of release."* You can only reuse code that is packaged, released, and versioned as a unit (so clients can track versions); therefore group classes that will be **reused together** into a releasable package (p.17).
- **CCP — Common Closure Principle:** *"Classes that change together, belong together."* Group classes that change for the same reasons and at the same times, so that any one change is confined to as few packages as possible (p.17). (CCP is **SRP for packages** — a package should have one reason to change.)
- **CRP — Common Reuse Principle:** *"Classes that aren't reused together should not be grouped together."* Because a dependency on a package is in effect a dependency on *everything* in it, don't bundle in classes a client doesn't use — otherwise a change to those unused classes still forces the client to revalidate and re-release (p.17–18).

What it's used for / why it matters. These principles let you partition a large system into packages that are pleasant either to *reuse* or to *maintain*. REP and CRP protect the **reuser**: they keep packages small and reusable so a consumer pulls in exactly what it needs and is disturbed only by changes to code it actually uses. CCP protects the **maintainer**: by herding same-reason-to-change classes together it minimises the number of packages any single change ripples across (directly shrinking the deploy/test footprint of a change). Getting package boundaries right is what keeps Impact Analysis and re-release cost manageable at the architectural scale.

When & how it's applied. Use CCP as the primary driver while a system is under heavy development (group by what changes together), and lean toward REP/CRP as components stabilise and become reuse targets (split into smaller, independently versioned packages). **Tension (key exam point):** the three "are mutually exclusive... cannot simultaneously be satisfied" — REP/CRP pull toward *small* packages (good for reusers) while CCP pulls toward *large* ones (good for maintainers). So the package structure should legitimately **jitter** over a project's life: CCP-dominant early, REP/CRP-dominant as it stabilises (p.18). A wrong answer claims you can maximise all three at once.

Package coupling principles — ADP, SDP, SAP

What it is. Where the cohesion principles decide *what goes in* a package, the coupling principles govern the **relationships between** packages — the shape and direction of the inter-package dependency graph (DesignPrinciplesAndPatterns p.18–27):

- **ADP — Acyclic Dependencies Principle:** *"The dependencies between packages must not form cycles."* The dependency graph must be a DAG. A single accidental dependency (e.g. `CommError` → `GUI`) can introduce a cycle that pulls a package's entire transitive closure into its build and test set, "increased by an abhorrent amount," and makes the packages impossible to release independently (p.18–20). **Break cycles** two ways: (1) create a new package that both cycle members depend on (extract `MessageManager`), or (2) apply DIP+ISP — invert one dependency by adding an interface placed *in the package that uses it* (p.20–22).

- **SDP — Stable Dependencies Principle:** "*Depend in the direction of stability.*" Stability = the effort required to change a package, and it is dominated by **incoming dependencies**: a package that many others depend on is **stable** (hard to change because so much would break — "responsible", "independent"); a package that nothing depends on is **instable** (easy to change — "irresponsible", "dependent") (p.22–24). **Stability metric:** $I = C_e / (C_a + C_e)$, where C_a = afferent (incoming) coupling and C_e = efferent (outgoing) coupling; $I = 0$ is maximally stable, $I = 1$ maximally instable. SDP restated quantitatively: "depend upon packages whose I metric is lower than yours" — dependencies should always flow toward *more* stable packages (p.23–24). Crucially, instability is **desirable** for the packages you *want* to keep changeable (p.24).
- **SAP — Stable Abstractions Principle:** "*Stable packages should be abstract packages.*" This resolves the dilemma SDP creates — if stable packages are hard to change, how do they accommodate new requirements? By being **abstract**: an abstract stable package is hard to *modify* but easy to *extend* via new implementations (OCP), so it can flex without changing (p.24–25). SAP is "just a restatement of the DIP" at package scale. **Abstractness metric:** $A = N_a / N_c$ (number of abstract classes ÷ total classes), range [0,1] (p.25–26).

What it's used for / why it matters. These principles, plus their metrics, turn "good architecture" into something you can *measure and enforce*. ADP keeps the build/test/release process tractable — cycles are what make it impossible to test or ship a package in isolation. SDP ensures volatile code never sits *below* and threatens stable code: dependencies always point at things less likely to change, so change naturally flows "downhill" without back-pressure. SAP keeps the stable, heavily-depended-upon packages *extensible* despite being frozen, by making them abstract. Together they describe the ideal: a system whose stable core is abstract (interfaces, policy) and whose changeable edges are concrete (details) — exactly the DIP shape at the largest scale.

When & how it's applied. Compute I and A for each package and check the dependency direction. **The Main Sequence** (p.26–27): plot every package on the **A-vs-I graph**; the ideal line $A + I = 1$ is the **Main Sequence**, where a package is either abstract-and-stable (upper-left: stable core, fine because it's extensible) or concrete-and-instable (lower-right: changeable detail, fine because nothing depends on it). Two forbidden corners to design *away* from: - **Zone of Pain** (lower-left, $A \approx 0, I \approx 0$): concrete *and* heavily depended-upon — can't be extended yet painful to change because everything relies on it (e.g. a concrete utility everyone imports) (p.27). - **Zone of Uselessness** (upper-right, $A \approx 1, I \approx 1$): abstract but nothing depends on it — abstraction nobody uses, i.e. dead weight (p.27).

The **distance metric** $D' = |A + I - 1|$ quantifies how far a package strays from the Main Sequence (0 = on it, 1 = as far as possible), giving you a single number to flag the worst offenders for refactoring (p.27). These formulas — $I = C_e / (C_a + C_e)$, $A = N_a / N_c$, $A + I = 1$ — are the most quantitative, most exam-tested items in the lecture.

Package cohesion — the chapter's full rationales for REP, CCP, CRP (DesignPrinciplesAndPatterns p.17–18)

REP, the release-management argument (p.17). Why is release the granule of reuse? Because reusers are *version consumers*: "Users will be unwilling to use the element if they are forced to upgrade every time the author changes it," so the author "must be willing to support and maintain older versions while his customers go about the slow business of getting ready to upgrade," and "clients will refuse to reuse an element unless the author promises to keep track of version numbers, and maintain old versions for awhile." Reuse therefore *requires* a release system with version tracking, and "since packages are the unit of release, they are also the unit of reuse" — group reusable classes together into releasable, versioned packages.

CCP, the release-workload argument (p.17). A large project is "a large network of inter[r]elated packages," and "the more packages that change in any given release, the greater the work to rebuild, test, and deploy the release." Hence: minimize the number of packages changed per release cycle by grouping classes expected to change together — which "requires a certain amount of presci[e]nce since we must anticipate the kinds of changes that are likely." CCP is forward-looking and probabilistic; you group by *predicted* co-change.

CRP, the OS-vendor analogy (p.17–18). "A dependency upon a package is a dependency upon everything within the package. When a package changes, and its release number is bumped, all clients of that package must verify that they work with the new package — **even if nothing they used within the package actually changed.**" The chapter's everyday analogy: an OS vendor ships a new version; "we have to upgrade sooner or later, because the vendor will not support the old version forever. So even though nothing of interest to us changed in the new release, we must go through the effort of upgrading and revalidating." Bundle un-co-used classes into one package and you inflict exactly this on your clients.

The tension, in the chapter's own words (p.18). "These three principles are mutually exclusive. They cannot simultaneously be satisfied. That is because each principle benefits a different group of people. The REP and CRP makes life easy for reusers, whereas the CCP makes life easier for maintainers. The CCP strives to make packages **as large as possible** (after all, if all the classes live in just one package, then only one package will ever change). The CRP, however, tries to make packages **very small.**" And the resolution: "packages are not fixed in stone ... it is the nature of packages to shift and jitter during the course of development" — early on, set the structure so **CCP dominates** (helping development and maintenance); later, "as the architecture stabilizes," refactor it toward **REP/CRP** for external reusers.

Package coupling — the CommError cycle story and both cycle-breaking techniques (DesignPrinciplesAndPatterns p.18–22)

Why packages focus engineers (p.18–19). Packages "tend to focus manpower": engineers typically work *inside one package* rather than across dozens — a tendency amplified by the cohesion principles, which group related classes. Changes are thereby "directed into just a few package[s]," and once made, the package can be released to the rest of the project — *after* it has been tested, which requires building it "with all the packages that it depends upon. Hopefully this number is small."

The example system (Figure 2-21, p.19). Seven packages: `GUI`, `Comm`, `ModemControl`, `Protocol`, `CommError`, `Analysis`, `Database`. The chapter openly flags the architecture as flawed even *before* the cycle: "The DIP seems to have been abandoned, and along with it the OCP. The GUI depends directly upon the communications package, and apparently is responsible for transporting data to the analysis package. **Yuk.**" Still, the structure is acyclic, and releasing `Protocol` is cheap: "build it with the latest release of the `CommError` package, and run their tests. `Protocol` has no other dependencies ... We can test and release with a minimal amount of work."

A cycle creeps in (Figure 2-22, p.20). An engineer working in `CommError` wants to display a message; the screen belongs to the `GUI`, so they call a GUI object — innocently creating `CommError` → `GUI`. Consequence for the `Protocol` team: "They have to build their test suite with **CommError, GUI, Comm, ModemControl, Analysis, and Database!** This is clearly disastrous. The workload of the engineers has been increased by an **abhor[r]ent amount, due to one single little dependency that got out of control.**" Hence the governance rule: "someone needs to be watching the package dependency structure with regularity, and breaking cycles wherever they appear. Otherwise the transitive dependencies between modules will cause **every module to depend upon every other module.**"

Cycle-break technique 1 — extract a new package (Figure 2-23, p.21). The classes `CommError` needed are pulled *out of* `GUI` into a new package `MessageManager`; both `GUI` and `CommError` then depend on

`MessageManager`, and the cycle is gone. The chapter's meta-observation: "This is an example of how the package structure tends to **jitter and shift** during development. New package[s] come into existence, and classes move from old package[s] to new packages, to help break cycles" — the same jitter the cohesion-tension section predicted.

Cycle-break technique 2 — DIP + ISP (Figure 2-24, p.21–22). Before: package 1 holds classes `A`, `B`; package 2 holds `X`, `Y`; `A` depends on `X` and `Y` depends on `B` — a two-package cycle. After: add an interface `BY` containing "all the methods that `Y` needs"; `Y` uses `BY` and `B` implements it — the `Y` → `B` dependency is inverted into `B` → `BY`, and the cycle is broken. The placement rule, verbatim and counter-intuitive: "Notice the placement of `BY`. It is placed **in the package with the class that uses it ... Interfaces are very often included in the package that uses them, rather than in the package that implements them**" (p.21–22). (Naïve intuition puts an interface next to its implementor; the chapter puts it next to its *client*, so the client's package owns its own dependency target — pure DIP.)

SDP/SAP — the penny analogy, responsibility vocabulary, and the Flexible/Stable violation (DesignPrinciplesAndPatterns p.22–25)

The penny analogy (p.22–23). "Stand a penny on its side. Is it stable in that position? Likely you'd say not. However, unless disturbed, it will remain in that position for a very very long time. Thus **stability has nothing direct[t]ly to do with frequency of change**. The penny is not changing, but it is hard to think of it as stable. ... Stability is related to the **amount of work required to make a change**. The penny is not stable because it requires very little work to topple it. On the other hand, a table is very stable because it takes a considerable amount of effort to turn it over." This is the chapter's defence against the most common misreading of "stable": stable ≠ unchanging; stable = *expensive to change*.

From furniture to software (p.23). Many factors make a package hard to change — "its size, complexity, clarity, etc." — but the chapter deliberately ignores them all and isolates one: "One sure way to make a software package difficult to change, is to make **lots of other software packages depend upon it**."

The responsibility vocabulary (p.23). Figure 2-25 shows package `X` with three incoming dependencies and none outgoing: `X` "has three good reasons not to change" — it is **responsible** (others rely on it) and **independent** (nothing external can force it to change). Figure 2-26 shows package `Y` with no incoming and three outgoing dependencies: it is **irresponsible** (nobody relies on it) and **dependent** (change can be forced on it from three external sources). Memorize the pairing: *stable = responsible + independent; instable = irresponsible + dependent*.

The SDP violation story (Figure 2-27, p.24). `Flexible` is a package *designed* to be easy to change (instable, high `I`). "However, some engineer, working in the package named `Stable`, hung a dependency upon `Flexible`." Now `Stable`'s low-`I` weight sits on top of `Flexible`: "Flexible will no longer be easy to change. A change to Flexible will force us to deal with Stable and all its dependents." One stray dependency *transfers* stability onto a package that wanted none — exactly the SDP's "depend only on packages whose `I` is lower than yours" violated in the upward direction.

The dilemma and the loophole (p.25). Picture the system with "instable packages at the top, and stable packages on the bottom," all dependencies pointing down. The dilemma: "Do we want packages in our design that are hard to change? Clearly, the more packages that are hard to change, the less flexible our overall design will be." The loophole is the OCP: "The highly stable packages at the bottom ... may be very difficult to change, but ... they do not have to be difficult to **extend**!" If the stable bottom packages are also highly *abstract*, "they can be easily extended," and the application composes "instable packages that are easy to change, and stable packages that are easy to extend. This is a good thing." Hence the SAP — explicitly "just a restatement of the DIP" at package granularity (p.25).

The complete metric set — Ca, Ce, I, Nc, Na, A, D, D' (DesignPrinciplesAndPatterns p.23–27)

Every metric the chapter defines, with its exact definition and range — the most calculation-friendly exam material in the lecture:

Metric	Name	Definition (chapter wording)	Range
Ca	Afferent Coupling	"The number of classes outside the package that depend upon classes inside the package (i.e. incoming dependencies)" (p.23)	0 ... n
Ce	Efferent Coupling	"The number of classes outside the package that classes inside the package depend upon (i.e. outgoing dependencies)" (p.24)	0 ... n
I	Instability	$I = Ce / (Ca + Ce)$ (p.24)	[0,1]; I=0 maximally stable, I=1 maximally instable
Nc	Class count	"Number of classes in the package" (p.26)	1 ... n
Na	Abstract class count	"Number of abstract classes in the package. Remember, an abstract class is a class with at least one pure interface , and cannot be instantiated " (p.26)	0 ... Nc
A	Abstractness	$A = Na / Nc$ (p.26)	[0,1]; A=0 no abstract classes, A=1 nothing but
D	Distance from Main Sequence	$D = (A + I - 1) / \sqrt{2}$ (p.27)	[0, ~0.707]
D'	Normalized Distance	$D' = \sqrt{ A + I - 1 }$ (p.27)	[0,1]; 0 = on the Main Sequence, 1 = as far away as possible

Supporting statements worth quoting exactly: "If there are no outgoing dependencies, then I will be zero and the package is stable. If there are no incoming dependencies then I will be one and the package is instable" (p.24). The SAP restated metrically: "**I should increase as A decreases**. That is, concrete packages should be instable while abstract packages should be stable" (p.26). On the A-vs-I graph, packages ideally sit at one of "the two black dots" — completely abstract & stable (upper left, A=1, I=0) or completely concrete & instable (lower right, A=0, I=1) (p.26) — or anywhere on the **Main Sequence** connecting them, where "the package is abstract in proportion to its incoming dependencies and is concrete in proportion to its outgoing dependencies. In other words, the classes in such a package are **conforming to the DIP**" (p.27). And the chapter's own caveat about its metrics: "They are imperfect, and **reliance upon them as the sole indicator of a sturdy architecture would be foolhardy**. However, they can be, and have been, used to help measure the dependency structure of an application" (p.27).

Worked calculation (application of the formulas above — the numbers are illustrative, not from the chapter). Suppose package P contains 4 classes of which 1 is abstract, 8 outside classes depend on classes in P, and classes in P depend on 2 outside classes. Then $Ca = 8$, $Ce = 2$, so $I = 2/(8+2) = 0.2$ (quite stable); $A = 1/4 = 0.25$; $D' = \sqrt{|0.25 + 0.2 - 1|} = 0.55$ — P is far off the Main Sequence on the *Zone-of-Pain side* (stable but concrete, with eight dependants and almost no abstraction). The SAP prescription: raise A (extract interfaces for what the eight dependants use) or shed afferent couplings.

Design Patterns — definition

What it is. A **design pattern** is a named, reusable solution to a commonly-recurring design problem: "repeating structures of design and architecture are known as design patterns... a well-worn and known good solution to a common problem" (DesignPrinciplesAndPatterns p.28). Patterns are *discovered, not invented* — "old techniques that have shown their usefulness over a period of many years" — and each captures the participants, their relationships, and the trade-offs of a proven design so it can be applied again.

What it's used for / why it matters. Patterns give two things: a **vocabulary** (saying "use an Observer here" conveys a whole design in two words to anyone who knows the catalogue, which is itself a clean-code, solution-domain-name benefit) and a **toolbox** of designs that already satisfy the OO principles — most GoF patterns are concrete ways to honour OCP/DIP/LSP. For maintenance this matters because pattern-rich code is more recognisable and more changeable: a reader who spots the pattern instantly understands the intended extension points, which speeds Concept Location and keeps the Impact Set small.

When & how it's applied. Apply a pattern when your problem matches its *intent* — never force a pattern for its own sake. The deck stresses one chapter can't cover patterns adequately and points to the **Gang of Four** book [GHJV94] for the full catalogue (p.28). The GoF taxonomy has three families — **creational** (object construction), **structural** (object composition), and **behavioral** (object interaction/responsibility) — and the five patterns below (the ones "you will come across while reading through the case studies," p.28) are each tagged with their GoF family and the principle they serve.

Pattern — Abstract Server (structural, "hinge point")

Intent / problem solved. When a client (`Consumer`) depends *directly* on a concrete server (`ResourceManager`), the **DIP is violated**: any change to the server propagates straight into the client, and the client is welded to that one server — it can't be given a different but similar implementation (DesignPrinciplesAndPatterns p.28). Abstract Server cures this by breaking the direct dependency.

Structure / participants. Insert an **abstract interface** (`ResourceManager_I`) between the `Consumer` and the concrete `ResourceManager`: the consumer now depends only on the interface, and the concrete server implements it. That interface "becomes a 'hinge point' upon which the design can flex," so "different implementations of the server can be bound to an unsuspecting client" (p.28). Participants: the *client*, the *abstract server interface*, and one or more *concrete server* implementations.

What it's used for / usage example. This is the simplest, most fundamental application of OCP/DIP — "program to an interface" made into a named move; GoF would file the idea under interface-programming rather than a single distinct pattern. Use it whenever you control the server and foresee swapping implementations: e.g. a `Consumer` that needs a `ResourceManager` should depend on `ResourceManager_I` so you can later substitute an in-memory, networked, or mock implementation without touching the consumer. In **JHotDraw** this is pervasive — clients work against the `Figure`, `Tool`, `Handle`, and `Connector` interfaces rather than concrete classes, which is exactly why you can add a new figure type without editing the framework (the OCP payoff, DesignPrinciplesAndPatterns p.4).

Pattern — Adapter (GoF structural)

Intent / problem solved. You want the Abstract-Server arrangement, but inserting the interface directly into the server is **infeasible** — the server is third-party, or it's so heavily depended-upon that changing it is too risky (DesignPrinciplesAndPatterns p.29). Adapter lets you bind a client to your desired abstract interface even though the real server can't implement that interface itself. (Equivalently: it makes two incompatible interfaces work together.)

Structure / participants. An `Adapter` object *implements* the abstract interface the client wants (`ResourceManager_I`) and holds a reference to the unchangeable concrete server, **delegating** each call to it: "every method of the adapter simply translates and then delegates" (p.29). Participants: the *Target* interface the client expects, the *Adaptee* (the existing/foreign server you can't change), and the *Adapter* that implements *Target* by translating to *Adaptee* calls.

What it's used for / usage example. Use Adapter to wrap a legacy or external API behind the interface your design wants, so the rest of the system depends only on your clean interface. Classic example: wrapping a third-party `LegacyTemperatureSensor` (with `getTempF()`) in an adapter that implements your `TemperatureProbe` interface (`readCelsius()`) by converting units and delegating. In **JHotDraw**, adapters bind external AWT/Swing components and other foreign types to JHotDraw's own interfaces, so the framework's code stays expressed in its own abstractions rather than the toolkit's (DesignPrinciplesAndPatterns p.29). The distinction to remember: **Abstract Server when you can change the server, Adapter when you can't.**

Pattern — Observer (GoF behavioral)

Intent / problem solved. "One element of a design needs to take some action when another element discovers that an event has occurred," but "we don't want the detector to know about the actor" (DesignPrinciplesAndPatterns p.29). The deck's example: a `Meter` must update whenever a `Sensor` reading changes, "but we don't want the sensor to know anything about the meter" (p.29). The problem Observer solves is establishing a *one-to-many notification* relationship while keeping the source decoupled from its consumers — the source must not depend on the (possibly many, possibly changing) things that react to it.

Structure / participants. `Sensor` derives from `Subject`, which holds a list of `Observer`s populated via `Register(Observer*)`; `Meter` implements the `Observer` interface (`Update()`) (p.30). Participants: the *Subject* (knows its observers only through the abstract `Observer` interface and offers register/unregister/notify), the abstract *Observer* (defines `Update()`), the *ConcreteSubject* (`Sensor`, holds the state of interest), and the *ConcreteObserver* (`Meter`, reacts to changes). **Dynamics:** the `Sensor` detects a change → calls `Notify` on its `Subject` → the `Subject` cycles every registered `Observer` calling `Update` → each `Meter` reads the new value and displays it (p.29–30).

What it's used for / usage example. Use Observer whenever multiple parts of a system must stay in sync with a changing piece of state but the state-holder shouldn't be coupled to them — GUI views tracking a model, listeners reacting to events, spreadsheets recomputing cells. Because the `Subject` depends only on the abstract `Observer`, you can add new kinds of observer (a logger, a second display) without touching the `Subject` — a direct OCP win. This is **JHotDraw's** central pattern: a `Figure` is a `Subject` and `DrawingView` / `FigureListener`s are `Observers`, so when a figure moves or changes it simply *notifies* its listeners, with no knowledge of which views are displaying it — exactly the `Sensor`→`Meter` scenario (DesignPrinciplesAndPatterns p.29–30). Its `Subject/Observer/Notify/Update` dynamics are especially exam-friendly.

Pattern — Bridge (GoF structural)

Intent / problem solved. Decouple an *abstraction* from its *implementation* so the two can vary independently. The problem it attacks: when you implement an abstraction using ordinary inheritance, the derived class becomes so tightly bound to its base that you can't reuse the implementation on its own or give the abstraction a *different* implementation (DesignPrinciplesAndPatterns p.30–31). The deck's example: a `MusicSynthesizer` whose `PlayMidi` translates MIDI into `EmitVoice` calls — but `EmitVoice` is "inextricably bound" to `PlayMidi`, so you can neither reuse `EmitVoice` elsewhere nor swap in a different voice

implementation (p.31). Bridge also prevents a combinatorial class explosion when an abstraction has several dimensions of variation.

Structure / participants. Create a **strong separation between interface and implementation** by giving the abstraction a *reference* to an implementation interface instead of inheriting it: `MusicSynthesizer` (with abstract `PlayMidi`) bridges to a separate `VoiceEmitter` interface. Now `EmitVoice` and `PlayMidi` are "decoupled" and each can be implemented "any number of different ways" independently (p.31). Participants: the *Abstraction* (holds a reference to an *Implementor*), *RefinedAbstractions*, the *Implementor* interface, and *ConcreteImplementors* — the two hierarchies vary on their own axes.

What it's used for / usage example. Use Bridge when a thing varies along *two independent dimensions* and you don't want one hierarchy multiplied by the other — e.g. `Shape` × `RenderingAPI`, where `Circle` / `Square` (abstraction) each delegate drawing to an injected `Renderer` (`OpenGLRenderer` / `SVGRenderer`), so adding a shape or a renderer is linear, not multiplicative. In **JHotDraw**, Bridge-style separation (together with the related **Strategy** and **Composite** patterns) lets a figure's *behaviour* vary independently of its *structure* — handles, connectors, and locators are factored out so figure rendering and figure interaction don't have to be subclassed together (DesignPrinciplesAndPatterns p.30–31).

Pattern — Abstract Factory (GoF creational)

Intent / problem solved. The DIP says depend on abstractions, not concretions — but the act of **creating an object necessarily names a concrete class**, so creation is the one place you're forced to violate DIP. Abstract Factory "allows that dependency upon the concrete class to exist in one, and only one, place" (DesignPrinciplesAndPatterns p.32), quarantining all concrete-construction knowledge so the rest of the system stays purely abstraction-dependent. (More generally it creates *families* of related objects through one abstract interface.)

Structure / participants. In the deck's modem example: clients create modems through a `ModemFactory` **interface** (reached via a global `GtheFactory`), calling `Make(string)` which returns a `Modem*` *interface* — the client never names a concrete modem. The concrete `ModemFactory_I`, which is the *only* module that knows the concrete `Hayes` / `Courrier` / `Ernie` classes, is instantiated **by main** and loaded into the global. Participants: the *AbstractFactory* (`ModemFactory`), the *ConcreteFactory* (`ModemFactory_I`), the *AbstractProduct* (`Modem`), and the *ConcreteProducts* (`Hayes`, etc.).

What it's used for / usage example. Result: "no module in the system knows about the concrete modem classes except for `ModemFactory_I`, and no module knows about `ModemFactory_I` except for `main`" (p.32) — all concrete-class knowledge is quarantined at the top of the system, so adding a new modem changes only the factory. Use Abstract Factory whenever clients must create objects but should remain ignorant of concrete types: pluggable look-and-feels, swappable persistence backends, test doubles injected in place of real services. In **JHotDraw**, tools and figures are created through factory abstractions, so the framework never hard-codes concrete figure classes — the same quarantine-of-concrete-knowledge that the modem factory provides (DesignPrinciplesAndPatterns p.32).

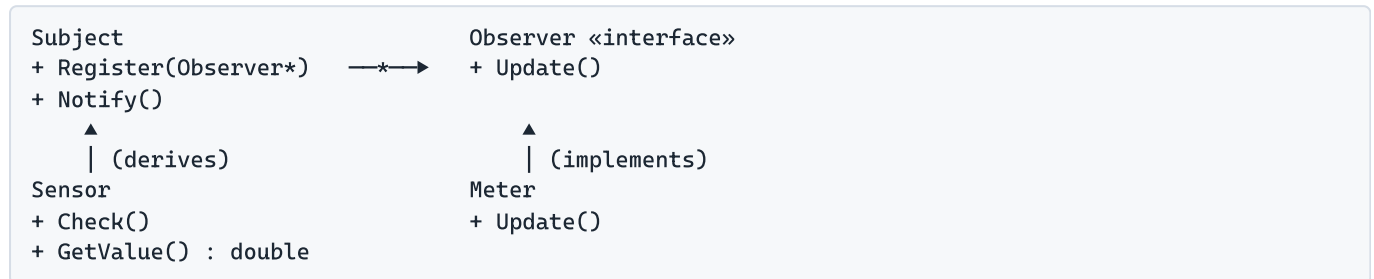
The pattern figures, box by box (DesignPrinciplesAndPatterns p.28–33)

The chapter draws one figure per pattern; being able to reproduce the participants and arrows is the most reliable way to answer structure questions.

Figure 2-30, Abstract Server (p.28): three boxes in a row — `Consumer` —uses—► `ResourceManager_I` «interface» ◀—implements— `ResourceManager`. One interface inserted between client and server; both arrows point *at the interface*.

Figure 2-31, Adapter (p.29): four boxes — `Consumer` —► `ResourceManager_I` «interface»
 ◀—implements— `ResourceManagerAdapter` —delegates—► `ResourceManager`. Compared with Abstract Server, exactly one box is new: the adapter, which implements the interface the consumer wants and forwards every call to the unchangeable server ("every method of the ad[a]pter simply translates and then delegates," p.29).

Figure 2-32, Observer structure (p.30):



`Subject` holds a `*`-multiplicity list of `Observer`s, loaded via `Register`. **Figure 2-33, Observer dynamics (p.29–30):** some entity calls `Check` on the `Sensor` → the `Sensor` determines its reading changed and calls `Notify` on its `Subject` base → `Subject` cycles through all registered `Observer`s calling `Update` on each → each `Meter` catches `Update`, calls `GetValue` back on the `Sensor`, and displays the `Value`. Note the *pull* style: the notification (`Update`) carries no data; the observer pulls the new value itself via `GetValue`.

Figure 2-34, the badly coupled synthesizer hierarchy (p.30–31): `MusicSynthesizer` {abstract} declares `+ PlayMidi` and abstract `- EmitVoice`; `MusicSynthesizer_I` derives from it and implements `+ EmitVoice`. The base translates MIDI into primitive `EmitVoice` calls implemented by the derived class — so `EmitVoice`, though "useful, in and of itself," is "inextricably bound to the `MusicSynthesizer` class and the `PlayMidi` function. ... Also, there is no way to create different implementations of the `PlayMidi` function that use the same `EmitVoice` function. In short, the hi[er]archy is just to[o] coupled" (p.31).

Figure 2-35, the hierarchy decoupled with Bridge (p.31–32): now *two* hierarchies joined by a reference: `MusicSynthesizer` {abstract} (`+ PlayMidi`, `- EmitVoice`) with derived `MusicSynthesizer_I` (`- PlayMidi`), bridging to `VoiceEmitter` «interface» (`+ EmitVoice`) implemented by `VoiceEmitter_I`. The text's wiring: the abstract `PlayMidi` is implemented by `MusicSynthesizer_I`; it calls the `EmitVoice` function implemented in `MusicSynthesizer`, which *delegates to the* `VoiceEmitter` interface; `VoiceEmitter_I` emits the actual sounds. Result, verbatim: "Now it is possible to implement both `EmitVoice` and `PlayMidi` separately from each other. The two functions have been decoupled. `EmitVoice` can be called without bringing along all the `MusicSynthesizer` baggage, and `PlayMidi` can be implemented any number of different ways, while still using the same `EmitVoice` function" (p.31).

Figure 2-36, Abstract Factory (p.32–33): participants — `Users` —► `ModemFactory` «interface» (`+ Make(string) : Modem*`); `GtheFactory` «global» (holds the pointer to the factory the users call through); `ModemFactory_I` (implements `ModemFactory`); `main` «main program», which «creates» `ModemFactory_I` and loads it into `GtheFactory`; `ModemFactory_I` «creates» `Hayes`, `Courrier`, `Ernie`; all three implement the `Modem` «interface» that `Make` returns. The callers select a product by passing "a string that uniquely defines the particular subclass of `Modem` that they want" (p.32) — type selection by *data*, not by code, which is why no caller needs a concrete type name.

The chapter's conclusion (p.32–33). Object-oriented architecture is defined as "the structure of classes and packages that keeps the software application **flexible, robust, reusable, and developable**" — a four-adjective definition worth memorizing. And the chapter signs off with humility: "This has been an overview ... It has been said that **a little knowledge is a dangerous thing**, and this chapter has provided a little

knowledge. We strongly urge you to search out the books and papers in the citations of this chapter to learn more" (p.33).

The chapter's own bibliography keys (p.1, p.33–34), exactly as printed: [Shaw96] Patterns of Software Architecture, Garlan and Shaw; [GOF96] *Design Patterns* (the Gang of Four book — note the chapter's own key is [GOF96]); [OOSC98] Meyer's *Object-Oriented Software Construction*; [OCP97] , [LSP97] , [DIP97] , [ISP97] , [Granularity97] , [Stability97] — Martin's six principle papers; [Liksov88] (sic) *Data Abstraction and Hierarchy*; [Martin99] *Designing Object Oriented Applications using UML*, 2d ed., Prentice Hall, 1999.

JHotDraw Connection

Grounding note: Neither LO6 deck names JHotDraw. The connection below is **inference**, tying this lecture's content to the course-wide case study. JHotDraw is, however, *the* canonical teaching example for exactly the material in this lecture, which is presumably why this cross-cutting lecture sits in the course.

JHotDraw is famous in the literature as a **textbook showcase of GoF design patterns** — Erich Gamma (a GoF author) co-wrote it precisely to demonstrate patterns in a real framework. Several patterns from this lecture appear directly in it:

- **Observer** — JHotDraw's `Figure / Drawing` model notifies views (`DrawingView`) and `FigureListener`s whenever a figure changes. This is the deck's Sensor→Meter scenario (*DesignPrinciplesAndPatterns* p.29–30): the figure (Subject) doesn't know about the views (Observers), it just `Notify`s.
- **Abstract Factory / factory methods** — tools and figures are created through factory abstractions so the framework never hard-codes concrete figure classes, exactly the modern-factory quarantine of concrete-class knowledge (*DesignPrinciplesAndPatterns* p.32).
- **Adapter** — JHotDraw adapts AWT/Swing components and external types to its own interfaces (*DesignPrinciplesAndPatterns* p.29).
- **Bridge / Strategy / Composite / Decorator** — JHotDraw uses Bridge-style separation (and the related Strategy and Composite patterns) to let figure behavior vary independently of figure structure (*DesignPrinciplesAndPatterns* p.30–31 for Bridge).

OO principles in JHotDraw: the framework is built to be **open for extension, closed for modification** (OCP, *DesignPrinciplesAndPatterns* p.4) — you add a new `Figure` subclass without editing the framework — which is exactly the OCP payoff Martin describes. Its small, single-responsibility classes (*CleanCode* p.79) and interface-based design (DIP, *DesignPrinciplesAndPatterns* p.12) are why the course uses it: a clean, pattern-rich codebase is one where **Concept Location** and **Impact Analysis** are tractable, because dependencies point at abstractions and changes don't cascade.

Connecting clean code to the change process on JHotDraw: when you perform a change on JHotDraw in the labs (Concept Location → Impact Analysis → Actualization → Postfactoring), the *reason* the impact set stays small is that the framework obeys these principles. Conversely, a JHotDraw that violated the Law of Demeter, returned nulls, and switched on type codes would exhibit the **rigidity/fragility** symptoms (*DesignPrinciplesAndPatterns* p.2–3) that make every maintenance task expensive.

Worked Example / Process Walkthrough

There is no lab for LO6, so this is a **synthesized walkthrough** showing how the lecture's rules combine to turn an OCP-violating, comment-smelly, hard-to-maintain fragment into clean code — and how that links to the change process. It is built entirely from techniques on the cited slides.

Starting point — code being maintained:

```
// Check to see if the employee is eligible for full benefits
public int payAmount(Employee e) {
    switch (e.getType()) {
        // smell: switch on type code
        case EmployeeType.ENGINEER: return e.m_sal; // smell: encoding (m_ prefix)
        case EmployeeType.SALESMAN: return e.m_sal + e.m_comm;
        case EmployeeType.MANAGER: return e.m_sal + e.m_bonus;
        default: return -1; // smell: error code / magic value
    }
}
```

Suppose Initiation requests a new `EmployeeType.CONTRACTOR`. Walk the process:

1. **Concept Location** finds `payAmount`. So far so good.
2. **Impact Analysis** discovers the `switch` is **duplicated** across the codebase (every operation on an employee switches on the same type code). The estimated impact set is *large* — a textbook **rigidity** symptom (DesignPrinciplesAndPatterns p.2). Each `switch` "must be appropriately modified" (DesignPrinciplesAndPatterns p.6).
3. **Prefactoring** (clean before you change): apply **OCP** by replacing the switch with **polymorphism** (CleanCode p.25–26; DesignPrinciplesAndPatterns p.4–6). Push creation into an **Abstract Factory** (DesignPrinciplesAndPatterns p.32) so the one remaining `switch` lives in one place:

```
abstract class EmployeeType { abstract int payAmount(Employee emp); } // OCP hinge point
class Salesman extends EmployeeType { int payAmount(Employee emp){ return emp.getMonthlySalary() +
emp.getCommission(); } }
class Manager extends EmployeeType { int payAmount(Employee emp){ return emp.getMonthlySalary() +
emp.getBonus(); } }
```

4. **Actualization:** adding `CONTRACTOR` is now **only new code** — a new `Contractor` extends `EmployeeType` — with **no edits to working code** (DesignPrinciplesAndPatterns p.7). The impact set shrinks to one new class.
5. **Postfactoring / clean-up** (Boy Scout Rule, CleanCode p.3): delete the misleading top comment and **express intent in code** (CleanCode p.34: `isEligibleForFullBenefits()`); drop the `m_` encodings (CleanCode p.15); replace the `-1` error code with an **exception** or Special-Case object (CleanCode p.65–66, 69–70); keep each method tiny and single-purpose (CleanCode p.23, p.79 SRP).
6. **Verification:** because each `payAmount` override is a tiny pure function, it's trivially unit-testable per the **Three Laws of TDD** and **F.I.R.S.T.** (CleanCode p.74, p.77) — one assert, one concept, fast and self-validating (CleanCode p.76).

Moral: the same change is cheap *because* the code obeys clean-code rules and OO principles. Clean code is not aesthetics; it is **lower maintenance cost** measured directly in the size of the impact set and the ease of verification.

Second walkthrough — breaking a package cycle during a change (DesignPrinciplesAndPatterns p.19–22)

A second synthesized walkthrough, this time at *package* scale, using the deck's own CommError example as the change request.

Starting point: the Figure 2-21 system — GUI, Comm, ModemControl, Protocol, CommError, Analysis, Database — acyclic, with Protocol depending only on CommError (DesignPrinciplesAndPatterns p.19).

1. **Initiation:** a change request lands on the CommError team: "communication errors must be shown to the user on screen."
2. **Concept Location:** screen output lives in GUI; the error detection lives in CommError.
3. **Impact Analysis — the trap.** The *quick* implementation is one line: call a GUI object from CommError. The impact *looks* tiny — that is the **viscosity** trap (the hack is easier than the design-preserving change, DesignPrinciplesAndPatterns p.3). The true impact is structural: the new edge CommError → GUI closes a **cycle**, and suddenly releasing Protocol requires building and testing against CommError, GUI, Comm, ModemControl, Analysis, Database — the workload "increased by an abhorrent amount, due to one single little dependency" (p.20).
4. **Prefactoring — choose a cycle-free design.** Two deck-sanctioned options: **(a)** extract the needed display classes from GUI into a new package MessageManager that both GUI and CommError depend on (p.21); or **(b)** apply DIP+ISP — define a message-display *interface* placed in CommError's own package (the package that *uses* it, per the placement rule on p.21–22), and let a GUI class implement it.
5. **Actualization:** CommError codes against the new package/interface; the concrete display wiring happens above, where dependencies may legally point downward toward the more stable CommError.
6. **Verification / Conclusion:** confirm with the metrics — recompute each affected package's Ca, Ce, I and check every dependency still points at lower- I packages (SDP, p.24); the dependency graph is a DAG again (ADP, p.18).

Moral: at package scale, Impact Analysis must include *structural* impact (cycles, stability direction), not just lines-to-edit. One innocent call can multiply another team's release cost sixfold.

Third walkthrough — a Boy-Scout comment-and-formatting pass (CleanCode p.34–60)

A micro-scale companion: what the Boy Scout Rule actually looks like applied to a file you are passing through anyway.

Starting point — a method you touched for an unrelated bugfix contains:

```
/* Added by Rick */
// Utility method that returns when this.closed is true.
public synchronized void waitForClose(final long timeoutMillis) throws Exception {
    if(!closed) {
        wait(timeoutMillis);
        if(!closed) throw new Exception("MockResponseSender could not be closed");
    } //if
    // InputStream resultsStream = formatter.getResultStream();
    // StreamReader reader = new StreamReader(resultsStream);
}
```

The pass, rule by rule: delete /* Added by Rick */ (attribution — version control's job, CleanCode p.47); delete the redundant header comment (it restates the signature less precisely than the code, CleanCode p.39); delete the commented-out lines (the VCS remembers them, CleanCode p.47); delete } //if (closing-brace

comment — and note the block is small enough not to need it, CleanCode p.46); restore the collapsed one-line `if` body onto its own indented line (breaking indentation, CleanCode p.59). Five deletions and one re-indent, zero behaviour change, perhaps ninety seconds of work — and the next reader's WTFs/minute (CleanCode p.10) drops measurably. That is the Boy Scout Rule (CleanCode p.3) operating as **Postfactoring** in miniature.

Definitions & Terminology

Term	Definition	Source
Boy Scout Rule	What: always leave code cleaner than you found it. Used for: continuous, no-cost-justification improvement so code trends cleaner over time. Applied: while editing for another reason, make one small improvement (rename a local, delete a stale comment) before committing.	CleanCode p.3
Intention-revealing name	What: a name that says why a thing exists, what it does, and how it's used. Used for: removing decoding cost, making Concept Location free, eliminating explanatory comments. Applied: <code>getFlaggedCells()</code> not <code>getThem()</code> ; rename instead of commenting.	CleanCode p.11
Disinformation	What: a name that actively misleads (visually confusable <code>l / 0</code> vs <code>1 / 0</code> , or <code>accountList</code> that isn't a list). Used for: the rule is to <i>avoid</i> it, because a wrong name causes bugs, not just slowness. Applied: never use <code>l / 0 / I</code> as names; name a <code>Map accounts</code> , not <code>accountList</code> .	CleanCode p.12
Hungarian notation	What: encoding a variable's type into its name (<code>phoneString</code>); an anti-pattern. Used for: the rule is to <i>avoid</i> it — the name becomes a lie when the type changes and the compiler already records the type. Applied: write <code>PhoneNumber phone;</code> , not <code>PhoneNumber phoneString;</code> .	CleanCode p.16
Member prefix	What: prefixing instance fields to mark them as members (<code>m_dsc</code>); avoid. Used for: the rule removes noise the eye must skip; IDEs already scope members. Applied: <code>private String description;</code> , not <code>private String m_dsc;</code> .	CleanCode p.15
Mental mapping	What: forcing readers to translate a cryptic name into the concept it means; avoid. Used for: preserving scarce working memory; use names readers already know. Applied: use the conventional <code>i / j</code> (or descriptive <code>row / col</code>), not invented <code>a / b</code> .	CleanCode p.17
Stepdown Rule	What: code reads top→bottom, each function placed above those one level of abstraction lower. Used for: letting a reader grasp flow at the top and descend into detail only where needed. Applied: order a file so callers sit above callees, descending one abstraction step at a time.	CleanCode p.24
One level of abstraction per function	What: don't mix high/intermediate/low abstraction in one function body. Used for: keeping each function a coherent single-altitude paragraph the reader can skim. Applied: <code>getHtml()</code> shouldn't both call <code>PathParser.render()</code> and hand-append <code>"\n"</code> ; extract the low-level work.	CleanCode p.24
Niladic/Monadic/Dyadic/Triadic	What: functions with 0/1/2/3 arguments; fewer is better (zero is ideal). Used for: fewer args = fewer ways to misuse the call and fewer test combinations. Applied: prefer <code>writeField(name)</code> over <code>writeField(stream, name)</code> ; bundle 3+ args into an object.	CleanCode p.27–30
Flag argument	What: a boolean parameter that selects behaviour, signalling the function does >1 thing; avoid. Used for: the rule splits the function so call sites and bodies are unambiguous. Applied: replace <code>render(true)</code> with <code>renderForSuite()</code> / <code>renderForSingleTest()</code> .	CleanCode p.29

Term	Definition	Source
Command/Query Separation	What: a function either <i>does</i> something (command, returns void) OR <i>answers</i> something (query, no side-effects), not both. Used for: making queries safe to call and commands unambiguous, avoiding side-effect bugs. Applied: split <code>if (set(...))</code> into <code>attributeExists(...)</code> (query) + <code>setAttribute(...)</code> (command).	CleanCode p.32
DRY	What: Don't Repeat Yourself; "duplication may be the root of all evil." Used for: so a rule lives in one place and changes once — duplication inflates the Impact Set and causes fragility. Applied: extract the copy-pasted block into one shared method called from each site.	CleanCode p.33
Mumbling / Redundant / Mandated / Journal / Noise / Scary Noise comments	What: the taxonomy of bad comments (terse-and-useless / restates-code / policy-forced / embedded-changelog / says-nothing / copy-pasted-and-wrong). Used for: recognising comments to <i>delete</i> because they add noise or drift into lies. Applied: delete them (info lives in code or Git) or rewrite the code so no comment is needed.	CleanCode p.38–44
Position marker / Closing-brace comment	What: banner <code>////////</code> section markers; <code>} //while</code> brace labels; both smells. Used for: the rule flags them — a brace comment means the block is too big. Applied: delete the banner; shorten the function instead of labelling its closing brace.	CleanCode p.46
Newspaper metaphor	What: a file should read like an article — headline/high-level at top, detail below. Used for: letting a reader understand purpose from the top without reading everything. Applied: put high-level public methods first, low-level helpers lower down.	CleanCode p.53
Vertical openness / density / distance	What: blank lines separate concepts (openness); related lines stay close (density); declare variables near use, callers above callees (distance). Used for: giving the reader a visual map of which code belongs together. Applied: blank-line between concepts; declare a local on the line before its first use.	CleanCode p.53–55
Team rules	What: team formatting conventions override personal preference. Used for: one consistent, predictable reading experience across the whole codebase. Applied: adopt and auto-enforce a single shared formatter; drop your personal style.	CleanCode p.60
Data abstraction	What: hide implementation behind an abstract interface (Cartesian-vs-polar <code>Point</code> , <code>getPercentFuelRemaining()</code> vs raw gallons). Used for: letting storage/representation change without breaking clients; raising the API to the problem domain. Applied: expose <code>getX()/getY()</code> not <code>public double x, y;</code>	CleanCode p.61
Data/object anti-symmetry	What: objects hide data + expose behaviour; data structures expose data + have no behaviour — exact opposites that ease opposite changes (objects → new types easy; data structures → new functions easy). Used for: choosing the right shape for the expected axis of change. Applied: pick one form, never a hybrid that exposes fields <i>and</i> has fat methods.	CleanCode p.63
Law of Demeter	What: "talk only to immediate friends" — don't call methods on objects <i>returned by</i> other calls; avoid train wrecks. Used for: limiting coupling so a change to a deep object's structure doesn't ripple to distant callers. Applied: <code>ctxt.createScratchFileStream(name)</code> , not <code>ctxt.getOptions().getScratchDir().getAbsolutePath()</code> .	CleanCode p.64
Special Case pattern	What: return an object that encodes the special/default case so the caller has no branch. Used for: keeping the normal flow free of <code>try/catch</code> or null-checks for routine alternatives. Applied: <code>getMeals()</code> returns a per-diem <code>MealExpenses</code> object instead of throwing <code>MealExpensesNotFound</code> .	CleanCode p.69–70
Three Laws of TDD	What: (1) no production code before a failing test, (2) only enough test to fail, (3) only enough code to pass. Used for: building a near-total regression safety net and forcing testable, decoupled design. Applied: red→green→refactor in seconds-long cycles; write the failing test first, even for bug fixes.	CleanCode p.74

Term	Definition	Source
F.I.R.S.T.	What: Fast, Independent, Repeatable, Self-validating, Timely tests. Used for: each property removes a reason a team would stop running the suite, keeping the safety net trustworthy. Applied: millisecond in-memory tests, order-independent, boolean pass/fail, written just before the code.	CleanCode p.77
SRP (Single Responsibility Principle)	What: a class/module has one, and only one, reason to change. Used for: stopping unrelated changes colliding in one file (a fragility source) and growing systems by adding classes. Applied: split a <code>Manager</code> doing UI + data + persistence into three cohesive classes.	CleanCode p.79; DesignPrinciplesAndPatterns p.4
Rules of Simple Design (Emergence)	What: in priority order — (1) runs all tests, (2) no duplication, (3) expressive, (4) minimal classes/methods. Used for: a complete recipe from which good architecture <i>emerges</i> via small rules. Applied: during refactor, satisfy the four in order; never cut a class (rule 4) if it breaks a test (rule 1).	CleanCode p.80
Rigidity	What: software hard to change because every change cascades through dependents. Symptom of: bad dependencies. Tell: a 2-day estimate becomes a multi-week marathon.	DesignPrinciplesAndPatterns p.2
Fragility	What: software that breaks in many, often <i>unrelated</i> , places when changed. Symptom of: bad dependencies. Tell: fixing module A breaks unrelated module B; breakage probability $\rightarrow 1$.	DesignPrinciplesAndPatterns p.2
Immobility	What: can't reuse a useful module because it drags too much dependency baggage. Symptom of: bad dependencies. Tell: the team rewrites rather than reuses.	DesignPrinciplesAndPatterns p.3
Viscosity	What: the design-preserving change is <i>harder</i> than a hack (viscosity of design) or slow tooling tempts shortcuts (viscosity of environment). Symptom of: bad dependencies + bad environment. Tell: "easy to do the wrong thing, hard to do the right thing."	DesignPrinciplesAndPatterns p.3
OCP (Open/Closed Principle)	What: open for extension, closed for modification. Used for: the chief cure for rigidity/fragility — add features as <i>new</i> code, never edit working code. Applied: route clients through an interface; add a new implementing class per new variant.	DesignPrinciplesAndPatterns p.4
LSP (Liskov Substitution Principle)	What: subtypes must be substitutable for their base; preconditions no stronger, postconditions no weaker ("expect no more, provide no less"). Used for: keeps OCP's polymorphism honest — a rogue subtype forces client edits. Applied: don't subclass if the child can't honour the base's contract; prefer composition.	DesignPrinciplesAndPatterns p.8, p.11
Design by Contract	What: Meyer's formalisation of pre/postconditions that LSP restates. Used for: giving a precise, checkable rule for valid inheritance. Applied: a derived method may weaken preconditions and strengthen postconditions, never the reverse.	DesignPrinciplesAndPatterns p.11
Circle/Ellipse dilemma	What: the classic LSP-violation example — <code>Circle</code> extends <code>Ellipse</code> breaks <code>SetFoci</code> 's contract. Used for: showing is-a in math $\neq$ behavioural substitutability in code. Applied: spotting that an overriding subtype that constrains an inherited operation is an LSP (and latent OCP) violation.	DesignPrinciplesAndPatterns p.9–11
DIP (Dependency Inversion Principle)	What: depend on abstractions, not concretions; abstractions are "hinge points." Used for: the <i>mechanism</i> that achieves OCP — stops volatile detail rippling into stable policy; enables reuse. Applied: inject an <code>OrderRepository</code> interface instead of <code>new MySQLOrderRepository()</code> .	DesignPrinciplesAndPatterns p.12–13
ISP (Interface Segregation Principle)	What: many client-specific interfaces beat one fat interface. Used for: a dependency firewall between clients so one client's change can't force-recompile the others. Applied: split a fat <code>Worker</code> into <code>Workable</code> / <code>Feedable</code> ; let the class implement both.	DesignPrinciplesAndPatterns p.14–16

Term	Definition	Source
REP (Release Reuse Equivalency Principle)	What: the granule of reuse is the granule of release. Used for: grouping reusable classes into versioned, releasable packages (protects reusers). Applied: package classes that will be reused together and version them as a unit.	DesignPrinciplesAndPatterns p.17
CCP (Common Closure Principle)	What: classes that change together belong together (SRP for packages). Used for: confining any one change to as few packages as possible (protects maintainers). Applied: put same-reason-to-change classes in one package.	DesignPrinciplesAndPatterns p.17
CRP (Common Reuse Principle)	What: classes not reused together shouldn't be grouped together. Used for: sparing clients from revalidating against unused classes (a package dependency = dependency on all of it). Applied: don't bundle unrelated classes a client won't use into its package.	DesignPrinciplesAndPatterns p.17–18
ADP (Acyclic Dependencies Principle)	What: package dependencies must form a DAG, no cycles. Used for: keeping packages independently buildable/testable/releasable. Applied: break a cycle by extracting a new shared package or inverting a dependency with DIP+ISP.	DesignPrinciplesAndPatterns p.18
SDP (Stable Dependencies Principle)	What: depend in the direction of stability; instability $I = Ce / (Ca + Ce)$ (0 = stable, 1 = instable). Used for: ensuring change flows "downhill" — volatile code never sits below stable code. Applied: only depend on packages with a lower I than yours.	DesignPrinciplesAndPatterns p.22–24
SAP (Stable Abstractions Principle)	What: stable packages should be abstract; abstractness $A = Na / Nc$. Used for: keeping the frozen, depended-upon core <i>extensible</i> (hard to modify, easy to extend via OCP) — DIP at package scale. Applied: make your stable core mostly interfaces/abstract classes.	DesignPrinciplesAndPatterns p.24–26
Ca / Ce	What: afferent (incoming) / efferent (outgoing) coupling counts for a package. Used for: computing stability $I = Ce / (Ca + Ce)$. Applied: high $Ca \rightarrow$ stable (many depend on it); high $Ce \rightarrow$ instable (depends on many).	DesignPrinciplesAndPatterns p.23–24
Main Sequence	What: the ideal line $A+I=1$ on the A-vs-I graph (abstract-stable or concrete-instable). Used for: a target every package should sit near. Applied: measure each package's A and I ; refactor those far off the line (distance $D' = A+I-1 $).	DesignPrinciplesAndPatterns p.26
Zone of Pain / Zone of Uselessness	What: concrete+heavily-depended-upon (low A, low I) / abstract+unused (high A, high I); both bad. Used for: naming the two corners to design away from. Applied: make a Zone-of-Pain utility more abstract; delete Zone-of-Uselessness dead abstractions.	DesignPrinciplesAndPatterns p.27
Abstract Server (pattern)	What: insert an abstract interface between client and server (DIP "hinge point"). Used for: breaking a client's direct dependency on a server you <i>can</i> change, so implementations swap freely. Applied: <code>Consumer</code> $\rightarrow$ <code>ResourceManager_I</code> $\leftarrow$ <code>ResourceManager</code> ; JHotDraw clients depend on <code>Figure</code> / <code>Tool</code> interfaces.	DesignPrinciplesAndPatterns p.28
Adapter (pattern, GoF structural)	What: an object that implements your abstract interface and delegates/translates to an existing server you <i>can't</i> change. Used for: binding a third-party or over-depended server to your interface. Applied: wrap a legacy API behind your interface; JHotDraw adapts AWT/Swing components.	DesignPrinciplesAndPatterns p.29
Observer (pattern, GoF behavioral)	What: a Subject notifies registered Observers of events without knowing their concrete types. Used for: one-to-many sync with the source decoupled from its consumers (OCP for reactors). Applied: <code>Sensor</code> $\rightarrow$ <code>Meter</code> via <code>Register/Notify/Update</code> ; JHotDraw <code>Figure</code> notifies <code>DrawingView</code> / <code>FigureListener</code> s.	DesignPrinciplesAndPatterns p.29–30

Term	Definition	Source
Bridge (pattern, GoF structural)	What: separate an abstraction from its implementation (by reference, not inheritance) so each varies independently. Used for: avoiding tight inheritance coupling and class explosion across two dimensions. Applied: <code>MusicSynthesizer.PlayMidi</code> bridges to a <code>VoiceEmitter</code> ; JHotDraw separates figure behaviour from structure.	DesignPrinciplesAndPatterns p.30–31
Abstract Factory (pattern, GoF creational)	What: quarantine concrete-class creation knowledge behind a factory interface, in one place. Used for: letting clients create objects without naming concrete types (DIP at construction). Applied: <code>ModemFactory.Make()</code> returns <code>Modem*</code> ; only <code>main</code> knows the concrete factory. JHotDraw creates tools/figures through factories.	DesignPrinciplesAndPatterns p.32
WTFs/minute	What: the cartoon's "only valid measurement of code quality" — the rate of reader-incomprehension events during code review. Used for: defining quality from the <i>reader's</i> seat; even good code emits some WTFs, bad code emits a storm. Applied: treat every WTF moment while reading as a marker for a rename/extract/comment-deletion.	CleanCode p.10
Static factory method	What: a named static creation method preferred over overloaded constructors. Used for: giving construction a descriptive, disambiguating name (constructors must all share the class name). Applied: <code>Complex.fromRealNumber(23.0)</code> "is generally better than" <code>new Complex(23.0)</code> .	CleanCode p.19
Argument object	What: an object that bundles arguments which travel together, shrinking the argument list and naming a hidden concept. Used for: reducing dyads/triads toward the monadic/niladic ideal. Applied: <code>makeCircle(double x, double y, double r) → makeCircle(Point center, double r)</code> .	CleanCode p.31
Common monadic forms	What: the legitimate one-argument shapes — question (<code>fileExists</code>), operate-and-return (<code>fileOpen</code>), transformation (return the transformed value, never mutate an out-param), and event (<code>passwordAttemptFailedNtimes</code>). Used for: predicting a function's behaviour from its signature alone. Applied: prefer <code>StringBuffer transform(StringBuffer in)</code> over <code>void transform(StringBuffer out)</code> .	CleanCode p.28
Dijkstra's rules (structured programming)	What: one entry, one exit per function (no extra <code>return</code> / <code>break</code> / <code>continue</code>). Used for: keeping <i>large</i> functions readable; in clean-code-small functions an occasional multiple return/break/continue "can sometimes even be more expressive." Applied: guard-clause early return in a 5-line function is fine; a <code>goto</code> -like maze in a 200-line one is not.	CleanCode p.33
Roach motel / official rigidity	What: rigid software where "engineers check in, but they don't check out"; when managers react by refusing changes, design deficiency becomes "official rigidity" — adverse management policy. Used for: describing the management spiral rigidity causes. Applied: recognizing that a change-freeze is a <i>symptom</i> of rot, not a cure.	DesignPrinciplesAndPatterns p.2
Dependency firewall	What: a boundary (an abstraction) across which "dependencies do not propagate." Used for: the chapter's name for what all the principles build — barriers stopping change-ripples. Applied: every interface inserted by Abstract Server/Adapter/Observer is a firewall.	DesignPrinciplesAndPatterns p.4
Hinge point	What: the chapter's metaphor for an abstraction — "the places where the design can bend or be extended, without themselves being modified (OCP)." Used for: explaining <i>why</i> depending on abstractions is safe: they're the flex-points. Applied: the <code>Modem</code> interface; the <code>ResourceManager_I</code> of Abstract Server.	DesignPrinciplesAndPatterns p.13, p.28
Penny analogy	What: a penny on its side isn't changing, yet isn't stable — stability is the <i>work required to change</i> , not the frequency of change. Used for: the precise meaning of "stable" behind the SDP. Applied: a package nobody has edited in years can still be <i>unstable</i> (nothing depends on it, trivial to change).	DesignPrinciplesAndPatterns p.22–23

Term	Definition	Source
Responsible / independent; irresponsible / dependent	What: the stability vocabulary — a stable package is <i>responsible</i> (many incoming dependencies) and <i>independent</i> (no outgoing); an instable one is <i>irresponsible</i> (no incoming) and <i>dependent</i> (many outgoing). Used for: reading stability off a dependency diagram without computing I . Applied: Figure 2-25's X (3 in, 0 out) vs Figure 2-26's Y (0 in, 3 out).	DesignPrinciplesAndPatterns p.23
D / D' (distance metrics)	What: distance from the Main Sequence — $D = (A+I-1)/\sqrt{2}$, range [0,~0.707]; normalized $D' = \sqrt{ A+I-1 }$, range [0,1]. Used for: a single number flagging packages that are too concrete-and-depended-upon or too abstract-and-unused. Applied: compute per package; refactor the highest- D' offenders first.	DesignPrinciplesAndPatterns p.27
Abstract class (chapter definition)	What: "a class with at least one pure interface, [which] cannot be instantiated" — the Na numerator of the abstractness metric. Used for: computing $A = Na/Nc$. Applied: count abstract classes/interfaces per package when plotting the A-vs-I graph.	DesignPrinciplesAndPatterns p.26
OO architecture (chapter definition)	What: "the structure of classes and packages that keeps the software application flexible, robust, reusable, and developable." Used for: the chapter's one-sentence definition of its whole subject. Applied: the four adjectives map to the four symptom-cures (flexible ↔ rigidity, robust ↔ fragility, reusable ↔ immobility, developable ↔ viscosity).	DesignPrinciplesAndPatterns p.32

Compare & Contrast Tables

Quick-reference tables for the lecture's most easily-confused item sets. Every cell is grounded in the cited pages.

The four rot symptoms, side by side

Symptom	One-line definition	Diagnostic question	Signature anecdote	Source
Rigidity	Hard to change; every change cascades into dependent modules	"Does a 2-day change become a multi-week marathon?"	Roach motel: engineers check in, don't check out; "official rigidity" when managers ban changes	DesignPrinciplesAndPatterns p.2
Fragility	Breaks in many — often conceptually <i>unrelated</i> — places when changed	"Does fixing A break unrelated B?"	Breakage probability asymptotically approaches 1; every fix introduces more problems than it solves; distrust reigns	DesignPrinciplesAndPatterns p.2–3
Immobility	Useful module can't be reused elsewhere — too much dependency baggage	"Is rewriting cheaper than reusing?"	Separating desirable from undesirable parts judged too much work and risk	DesignPrinciplesAndPatterns p.3
Viscosity	The design-preserving change is harder than the hack (design); slow tooling rewards shortcuts (environment)	"Is it easy to do the wrong thing, hard to do the right thing?"	Long compiles → recompile-avoiding hacks; hours-long check-ins → check-in-minimizing hacks	DesignPrinciplesAndPatterns p.3

The four class principles, side by side

Principle	Statement	Role in the architecture	Canonical example	Violation tell	Source
OCP	Open for extension, closed for modification	The goal of OO architecture	Modem LogOn against the Modem interface	New variant ⇒ editing existing switch/if-else chains	DesignPrinciplesAndPatterns p.4–7
LSP	Subtypes substitutable for their bases	The precondition that keeps OCP's polymorphism honest	Circle/Ellipse and SetFoci	Client adds typeid / instanceof checks to defend itself	DesignPrinciplesAndPatterns p.8–12
DIP	Depend on abstractions, never concretions	The mechanism that achieves OCP	Policy → interface ← implementation (Figure 2-18)	High-level module new s or names a concrete class	DesignPrinciplesAndPatterns p.12–14
ISP	Many client-specific interfaces beat one fat one	The firewall between clients of one service	ServiceA/B/C interfaces multiply implemented by Service	Changing one client's methods forces others to recompile/redeploy	DesignPrinciplesAndPatterns p.14–16

The six package principles, side by side

Principle	Statement	Protects	Pulls package size	Source
REP	Granule of reuse = granule of release	Reusers (version consumers)	toward releasable, versioned units	DesignPrinciplesAndPatterns p.17
CCP	Classes that change together belong together	Maintainers (release workload)	larger	DesignPrinciplesAndPatterns p.17
CRP	Classes not reused together don't belong together	Reusers (revalidation cost)	smaller	DesignPrinciplesAndPatterns p.17–18
ADP	No cycles in the package dependency graph	Release/test independence	— (graph shape, not size)	DesignPrinciplesAndPatterns p.18–22
SDP	Depend in the direction of stability (lower I)	Volatile packages' changeability	— (dependency direction)	DesignPrinciplesAndPatterns p.22–24
SAP	Stable packages should be abstract	Stable packages' extensibility	— (package content)	DesignPrinciplesAndPatterns p.24–26

The five patterns, side by side

Pattern	GoF family	Problem trigger	Key participants	Principle served	Source
Abstract Server	(interface-programming idiom)	Client depends directly on a server you can change	Client, interface, concrete server	DIP/OCP	DesignPrinciplesAndPatterns p.28
Adapter	Structural	The server is third-party or too heavily depended-upon to change	Target interface, Adapter (translates+delegates), Adaptee	DIP where you can't touch the server	DesignPrinciplesAndPatterns p.29
Observer	Behavioral	Detector must trigger actor without knowing it	Subject (Register/Notify), Observer (Update), Sensor, Meter	OCP/DIP for event flows	DesignPrinciplesAndPatterns p.29–30

Pattern	GoF family	Problem trigger	Key participants	Principle served	Source
Bridge	Structural	Inheritance welds abstraction to implementation; both must vary	Abstraction (PlayMidi side), Implementor interface (VoiceEmitter side)	DIP applied <i>twice</i> , decoupling two hierarchies	DesignPrinciplesAndPatterns p.30–32
Abstract Factory	Creational	Creation must name a concrete class (unavoidable DIP breach)	AbstractFactory, ConcreteFactory (quarantine), products, main	DIP at construction time	DesignPrinciplesAndPatterns p.32–33

Micro ↔ macro rule pairs (same idea, two altitudes)

Clean Code rule (micro)	Design principle (macro)	The shared idea
Replace switch with polymorphism (CleanCode p.25–26)	OCP via dynamic polymorphism (DesignPrinciplesAndPatterns p.5–7)	New variants as new code, not edits
Classes small, one reason to change — SRP (CleanCode p.79)	CCP, "SRP for packages" (DesignPrinciplesAndPatterns p.17)	One reason to change per unit
Law of Demeter — don't talk to strangers (CleanCode p.64)	Dependency firewalls (DesignPrinciplesAndPatterns p.4)	Limit who may know whose internals
Data abstraction — hide representation (CleanCode p.61–62)	DIP — depend on abstractions (DesignPrinciplesAndPatterns p.12)	Clients see contracts, not implementations
One word per concept / solution-domain names (CleanCode p.20–21)	Patterns as shared vocabulary (DesignPrinciplesAndPatterns p.28)	Names that transmit whole designs
F.I.R.S.T. tests as safety net (CleanCode p.74–77)	"If you don't change working code, you won't break it" (DesignPrinciplesAndPatterns p.7)	Two complementary defences against regression

Quote Bank — exact slide/chapter wording for memorization

Short, verbatim, citation-ready lines. (Ellipses mark trimmed words; bracketed letters fix the source's typos.)

From CleanCode : - "You should always leave the code cleaner than you found it." (p.3) - "The only valid measurement of code quality: WTFs/minute." (p.10) - "Clean code is simple and direct. Clean code reads like well-written prose." — Grady Booch (p.5) - "Clean code always looks like it was written by someone who cares." — Michael Feathers (p.7) - "You know you are working on clean code when each routine you read turns out to be pretty much what you expected." — Ward Cunningham (p.9) - "FUNCTIONS SHOULD DO ONE THING. THEY SHOULD DO IT WELL. THEY SHOULD DO IT ONLY." (p.23) - "The ideal number of arguments for a function is zero." (p.27) - "Duplication may be the root of all evil in software." (p.33) - "Don't comment bad code, rewrite it!" (p.34) - "There is nothing quite so helpful and satisfying as a well-described public API." (p.37) - "Short functions don't need much description." (p.51) - "Every programmer has his own favorite formatting rules, but if he works in a team, then the team rules." (p.60) - "You may not write more of a unit test than is sufficient to fail, and not compiling is failing." (p.74) - "Test code is just as important as production code." (p.75) - "What makes a clean test? Three things: readability, readability, and readability." (p.75) - "A class or module should have one, and only one, reason to change." (p.79)

From DesignPrinciplesAndPatterns : - "The software starts to rot. ... [U]gly festering sores and boils accumulate until they dominate the design." (p.1–2) - "Engineers check in, but they don't check out." (p.2) - "It is easy to do the wrong thing, but hard to do the right thing." (p.3) - "If our designs are failing due to the

constant rain of changing requirements, it is our designs that are at fault." (p.4) - "A module should be open for extension but closed for modification." (p.4) - "If you don't have to change working code, you aren't likely to break it." (p.7) - "Derived methods should expect no more and provide no less." (p.11) - "Violations of LSP are latent violations of OCP." (p.12) - "Depend upon Abstractions. Do not depend upon concretions." (p.12) - "Concrete things change a lot, abstract things change much less frequently. ... [A]bstractions are 'hinge points'." (p.13) - "Non-volatility is not a replacement for the substitutability of an abstract interface." (p.14) - "The granule of reuse is the granule of release." (p.17) - "Classes that change together, belong together." (p.17) - "A dependency upon a package is a dependency upon everything within the package." (p.17) - "These three principles are mutually exclusive. They cannot simultaneously be satisfied." (p.18) - "Interfaces are very often included in the package that uses them, rather than in the package that implements them." (p.22) - "Depend upon packages whose I metric is lower than yours." (p.24) - "Stable packages should be abstract packages." (p.24) - "I should increase as A decreases." (p.26) - "The essential definition of a design pattern is a well[-]worn and known good solution to a common problem." (p.28) - "Every method of the adapter simply translates and then delegates." (p.29) - "It has been said that a little knowledge is a dangerous thing, and this chapter has provided a little knowledge." (p.33)

Common Pitfalls / Gotchas

- **Treating clean code as "style/aesthetics."** It's about **maintenance cost**: the deck's whole macro narrative is that messy dependencies cause rigidity/fragility/immobility/viscosity (DesignPrinciplesAndPatterns p.2–3). Clean code shrinks the impact set; that's the exam-worthy *why*.
- **Confusing "comments are good" with "more comments are good."** The deck spends ~18 slides on **bad** comments vs. ~3 on good ones (CleanCode p.34–52). The default position is "explain yourself in code"; a comment is a small failure. Know the bad-comment taxonomy names — they're easy MCQ fodder.
- **Mixing up the four rotting-design symptoms.** Rigidity = hard to change (cascades); Fragility = breaks unexpectedly when changed; Immobility = can't reuse; Viscosity = the hack is easier than the right way (DesignPrinciplesAndPatterns p.2–3). Don't swap rigidity and fragility.
- **Getting the LSP contract direction backwards.** Subtype **preconditions no stronger, postconditions no weaker** ("expect no more, provide no less") (DesignPrinciplesAndPatterns p.11). A common wrong answer flips these.
- **Thinking Circle-is-an-Ellipse is fine because it's true in math.** Inheritance models *behavioral substitutability*, not is-a taxonomy; the contract breaks (DesignPrinciplesAndPatterns p.9–11). LSP violations are also **latent OCP violations** (p.12) — a frequently-tested subtlety.
- **Believing "stable" means "good" and "unstable" means "bad."** Instability is *desirable* for packages you want to keep changeable; the SDP just says don't let a stable package depend on an unstable one (DesignPrinciplesAndPatterns p.24). And **SAP** says stable should also be **abstract** (so it's extendable despite being hard to modify).
- **Forgetting the stability/abstractness formulas.** $I = Ce / (Ca + Ce)$ (instability) and $A = Na / Nc$ (abstractness); the Main Sequence is $A + I = 1$; Zone of Pain = low-A/low-I, Zone of Uselessness = high-A/high-I (DesignPrinciplesAndPatterns p.23–27). These are the most "quantitative" things on the exam.
- **The three package-cohesion principles are in tension and cannot all be satisfied at once** (DesignPrinciplesAndPatterns p.18). A wrong answer claims you can maximize REP, CCP, and CRP simultaneously.
- **Confusing Abstract Server, Adapter, and Bridge.** Abstract Server = insert an interface when you *can* change the server; Adapter = when you *can't* (third-party/over-depended); Bridge = separate an abstraction from its implementation so *both* vary independently (DesignPrinciplesAndPatterns p.28–31).

- **null discipline:** "don't return null" and "don't pass null" are *two separate rules* (CleanCode p.71–73). Returning an empty collection $\neq$ returning null.
- **Command/Query Separation vs. CQRS:** the deck's rule is the small one — a function does *or* answers, not both (CleanCode p.32) — not the architectural CQRS pattern.
- **Switch statements aren't always wrong:** one switch, hidden inside a factory that creates polymorphic objects, is acceptable; *scattered* switches on a type code are the smell (CleanCode p.26; DesignPrinciplesAndPatterns p.6).
- **Stability $\neq$ "hasn't changed lately."** The penny analogy: a penny on its side isn't changing but isn't stable — stability is the *work needed to change*, dominated by incoming dependencies (DesignPrinciplesAndPatterns p.22–23). An exam answer defining stable as "rarely modified" is wrong by the chapter's definition.
- **Ca vs Ce direction.** Ca = afferent = *incoming* (outside classes that depend on the package); Ce = efferent = *outgoing* (outside classes the package depends on). Since $I = Ce / (Ca + Ce)$, a package with **no outgoing** dependencies has $I = 0$ (stable); with **no incoming**, $I = 1$ (instable) (DesignPrinciplesAndPatterns p.23–24). Swapping Ca/Ce flips every answer.
- **D vs D' ranges.** $D = (A + I - 1) / \sqrt{2}$ ranges $[0, \sim 0.707]$; the *normalized* $D' = |A + I - 1|$ ranges $[0, 1]$ (DesignPrinciplesAndPatterns p.27). If an exam option says "distance metric ranges 0–1," that's D' , not D .
- **Interface placement under DIP+ISP cycle-breaking is counterintuitive:** the new interface goes **in the package of the class that uses it**, not with its implementor (DesignPrinciplesAndPatterns p.21–22). Most students guess the opposite.
- **"Not compiling is failing"** — TDD's second law counts a non-compiling test as a failing test, so you may switch to production code as soon as the test references something that doesn't exist (CleanCode p.74). A trick question may claim you must first make the test compile.
- **One assert vs one concept:** the *ideal* is one assert per test, but the operative rule is *minimize asserts and test one concept per test function* — several asserts on a single concept are acceptable (CleanCode p.76).
- **The function-size numbers:** < 150 characters per line and < 20 lines per function (CleanCode p.23) — concrete figures the deck states and exams love; don't confuse with the *class* rule, which is measured in **responsibilities**, not lines (CleanCode p.79).
- **assertEquals(expected, actual) is the deck's example of a problematic dyad** (no natural argument order, routinely transposed), and **assertEquals(message, expected, actual)** its triad-to-avoid; the keyword fix is **assertExpectedEqualsActual** (CleanCode p.30–31). Don't cite it as a *good* API.
- **ISP doesn't mean one interface per client object** — clients are grouped *by type*, and a method needed by two client types is deliberately duplicated into both interfaces ("neither harmful nor confusing," DesignPrinciplesAndPatterns p.15). Over-segregation ("hundreds of interfaces ... segregated by client and version") is its own failure mode (p.16).
- **The deck's GoF citation key is [GOF96]** (DesignPrinciplesAndPatterns p.1, p.28, p.33) — the Gang of Four book referenced throughout this guide as [GHJV94] (its publication-year key). Same book; don't be thrown if the exam uses either key.
- **The Bridge wiring is easy to garble:** in Figure 2-35, the abstract **PlayMidi** is implemented by **MusicSynthesizer_I**, and **EmitVoice** (implemented in **MusicSynthesizer**) *delegates to the* **VoiceEmitter** interface, implemented by **VoiceEmitter_I** (DesignPrinciplesAndPatterns p.31–32). The two suffix-**_I** classes implement *different* things — one the abstraction's operation, one the implementor interface.
- **Observer is pull-style in this deck:** **Update()** carries no payload; the **Meter** calls **GetValue()** back on the **Sensor** after being notified (DesignPrinciplesAndPatterns p.29–30). An answer where the Subject

pushes the new value in `Update(value)` describes a variant, not the deck's figure.

- **Local optimizations of switch chains are worse than the chains:** `if (type == ernie) ... else SendHayes(...)` silently lumps every other (and future) type into the `else` (DesignPrinciplesAndPatterns p.6) — the structure-hiding form fails *invisibly* when a type is added.

Exam Focus

High-yield items, ranked by how likely they are to be tested:

1. **The four rotting-design symptoms** — name + one-line definition each (rigidity, fragility, immobility, viscosity), and that they all stem from **bad dependencies** (DesignPrinciplesAndPatterns p.2–4). Classic short-answer.
2. **The SOLID-style principles by their deck names: OCP, LSP, DIP, ISP** — state each in one sentence and give its mechanism (OCP=abstraction, LSP=substitutability/DbC, DIP=depend on abstractions/"hinge points", ISP=client-specific interfaces) (DesignPrinciplesAndPatterns p.4–16). The deck also implicitly covers **SRP** (CleanCode p.79). Note: the deck predates the "SOLID" acronym but covers all five.
3. **OCP via the modem LogOn example** and the equivalence to "replace switch with polymorphism" (DesignPrinciplesAndPatterns p.5–7; CleanCode p.25–26). Be ready to refactor a switch.
4. **LSP Circle/Ellipse + Design-by-Contract** (preconditions no stronger, postconditions no weaker; LSP violations are latent OCP violations) (DesignPrinciplesAndPatterns p.9–12).
5. **The five named patterns** — intent + structure of Abstract Server, Adapter, Observer, Bridge, Abstract Factory, and which principle each serves (DesignPrinciplesAndPatterns p.28–32). Observer's Subject/Observer/Notify/Update dynamics are especially exam-friendly.
6. **The package metrics** — $I = Ce / (Ca + Ce)$, $A = Na / Nc$, the Main Sequence $A + I = 1$, Zone of Pain / Zone of Uselessness, and the three cohesion (REP/CCP/CRP) + three coupling (ADP/SDP/SAP) principles, including the cohesion tension (DesignPrinciplesAndPatterns p.16–27).
7. **Clean Code function rules** — small, one thing, one level of abstraction, argument count, flag args, Command/Query Separation, DRY (CleanCode p.23–33).
8. **Meaningful Names rules** — the full list (intention-revealing → no gratuitous context), especially "avoid encodings" (no Hungarian, no member prefixes) (CleanCode p.11–22).
9. **The bad-comment taxonomy** + "don't comment bad code, rewrite it" + "explain yourself in code" (CleanCode p.34–52).
10. **Three Laws of TDD** and **F.I.R.S.T.** (CleanCode p.74, p.77); **Rules of Simple Design / Emergence** in priority order (CleanCode p.80); **Law of Demeter** & train wrecks (CleanCode p.64); **don't return/pass null** (CleanCode p.71–73).
11. **The cross-cutting tie-in** (likely an essay prompt): explain *why* clean code and OO principles reduce maintenance cost in terms of the change process — smaller estimated impact set, cheaper Impact Analysis, easier Verification, and JHotDraw as the GoF-pattern case study.
12. **Quote attribution** — match the six clean-code definitions to their authors: Stroustrup (elegance / "does one thing well"), Booch (simple, direct, prose), Dave Thomas (literate), Feathers (care), Jeffries (reduced duplication / expressiveness / simple abstractions), Cunningham ("what you expected") (CleanCode p.4–9) — plus the WTFs/minute cartoon as "the only valid measurement of code quality" (CleanCode p.10).
13. **Metric mechanics under time pressure** — given a small dependency diagram, compute Ca , Ce , $I = Ce / (Ca + Ce)$, $A = Na / Nc$, and $D' = |A + I - 1|$, then classify the package (Main Sequence / Zone of Pain / Zone of Uselessness) and name the fix (raise A , shed dependants, or delete the unused abstraction)

(DesignPrinciplesAndPatterns p.23–27). Remember the vocabulary: responsible/independent (stable) vs irresponsible/dependent (instable) (p.23).

14. **Code-to-category recognition** — be ready to see a snippet and name the violated rule: `genymdhms` (pronounceable, CleanCode p.13), `m_dsc` (member prefixes, p.15), `phoneString` (Hungarian, p.16), `render(true)` (flag argument, p.29), `if (set(...))` (Command/Query Separation, p.32), `} //while` (closing-brace comment, p.46), `/* Added by Rick */` (attribution, p.47), `ctxt.getOptions().getScratchDir().getAbsolutePath()` (Law of Demeter, p.64), `if (employees != null)` (don't return null, p.71–72), `typeid(e) == typeid(Ellipse)` (LSP-violation patch, itself an OCP violation, DesignPrinciplesAndPatterns p.11–12).
15. **The two cycle-breaking techniques** — extract a shared package (`MessageManager`) vs invert one dependency with DIP+ISP (interface `BY` placed in the *user's* package), and the consequence story (Protocol's test-build set exploding from one package to six) (DesignPrinciplesAndPatterns p.19–22).

How Lecture 6 Hooks into the Eight-Step Change Process

Grounding note: the phase mapping is **inference/connective tissue** — neither deck names the course's eight steps — but every cell's *content* is cited to the decks. This is the table to rehearse for the essay-style "why does clean code lower maintenance cost?" prompt.

Change - process phase	What this lecture contributes	Deck grounding
Initiation	Requirements <i>will</i> keep changing — "the requirements document is the most volatile document in the project" — so accepting change requests is normal, and a design that can't absorb them is the defect	DesignPrinciplesAndPatterns p.3–4
Concept Location	Intention-revealing, searchable, pronounceable names make concepts greppable and discussable (<code>WORK_DAYS_PER_WEEK</code> vs a bare <code>5</code>); the newspaper metaphor and Stepdown Rule make files skimmable top-down; solution-domain names (<code>AccountVisitor</code>) tell a programmer instantly which pattern they are inside	CleanCode p.11–14, p.21, p.24, p.53
Impact Analysis	The estimated impact set is exactly what the macro principles bound: Demeter violations and train wrecks widen it (p.64); duplicated switches widen it (one per copy); cycles widen the <i>build/test</i> set "by an abhorrent amount"; SDP direction tells you which way change-pressure flows	CleanCode p.33, p.64; DesignPrinciplesAndPatterns p.6, p.20, p.24
Prefactoring	Make the change easy before making it: replace the scattered switch with polymorphism (OCP), insert an Abstract Server/Adapter hinge point, break the cycle (<code>MessageManager</code> or <code>BY</code> interface) so the change lands in one package	CleanCode p.25–26; DesignPrinciplesAndPatterns p.5–7, p.21–22, p.28–29
Actualization	With OCP achieved, the change is <i>only new code</i> — a new <code>EmployeeType</code> subclass, a new <code>Modem</code> implementation — "if you don't have to change working code, you aren't likely to break it"	DesignPrinciplesAndPatterns p.7; CleanCode p.26
Postfactoring	The Boy Scout Rule institutionalized: delete bad comments, fix encodings and train wrecks you passed through, re-run the four Simple Design Rules in priority order	CleanCode p.3, p.38–52, p.80
Conclusion	Package hygiene at release time: CCP keeps the change confined to few packages; REP/CRP keep reusers' revalidation cost down; ADP keeps the release buildable in isolation	DesignPrinciplesAndPatterns p.17–20

Change - process phase	What this lecture contributes	Deck grounding
Verifica tion	The Three Laws guarantee near-total coverage written <i>before</i> the code; F.I.R.S.T. keeps the suite fast and trusted; one-concept tests make failures diagnostic; clean tests survive long enough to protect the next change	CleanCode p.74–77

The compounding claim across all eight rows: every principle in this lecture is a **cost-of-change** lever. Rot (rigidity, fragility, immobility, viscosity) is what unmanaged dependencies do to those levers over time (DesignPrinciplesAndPatterns p.2–4); clean code and the OO principles are the maintenance discipline that keeps each phase O(change) instead of O(system).

Mnemonics & Memory Hooks

Compact retention aids; each hook's content is cited, the mnemonic itself is study scaffolding.

- **R-F-I-V** — the rot symptoms in deck order: **R**igidity, **F**ragility, **I**mmobility, **V**iscosity ("Rotten Fruit Is Viscous") (DesignPrinciplesAndPatterns p.2–3).
- **Goal–Gate–Engine–Fence** — the four class principles by role: OCP is the *goal*, LSP the *gate* (what subtypes must pass), DIP the *engine* (how the goal is achieved), ISP the *fence* (between client types) (DesignPrinciplesAndPatterns p.4–16, esp. "the OCP states the goal ... the DIP states the primary mechanism," p.12).
- **"Expect no more, provide no less"** — LSP's contract clause in five words: preconditions no stronger (expect no more), postconditions no weaker (provide no less) (DesignPrinciplesAndPatterns p.11).
- **Cohesion trio vs coupling trio** — REP/CCP/CRP say *what goes in* a package; ADP/SDP/SAP say *how packages may point at each other* (DesignPrinciplesAndPatterns p.16–26). The cohesion trio fights itself (tension, p.18); the coupling trio stacks: DAG (ADP) → arrows point at lower **I** (SDP) → low-**I** packages are abstract (SAP).
- **I is "outgoing over everything"** — Instability $I = Ce / (Ca + Ce)$: the *e* in **Ce** = *e*fferent = *e*xiting (outgoing). All arrows out, none in $\Rightarrow I = 1 \Rightarrow$ free to change (DesignPrinciplesAndPatterns p.23–24).
- **Main Sequence: A + I = 1** — abstractness and instability are a *budget that sums to one*: every unit of incoming responsibility (low I) must be paid for with a unit of abstractness (high A) (DesignPrinciplesAndPatterns p.26–27). Pain corner = broke (0,0 — everyone leans on concrete you); Useless corner = hoarding (1,1 — abstraction nobody uses).
- **Pattern picker in one breath**: *can change the server* → Abstract Server; *can't* → Adapter; *detector mustn't know actor* → Observer; *two things vary independently* → Bridge; *must create without naming* → Abstract Factory (DesignPrinciplesAndPatterns p.28–32).
- **Function-rule ladder, descending**: small (<20 lines) → one thing → one abstraction level → descriptive name → fewest arguments (0>1>2>3) → no flags → command *or* query → DRY (CleanCode p.23–33) — each rule is the previous one applied to a finer grain.
- **Comment triage**: *why?* → maybe keep (intent, warning, legal, clarification, amplification, public-API Javadoc — the six good kinds plus TODO, CleanCode p.35–37); *what?* → rewrite the code instead; *who/when?* → delete, Git knows (attributions, journals, p.42, p.47).

Self-Test Question Bank (with answers)

Closed-book drill questions built strictly from the two decks. Answer first, then check.

- Q1.** State the deck's two reasons for caring about clean code, and the Boy Scout Rule. **A.** "1. You are a programmer. 2. You want to be a better programmer" (CleanCode p.2); "You should always leave the code cleaner than you found it" (CleanCode p.3).
- Q2.** Who defined clean code as reading "like well-written prose," and who defined it by "what you expected"? **A.** Grady Booch (CleanCode p.5); Ward Cunningham (CleanCode p.9).
- Q3.** What, per the cartoon, is "the only valid measurement of code quality" — and what is the subtle second lesson of the drawing? **A.** WTFs/minute, measured in code review (CleanCode p.10); even the *good*-code door emits some WTFs — good and bad differ in rate, not in total absence of surprises.
- Q4.** A teammate names a map of accounts `accountList` and a class `DataManager`. Which two naming rules are violated? **A.** `accountList` is disinformation — the name implies a `List` it isn't (CleanCode p.12); `DataManager` violates the class-name rule — vague noise nouns like `Manager` / `Data` name no responsibility (CleanCode p.18).
- Q5.** Give the deck's numeric limits for functions, and the unit in which *class* size is measured. **A.** < 150 characters per line, < 20 lines per function (CleanCode p.23); classes are measured in **responsibilities** (CleanCode p.79).
- Q6.** Rank function arities from best to tolerated, and name the two refactorings for an over-long argument list. **A.** Niladic (ideal: zero) > monadic > dyadic > triadic (avoid) (CleanCode p.27, p.30); introduce an **argument object** (`makeCircle(Point center, double radius)`) or encode meaning in the name via **verbs and keywords** (`assertExpectedEqualsActual`) (CleanCode p.31).
- Q7.** Why is `if (set("username", "unclebob"))` bad, and what is the fix? **A.** `set` is simultaneously a command and a query — in an `if` you can't tell verb from adjective; apply Command/Query Separation: `if (attributeExists("username")) { setAttribute("username", "unclebob"); }` (CleanCode p.32).
- Q8.** Name the comment smell: (a) `} //while;` (b) `/* Added by Rick */;` (c) a field `info` documented as `/** The version. */;` (d) an RFC excerpt in a Javadoc. **A.** (a) closing-brace comment (CleanCode p.46); (b) attribution/byline (p.47); (c) scary noise — copy-pasted noise that became wrong (p.44); (d) too much information (p.50).
- Q9.** What are the deck's three rules of a clean *test*, and the five F.I.R.S.T. properties? **A.** "Readability, readability, and readability" (CleanCode p.75); Fast, Independent, Repeatable, Self-validating, Timely (p.77).
- Q10.** Recite TDD's second law, including its crucial clause. **A.** "You may not write more of a unit test than is sufficient to fail — **and not compiling is failing**" (CleanCode p.74).
- Q11.** Give the standard class-organization order from the deck. **A.** Public static constants → private static variables → private instance variables → public functions, with each private utility placed right after the public function that calls it (CleanCode p.78).
- Q12.** The four Rules of Simple Design, in priority order? **A.** 1. Runs all the tests; 2. No duplication; 3. Expressive; 4. Minimal classes and methods (CleanCode p.80).

Q13. Define rigidity and fragility so they cannot be confused. **A.** Rigidity: change is *hard* because each change cascades through dependents (2-day fix → multi-week marathon). Fragility: change *breaks* the system in many, often conceptually unrelated, places (probability of breakage → 1) (DesignPrinciplesAndPatterns p.2).

Q14. What single root cause do all four rot symptoms share, and what is the chapter's name for the cure? **A.** Improper dependencies between modules; the cure is dependency management via **dependency firewalls** across which dependencies do not propagate (DesignPrinciplesAndPatterns p.4).

Q15. Why does Listing 2-1's design force recompiling *every* modem when one is added? **A.** Every modem struct embeds the shared `Modem::Type` enumeration; adding an enumerator changes the type all of them depend on (DesignPrinciplesAndPatterns p.5).

Q16. State the OCP and the DIP, and their relationship. **A.** OCP: "a module should be open for extension but closed for modification" (p.4); DIP: "depend upon abstractions; do not depend upon concretions" (p.12); "if the OCP states the **goal** of OO architecture, the DIP states the primary **mechanism**" (p.12) — abstractions are the hinge points that let modules extend without modification (p.13).

Q17. Give the Design-by-Contract formulation of LSP, and the slogan form. **A.** A derived class is substitutable iff its preconditions are **no stronger** and its postconditions are **no weaker** than the base method's; "derived methods should expect no more and provide no less" (DesignPrinciplesAndPatterns p.11).

Q18. Why is the `typeid` patch in Listing 2-6 doubly damned? **A.** It is the symptom of an LSP violation (a client defending against a rogue subtype) *and* is itself an OCP violation — every new `Ellipse` derivative forces this function to be re-checked; "violations of LSP are latent violations of OCP" (DesignPrinciplesAndPatterns p.11–12). Worse, the exact-type check rejects even future well-behaved subtypes.

Q19. When is depending on something concrete acceptable under the DIP, and what is the limit of that excuse? **A.** When the concretion is stable/non-volatile (e.g. `string.h`) — but "non-volatility is not a replacement for the substitutability of an abstract interface": a stable concretion still can't be swapped (e.g. the forced move to UNICODE) (DesignPrinciplesAndPatterns p.14).

Q20. Your service interface serves three client types and one method is needed by two of them. What does ISP say? **A.** Create one interface per client *type* (not per client object) and add the shared method to **both** interfaces — "neither harmful nor confusing"; the concrete service multiply implements all of them (DesignPrinciplesAndPatterns p.15–16).

Q21. Name the three package-cohesion principles and who each protects; why can't all three be maximized? **A.** REP (granule of reuse = granule of release) and CRP (don't group what isn't reused together) protect *reusers*; CCP (classes that change together belong together) protects *maintainers*. CCP pushes packages large, CRP small — "mutually exclusive ... cannot simultaneously be satisfied"; structure shifts CCP→REP/CRP as the project matures (DesignPrinciplesAndPatterns p.17–18).

Q22. The CommError engineer adds one call into the GUI. List the packages the Protocol team must now build with, and the two ways to fix the cycle. **A.** CommError, GUI, Comm, ModemControl, Analysis, Database (p.20). Fix 1: extract the needed classes into a new package (`MessageManager`) both depend on (p.21). Fix 2: DIP+ISP — invert one dependency by introducing interface `BY`, placed in the *using* class's package (p.21–22).

Q23. Compute I for a package with $C_a = 3$, $C_e = 1$, and say whether a package with $I = 0.9$ may depend on it. **A.** $I = 1/(3+1) = 0.25$. Yes: SDP requires depending on packages whose I is *lower* than yours, and $0.25 < 0.9$ (DesignPrinciplesAndPatterns p.24).

Q24. Define both corners to avoid on the A-vs-I graph and the formula that measures distance from the Main Sequence. **A.** Zone of Pain: concrete and heavily depended-upon ($A \approx 0$, $I \approx 0$) — can't be extended, painful to change; Zone of Uselessness: abstract with no dependants ($A \approx 1$, $I \approx 1$); normalized distance $D' = |A + I - 1|$, 0 on the line, 1 maximally off it (DesignPrinciplesAndPatterns p.26–27).

Q25. In the deck's Observer, what does `Update()` carry, and how does the Meter get the new reading? **A.** Nothing — it is a pure notification; the Meter *pulls* by calling `GetValue()` on the Sensor and then displays the value (DesignPrinciplesAndPatterns p.29–30).

Q26. In the Abstract Factory figure, which module knows the concrete modem classes, and which module knows *that* module? **A.** Only `ModemFactory_I` knows Hayes / Courier / Ernie; only `main` knows `ModemFactory_I` (it creates it and loads the `GtheFactory` global) (DesignPrinciplesAndPatterns p.32).

Q27. Choose the pattern: (a) you must use a vendor SDK class but want your own interface; (b) a `Meter` must track a `Sensor` without the sensor knowing; (c) `PlayMidi` and `EmitVoice` must vary independently; (d) clients must create objects without naming concrete classes; (e) you own both client and server and just need a swap point. **A.** (a) Adapter (p.29); (b) Observer (p.29–30); (c) Bridge (p.30–32); (d) Abstract Factory (p.32); (e) Abstract Server (p.28).

Q28. Give the chapter's four-adjective definition of OO architecture. **A.** "The structure of classes and packages that keeps the software application **flexible, robust, reusable, and developable**" (DesignPrinciplesAndPatterns p.32).

Q29. `Date genymdhms;` — name the violated rule and write the fix. **A.** Use pronounceable names (CleanCode p.13); `Date generationTimestamp;`.

Q30. Which of the four common monadic forms does `void transform(StringBuffer out)` get wrong, and why? **A.** The transformation form: a transforming function should return the transformed value (`StringBuffer transform(StringBuffer in)`), not mutate through an output argument — the call site otherwise hides the mutation (CleanCode p.28).

Q31. What's the difference between the deck's two `Vehicle` interfaces, given both use the `interface` keyword? **A.** `getFuelTankCapacityInGallons()` / `getGallonsOfGasoline()` is *concrete in spirit* — a window onto the stored data and its units — while `getPercentFuelRemaining()` abstracts both the unit and the arithmetic; abstraction is about what the API reveals, not the keyword (CleanCode p.62).

Q32. Identify every smell: `final String outputDir = ctxt.options.scratchDir.absolutePath;` **A.** The worst Law-of-Demeter form — reaching through *exposed fields* of two foreign objects; also data-structure/object hybridization (objects exposing fields) and a coupling that breaks when `Options` or `ScratchDir` changes shape (CleanCode p.63–64).

Q33. Why does the deck call abandoned dirty tests *worse* than no tests? **A.** You lose the safety net (no regression detection) but keep paying to maintain the rotting test code — cost without protection; hence "test code is just as important as production code" (CleanCode p.75).

Q34. Dynamic vs static polymorphism as OCP techniques — name the deck's example of each and one trade-off. **A.** Dynamic: the `Modem` pure-virtual interface (Listing 2-2); static: the `template <typename MODEM> LogOn` (Listing 2-3). Static binds at compile time — no virtual dispatch and no common base needed, but no run-time substitution (DesignPrinciplesAndPatterns p.6–7).

Q35. Why does *viscosity of environment* count as a design symptom when it's about tooling? **A.** Because slow builds/check-ins *change engineer behaviour*: they pick changes that avoid recompiles or minimize check-ins

"regardless of whether the design is preserved" — tooling friction converts directly into design decay (DesignPrinciplesAndPatterns p.3).

Q36. An old client holds a `Service*` and a new capability was added per the ISP's "changing interfaces" advice. Show how a client reaches the new methods. **A.** The capability ships as a *new* interface; clients query for it: `if (NewService* ns = dynamic_cast<NewService*>(s)) { /* use the new service interface */ }` — old clients and the old interface stay untouched (DesignPrinciplesAndPatterns p.15–16).

Q37. State REP's argument in terms of what a reuser demands of an author. **A.** Reusers refuse to depend on an element unless it is released and versioned — the author must "keep track of version numbers, and maintain old versions for awhile" — so only released packages are reusable: the granule of reuse is the granule of release (DesignPrinciplesAndPatterns p.17).

Q38. Why does the deck place a figure's `Notify` on `Subject` and `Check` on `Sensor` rather than merging them? **A.** Separation of generic from specific: `Subject` owns the generic observer bookkeeping (Register/Notify over the `Observer*` list) while `Sensor` owns the domain event detection (`Check`, `GetValue`); the `Sensor` inherits `Subject` and calls `Notify` when its reading changes (DesignPrinciplesAndPatterns p.29–30).

Q39. In the Bridge solution, which class implements `PlayMidi`, and which implements the voice output? **A.** `MusicSynthesizer_I` implements the abstract `PlayMidi`; the `VoiceEmitter` interface's `EmitVoice` is implemented by `VoiceEmitter_I` — two parallel hierarchies joined by delegation, so each can vary "any number of different ways" (DesignPrinciplesAndPatterns p.31–32).

Q40. Your package has `Nc = 10`, `Na = 9`, `Ca = 0`, `Ce = 0` — wait, `I` is undefined; assume `Ce = 1`, `Ca = 0`. Where does it sit and what is it called? **A.** `A = 0.9`, `I = 1/(0+1) = 1.0`, `D' = |0.9 + 1.0 - 1| = 0.9` — deep in the **Zone of Uselessness**: maximally abstract with no dependants; the deck's prescription is that such unused abstraction is dead weight to remove (DesignPrinciplesAndPatterns p.26–27).

Q41. Quote the deck's exact wording for why one should not comment bad code, and the positive counterpart rule. **A.** "Don't comment bad code — rewrite it!" (CleanCode p.34); the counterpart is *explain yourself in code*: `if (employee.isEligibleForFullBenefits())` replaces the comment-plus-cryptic-condition (CleanCode p.34).

Q42. Name all six *good*-comment categories the deck illustrates with code, plus the two more it names. **A.** Legal, informative, explanation of intent, clarification, amplification, Javadocs in public APIs (CleanCode p.35–37) — plus warning of consequences and TODO comments (p.37).

Q43. What does the deck say happens to the package structure over a project's life, and why is that *healthy*? **A.** It "jitters and shifts": new packages appear and classes migrate — to break cycles (ADP) and to move from CCP-dominant grouping early to REP/CRP-dominant grouping as the architecture stabilizes; the structure is supposed to be refactored, not fixed in stone (DesignPrinciplesAndPatterns p.18, p.21).

Q44. Why must the *caller* sit above the *callee* in a source file? **A.** Vertical distance/dependent functions: reading top-down then follows the call flow — the newspaper metaphor's high-level-to-detail descent applied to function placement (CleanCode p.53, p.55).

Q45. The deck shows `int realDaysPerIdealDay = 4;` as a plain variable but `WORK_DAYS_PER_WEEK = 5` as a `const`. What rule do both serve, and what additional benefit does naming give beyond searchability? **A.** Use searchable names (CleanCode p.14); beyond making the values greppable, the names *document the domain meaning* of the former magic numbers (a 4:1 real-to-ideal day ratio; a 5-day work week), so the formula `taskEstimate[j] * realDaysPerIdealDay / WORK_DAYS_PER_WEEK` explains itself.

Q46. Why does the deck consider `assertEquals(expected, actual)` "problematic" but `new Point(0,0)` "perfectly reasonable," when both are dyads? **A.** A dyad is acceptable when its two arguments are *naturally ordered components of one concept* (x then y of a point); `expected / actual` have no natural order, so callers routinely transpose them (CleanCode p.30). The fix encodes the order in the name: `assertExpectedEqualsActual` (p.31).

Q47. What two separate costs does the deck attach to the `m_dsc` example, and which third smell does its setter add? **A.** The `m_` member prefix (noise the eye must skip; IDEs already scope members) and the unpronounceable abbreviation `dsc` requiring an explanatory comment (CleanCode p.15, p.13); the setter `setName` is also dishonest — it sets the *description* — fixed by `setDescription`.

Q48. Per the deck, when may a function legitimately contain a `try` keyword, and where? **A.** Only as "the very first word in the function," with "nothing after the catch/finally blocks" — because error handling is one thing, and a function that handles errors should do nothing else (CleanCode p.68).

Q49. Contrast what CCP minimizes with what CRP minimizes. **A.** CCP minimizes the number of *packages touched per change/release* (maintainer's rebuild-test-deploy work, DesignPrinciplesAndPatterns p.17); CRP minimizes *clients' exposure to irrelevant change* — revalidation forced by version bumps of classes they never used (p.17–18).

Q50. Why does the SAP describe the Main Sequence as "conforming to the DIP"? **A.** A package on the line is "abstract in proportion to its incoming dependencies and concrete in proportion to its outgoing dependencies" (DesignPrinciplesAndPatterns p.27) — i.e. whatever is depended upon is abstract, and whatever depends outward is concrete detail: exactly "depend upon abstractions, do not depend upon concretions" aggregated to package granularity (p.12).

Source Map

Topic	Deck & pages	Reading key
Two reasons; Boy Scout Rule; elegance ideals	CleanCode p.2–9	[Martin]
Meaningful Names (all rules)	CleanCode p.11–22	[Martin]
Functions (small, one thing, abstraction, switch, args, flags, CQS, DRY, structured programming)	CleanCode p.23–33	[Martin]
Comments — good	CleanCode p.34–37	[Martin]
Comments — bad (full taxonomy)	CleanCode p.38–52	[Martin]
Formatting (purpose, newspaper, vertical/horizontal, team rules)	CleanCode p.53–60	[Martin]
Objects & Data Structures (abstraction, anti-symmetry, Law of Demeter)	CleanCode p.61–64	[Martin]
Error Handling (exceptions, extract try/catch, normal flow, no null)	CleanCode p.65–73	[Martin]
Unit Tests (Three Laws of TDD, clean tests, one assert/concept, F.I.R.S.T.)	CleanCode p.74–77	[Martin]
Classes (organization, small, SRP, cohesion)	CleanCode p.78–79	[Martin]
Emergence (four Rules of Simple Design)	CleanCode p.80	[Martin] (Beck's rules)
Clean Code book reference	CleanCode p.81	[MC09]
Symptoms of rotting design	DesignPrinciplesAndPatterns p.1–4	[Martin]

Topic	Deck & pages	Reading key
Dependency management / firewalls	DesignPrinciplesAndPatterns p.4	[Martin]
OCP	DesignPrinciplesAndPatterns p.4–7	[OCP97], [GHJV94]
LSP / Design by Contract / Circle-Ellipse	DesignPrinciplesAndPatterns p.8–12	[LSP97], [Liskov88]
DIP	DesignPrinciplesAndPatterns p.12–14	[DIP97]
ISP	DesignPrinciplesAndPatterns p.14–16	[ISP97]
Package cohesion (REP/CCP/CRP) + tension	DesignPrinciplesAndPatterns p.16–18	[Granularity97]
Package coupling (ADP/SDP/SAP) + metrics + Main Sequence	DesignPrinciplesAndPatterns p.18–27	[Stability97]
Design patterns — definition	DesignPrinciplesAndPatterns p.28	[GHJV94]
Abstract Server	DesignPrinciplesAndPatterns p.28	[GHJV94]
Adapter	DesignPrinciplesAndPatterns p.29	[GHJV94]
Observer	DesignPrinciplesAndPatterns p.29–30	[GHJV94]
Bridge	DesignPrinciplesAndPatterns p.30–31	[GHJV94]
Abstract Factory	DesignPrinciplesAndPatterns p.32	[GHJV94]
Conclusion & bibliography	DesignPrinciplesAndPatterns p.32–34	[Martin], [GHJV94], [Martin99]
JHotDraw as GoF case study	— (inference)	[Raj13], [GHJV94]
Change-process framing (8 steps)	— (inference)	[Raj13]
Title slide (course lecturer's deck)	CleanCode p.1	[Martin]
WTFs/minute code-quality cartoon	CleanCode p.10	[Martin]
Clean-code definitions by named authority (Stroustrup, Booch, Thomas, Feathers, Jeffries, Cunningham)	CleanCode p.4–9	[Martin]
Architecture as multitiered; chapter scope	DesignPrinciplesAndPatterns p.1	[Shaw96], [Martin99]
Why redesigns rarely succeed (moving target)	DesignPrinciplesAndPatterns p.2	[Martin]
Changing Requirements ("designs at fault")	DesignPrinciplesAndPatterns p.3–4	[Martin]
OCP modem listings 2-1/2-2/2-3; local-optimization trap	DesignPrinciplesAndPatterns p.5–7	[OCP97]

Topic	Deck & pages	Reading key
LSP listings 2-4/2-5/2-6; Eiffel & explicit contracts; NotAnEllipse	DesignPrinciplesAndPatterns p.8–12	[LSP97], [OOSC98]
DIP figures 2-17/2-18; COM/CORBA/EJB; string.h & UNICODE caution	DesignPrinciplesAndPatterns p.12–14	[DIP97]
ISP figures 2-19/2-20; client types; changing interfaces via dynamic_cast	DesignPrinciplesAndPatterns p.15–16	[ISP97]
ADP example system; CommError cycle; MessageManager & BY-interface fixes; interface-placement rule	DesignPrinciplesAndPatterns p.19–22	[Granularity97]
Penny analogy; responsible/independent vocabulary; Flexible/Stable violation	DesignPrinciplesAndPatterns p.22–24	[Stability97]
Full metric set incl. D and D'; metric imperfection caveat	DesignPrinciplesAndPatterns p.23–27	[Stability97]
Pattern figures 2-30 ... 2-36 (participants & dynamics)	DesignPrinciplesAndPatterns p.28–33	[GOF96]
OO-architecture definition (flexible, robust, reusable, developable); "little knowledge" close	DesignPrinciplesAndPatterns p.32–33	[Martin]